

HỒ SƠ BẢO VỆ VÀ PHẢN BIỆN

SƠ ĐỒ THỰC THỂ – LIÊN KẾT

HỆ THỐNG JAPANO

Tài liệu chuẩn bị cho tình huống hội đồng chỉ ngẫu nhiên vào một bảng, một cột hoặc một đường nối bất kỳ trên sơ đồ và yêu cầu giải thích.

Phạm vi phân tích	36 thực thể logic · 50 quan hệ · 35 collection vật lý
Nguồn đối chiếu	erd.drawio · MongoDB japono đang chạy · mã nguồn backend
Ngày lập	18/08/2026
Cách dùng	Đọc Phần 0 trước. Phần 20–21 dùng để luyện chỉ ngẫu nhiên và ôn nhanh.

MỤC LỤC

PHẦN 0. CÁCH DÙNG TÀI LIỆU VÀ CÔNG THỨC TRẢ LỜI	6
0.1. Ba nhân dùng xuyên suốt	6
0.2. Năm công thức trả lời	6
0.3. Cách viết câu trả lời	7
PHẦN 1. PHƯƠNG PHÁP ĐỐI CHIẾU VÀ KẾT QUẢ	8
1.1. Vì sao 36 thực thể nhưng chỉ 35 collection	8
1.2. Bốn tầng dữ liệu	8
1.3. Phân hạng bảng theo mức độ dễ bị hỏi	9
1.4. Hình dạng của sơ đồ	10
PHẦN 2. HỒ SƠ BẢO VỆ CÁC BẢNG HẠNG S	11
2.1. Bảng users	12
2.2. Bảng products và product_variants	16
2.3. Bảng orders và order_items	19
2.4. Bảng payments	22
PHẦN 3. HỒ SƠ BẢO VỆ CÁC BẢNG HẠNG A	24
3.1. Bảng reviews	24
3.2. Bảng wishlist_items và cart_items	25
3.3. Bảng vouchers và voucher_redemptions	25
3.4. Bảng return_requests	26
3.5. Bảng addresses	26
PHẦN 4. PHÂN TÍCH ĐƯỜNG NỐI	27
4.1. Nguyên tắc chung: khoá ngoại nằm ở phía nào	27
4.2. users (1) → (N) orders	27
4.3. orders (1) → (N) order_items (N) → (1) products	27
4.4. orders (1) → (N) payments	28
4.5. products (1) → (N) product_variants (N) → (1) Màu Sắc / Kích Thước	28
4.6. Bảng tra nhanh toàn bộ quan hệ	29
PHẦN 5. DỮ LIỆU CHỤP LẠI VÀ CHUẨN HOÁ	33
5.1. Câu hỏi khó nhất về chuẩn hoá	33
5.2. Toàn bộ dữ liệu được chụp lại trong hệ thống	33
5.3. Bốn loại bất thường và cách hệ thống tránh	34
PHẦN 6. ĐỒNG THỜI, GIAO DỊCH VÀ TÍNH NGUYÊN TỬ	35
6.1. Bài toán hai người cùng mua sản phẩm cuối cùng	35
6.2. Hệ thống có dùng giao dịch không	35

6.3. Nếu một bước trong luồng đặt hàng thất bại	36
PHẦN 7. TOÀN VẬN THAM CHIẾU VÀ CHIẾN LƯỢC XOÁ	38
7.1. MongoDB không có khoá ngoại — vậy 50 quan hệ là gì	38
7.2. Sản phẩm bị xoá thì đơn hàng cũ ra sao	38
7.3. Bảng chiến lược xoá cho từng quan hệ	39
PHẦN 8. RÀNG BUỘC DUY NHẤT VÀ CHỈ MỤC	40
8.1. Toàn bộ ràng buộc duy nhất thật trong hệ thống	40
8.2. Vì sao cơ sở dữ liệu vẫn cần ràng buộc dù ứng dụng đã kiểm tra	40
8.3. Chỉ mục và truy vấn tương ứng	41
PHẦN 9. BẢO MẬT Ở TẦNG DỮ LIỆU	42
PHẦN 10. NẾU HỆ THỐNG CÓ MỘT TRIỆU NGƯỜI DÙNG	43
PHẦN 11. NẾU GIẢNG VIÊN ĐỐI YÊU CẦU TẠI CHỖ	44
PHẦN 12. ĐIỂM YẾU THẬT — KHÔNG BẢO VỆ MÙ QUÁNG	45
PHẦN 13. KHI GIẢNG VIÊN CỐ TÌNH BẮT BÊ	46
PHẦN 14. BUỔI PHẢN BIỆN MÔ PHÒNG	48
14.1. Mô phỏng — hội đồng chỉ vào users	49
14.2. Mô phỏng — hội đồng chỉ vào products	50
14.3. Mô phỏng — hội đồng chỉ vào orders	51
14.4. Mô phỏng — hội đồng chỉ vào payments	53
14.5. Mô phỏng — hội đồng chỉ vào một bảng khó: voucher_redemptions	55
14.6. Mô phỏng — kiến trúc database, consistency và scale	56
PHẦN 15. TỰ CHẤM NHƯ MỘT GIẢNG VIÊN PHẢN BIỆN	57
PHẦN 16. NĂM MƯƠI CÂU HỎI THEO CẤP ĐỘ	58
Cấp 1 — Nhận biết	58
Cấp 2 — Giải thích	58
Cấp 3 — Tình huống thực tế	59
Cấp 4 — Phản biện	59
Cấp 5 — Khó	60
PHẦN 17. TRANG TRA NHANH TRƯỚC KHI VÀO PHÒNG	61
17.1. Nếu hội đồng chỉ vào bảng này thì nói câu này trước	61
17.2. Mười con số phải nhớ	61
17.3. Ba câu cứu nguy khi bí	62
PHẦN 18. DEFENSE CARD CHO TOÀN BỘ 36 THỰC THỂ	63
18.1. Danh Mục Sản Phẩm	63
18.2. Sản Phẩm	63
18.3. Chi Tiết Sản Phẩm	64
18.4. Biến Thể Sản Phẩm	64

18.5. Hình Ảnh	65
18.6. Mô Tả Sản Phẩm Tạo Bởi AI	65
18.7. Đơn Hàng	66
18.8. Chi Tiết Đơn Hàng	66
18.9. Thanh Toán	67
18.10. Yêu Cầu Trả Hàng	67
18.11. Người Dùng	68
18.12. Địa Chỉ	69
18.13. Hồ Sơ Người Dùng	69
18.14. Cấu Hình Cửa Hàng	70
18.15. Thông Báo	70
18.16. Chi Tiết Giỏ Hàng	71
18.17. Danh Sách Sản Phẩm Yêu Thích	71
18.18. Mục Tiêu Tiết Kiệm	72
18.19. Lịch Sử Tìm Kiếm	72
18.20. Tin Nhắn	73
18.21. Tương Tác Người Dùng	73
18.22. Lịch Sử Thử Đồ	74
18.23. Phiếu Giảm Giá	74
18.24. Lướt Sử Dụng Phiếu	75
18.25. Quy Tắc Giảm Giá	75
18.26. Thành Viên VIP	76
18.27. Đánh Giá Sản Phẩm	76
18.28. Tương Tác Đánh Giá	77
18.29. Mẫu Kiểm Duyệt	77
18.30. Đánh Giá Địa Điểm Nhật Bản	78
18.31. Hình Ảnh Giao Diện Sản Phẩm	78
18.32. Thẻ Địa Danh	79
18.33. Bộ Sưu Tầm Thẻ Của Người Dùng	79
18.34. Quảng Cáo	80
18.35. Màu Sắc	80
18.36. Kích Thước	80
PHẦN 19. WHY TREE CHO SÁU THỰC THỂ HẠNG S	82
19.1. WHY TREE — users	82
19.2. WHY TREE — products	82
19.3. WHY TREE — product_variants	82
19.4. WHY TREE — orders	82

19.5. WHY TREE — order_items	83
19.6. WHY TREE — payments	83
PHẦN 20. GIẢNG VIÊN CHỈ NGẪU NHIÊN	84
PHẦN 21. NĂM CHEAT SHEET CUỐI	87
21.1. SHEET 1 — ALL TABLES	87
21.2. SHEET 2 — RELATIONSHIPS	89
21.3. SHEET 3 — TOP 25 QUESTIONS	92
21.4. SHEET 4 — PROJECT WEAKNESSES	93
21.5. SHEET 5 — CÂU CỨU NGUY	94

PHẦN 0. CÁCH DÙNG TÀI LIỆU VÀ CÔNG THỨC TRẢ LỜI

Tài liệu này không mô tả cơ sở dữ liệu. Nó chuẩn bị cho một tình huống cụ thể: hội đồng chiếu sơ đồ lên màn hình, chỉ ngẫu nhiên vào một chỗ, và hỏi liên tiếp cho tới khi sinh viên hết lý lẽ. Vì vậy mọi mục đều viết theo hướng *tại sao*, không phải là *gì*.

0.1. Ba nhãn dùng xuyên suốt

Mỗi khẳng định trong tài liệu thuộc đúng một trong ba loại. Khi bảo vệ, tuyệt đối không trình bày nhãn thứ ba như thể nó đã có trong hệ thống.

Bảng 0.1: Ba mức độ khẳng định

Nhãn	Nghĩa	Cách nói trước hội đồng
[THỰC TẾ]	Xác nhận được bằng mã nguồn, cấu trúc collection hoặc dữ liệu đang chạy.	"Hiện tại hệ thống đang..." — nói chắc chắn.
[SUY LUẬN]	Suy ra hợp lý từ mã nguồn nhưng chưa có kiểm chứng trực tiếp.	"Theo cách cài đặt hiện tại thì..." — nói có mức độ.
[ĐỀ XUẤT]	Thiết kế nên cải tiến, chưa tồn tại trong hệ thống.	"Nếu làm tiếp, em sẽ..." — phải nói rõ là chưa làm.

NGUYÊN TẮC QUAN TRỌNG NHẤT

Khi thiết kế hiện tại có điểm yếu thật, **đừng cố bảo vệ**. Hội đồng chấm khả năng nhận ra và lập luận, không chấm việc hệ thống hoàn hảo. Mẫu câu an toàn: *"Ở phiên bản hiện tại nhóm triển khai theo cách A. Khi rà soát lại chúng em nhận thấy cách A có hạn chế ở tình huống B. Thiết kế tốt hơn là C, và đây là lý do"*.
Tài liệu này đánh dấu sẵn những chỗ **không nên bảo vệ** — xem Phần 14.

0.2. Năm công thức trả lời

Học thuộc năm khung này. Khi bị hỏi bất ngờ, chỉ cần xác định câu hỏi thuộc loại nào rồi chạy theo khung, sẽ không bao giờ bị đứng hình.

Bảng 0.2: Năm khung trả lời phản biện

Bị hỏi về	Khung trả lời theo thứ tự
Một BẢNG	THỰC THỂ → MỤC ĐÍCH → PHÂN LOẠI DỮ LIỆU → KHOÁ CHÍNH → CÁC QUAN HỆ → TÌNH HUỐNG THỰC TẾ
Một ĐƯỜNG NỐI	NGHIỆP VỤ → BẢN SỐ → KHOÁ NGOẠI NÀO → ĐẶT Ở PHÍA NÀO → VÌ SAO PHÍA ĐÓ → TOÀN VỆN → CHIẾN LƯỢC XOÁ
Một CỘT	Ý NGHĨA THỰC TẾ → VÌ SAO PHẢI LƯU → AI GHI VÀO → RÀNG BUỘC → CÓ ĐỔI THEO THỜI GIAN KHÔNG → RỦI RO NẾU SAI
"Tại sao không làm cách	PHƯƠNG ÁN HIỆN TẠI → PHƯƠNG ÁN KIA → ĐÁNH ĐỔI CỦA TỪNG

khác?"	BÊN → BỐI CẢNH ĐỒ ÁN → KẾT LUẬN
Tình huống thực tế	SỰ KIỆN → BẢNG NÀO THAM GIA → BẢN GHI NÀO ĐỐI → RÀNG BUỘC NÀO BẢO VỆ → NẾU HỎNG GIỮA CHỪNG → CÁCH KHÔI PHỤC

0.3. Cách viết câu trả lời

Câu trả lời chung chung là thứ hội đồng phát hiện nhanh nhất. So sánh hai câu cùng nói về một ràng buộc:

TRẢ LỜI YẾU — SẼ BỊ HỎI TIẾP NGAY

"Ràng buộc duy nhất giúp tối ưu hoá hệ thống và đảm bảo tính toàn vẹn dữ liệu."

TRẢ LỜI TỐT — NÊU NGUYÊN NHÂN VÀ HẬU QUẢ CỤ THỂ

"Nếu không có ràng buộc duy nhất trên cặp `userId` và `productId`, hai yêu cầu gửi lên cùng lúc đều có thể vượt qua bước kiểm tra ở tầng ứng dụng và tạo ra hai bản ghi yêu thích giống hệt nhau. Kiểm tra ở ứng dụng không đủ vì giữa lúc đọc và lúc ghi có một khoảng trống. Ràng buộc ở cơ sở dữ liệu là lớp bảo vệ cuối cùng."

Luôn đi theo **nguyên nhân** → **hậu quả**, hoặc **quy tắc nghiệp vụ** → **quyết định thiết kế**.

PHẦN 1. PHƯƠNG PHÁP ĐỐI CHIẾU VÀ KẾT QUẢ

Trước khi lập tài liệu, ba nguồn được đối chiếu chéo với nhau. Đây là căn cứ cho mọi con số phía sau, và cũng là câu trả lời nếu hội đồng hỏi "*em lấy thông tin này ở đâu*".

Bảng 1.1: Nguồn dữ liệu và phạm vi đối chiếu

Nguồn	Đọc được gì	Kết quả
erd.drawio	Thực thể, cột, khoá chính, khoá ngoại, đường nối	36 thực thể · 50 quan hệ · 407 dòng cột
MongoDB japono	Collection thật, tên trường, kiểu, tỉ lệ có mặt, chỉ mục	35 collection · 21 collection có chỉ mục ngoài _id
backend/routes/	Ai được ghi, kiểm tra gì trước khi ghi	17 tệp định tuyến
backend/lib/store.js	Cách một lượt ghi diễn ra, tính nguyên tử	Sửa đổi đồng bộ, một luồng
lib/mongoCollections.js	Hàm kiểm tra toàn vẹn tham chiếu	relationshipErrors()

1.1. Vì sao 36 thực thể nhưng chỉ 35 collection

Đây là câu hỏi gần như chắc chắn bị hỏi, và có câu trả lời chính xác. Chênh lệch đi theo hai chiều ngược nhau chứ không phải thiếu một bảng.

Bảng 1.2: Đối chiếu 36 thực thể logic với 35 collection vật lý

Chiều lệch	Tên	Lý do
Có trên sơ đồ, không thành collection	Màu Sắc	Nhúng thẳng vào product_variants dưới dạng colorName và colorHex. Không có màn hình nào cần liệt kê toàn bộ màu độc lập.
Có trên sơ đồ, không thành collection	Kích Thước	Nhúng vào product_variants dưới dạng size. Tập giá trị đóng và hiếm thay đổi.
Có trên sơ đồ, không thành collection	Hình Ảnh	Trùng vai trò với Hình Ảnh Giao Diện Sản Phẩm. Chỉ collection product_media tồn tại. Đây là lỗi thiết kế còn sót — xem Phần 14.
Có collection, không lên sơ đồ	japan_spot_suggestions	Hàng chờ duyệt nội dung do người dùng đóng góp. Thuần vận hành.
Có collection, không lên sơ đồ	push_tokens	Mã thiết bị để gửi thông báo đẩy. Thuần kỹ thuật, không thuộc mô hình nghiệp vụ.

Phép tính: $36 - 3 + 2 = 35$. [THỰC TẾ]

1.2. Bốn tầng dữ liệu

Cách phân loại này giúp trả lời nhanh câu "*bảng này thuộc loại dữ liệu gì*" — một câu hỏi rất hay được dùng để mở đầu.

Bảng 1.3: Phân tầng dữ liệu trong hệ thống

Tầng	Đặc điểm	Có được sửa/xoá không	Bảng tiêu biểu trong JAPANO
Dữ liệu gốc (Master)	Mô tả thực thể tồn tại độc lập, ít thay đổi	Được sửa. Xoá phải cân nhắc.	users products categories vouchers
Dữ liệu giao dịch (Transaction)	Ghi lại một sự kiện đã xảy ra tại một thời điểm	Không xoá thật. Sửa phải để lại vết.	orders payments return_requests
Dữ liệu chi tiết (Detail)	Không tồn tại nếu thiếu bản ghi cha	Sống chết theo cha.	order_items product_variants product_media
Dữ liệu bắc cầu (Mapping)	Chỉ tồn tại để nối hai bảng khác	Xoá thoải mái, không mất thông tin gốc.	wishlist_items cart_items flagcard_collections
Dữ liệu nhật ký (Log)	Ghi hành vi để phân tích, không phải chứng từ	Được dọn theo thời gian.	interactions search_logs tryon_history

VÌ SAO PHÂN BIỆT NÀY QUAN TRỌNG KHI BỊ HỎI

Câu "nếu xoá thì sao" có đáp án khác hẳn nhau tùy tầng. Xoá một bản ghi bắc cầu (bỏ yêu thích) là chuyện bình thường. Xoá một bản ghi giao dịch (đơn hàng đã thanh toán) là **phá huỷ chứng từ** — sai về kế toán, sai về đối soát với cổng thanh toán, và làm mất khả năng truy vết khi có khiếu nại.

1.3. Phân hạng bảng theo mức độ dễ bị hỏi

Thời gian ôn có hạn nên phải phân bổ đúng chỗ. Hạng S là các bảng nằm ở trung tâm sơ đồ, hút nhiều đường nối nhất, và cũng là nơi nghiệp vụ phức tạp nhất.

Bảng 1.4: Phân hạng bảng theo xác suất bị hỏi

Hạng	Bảng	Vì sao dễ bị hỏi
S — rất dễ hỏi	users products product_variants orders order_items payments	Nhận nhiều đường nối nhất. Chứa toàn bộ câu hỏi kinh điển: khoá chính, ảnh chụp giá, tồn kho, đồng thời, giao dịch.
A — dễ hỏi	reviews cart_items wishlist_items return_requests vouchers voucher_redemptions categories addresses	Là nơi kiểm tra hiểu biết về bảng bắc cầu, ràng buộc duy nhất và quy tắc nghiệp vụ.
B — có thể hỏi	profiles vip_memberships discount_rules notifications product_media product_details flagcards flagcard_collections goals japan_spot_reviews review_reactions	Hỗ trợ nghiệp vụ. Thường chỉ bị hỏi một hoặc hai câu về lý do tách bảng.
C — ít hỏi	interactions search_logs tryon_history chats ai_descriptions moderation_samples banners settings push_tokens japan_spot_suggestions	Nhật ký và cấu hình. Câu hỏi thường xoay quanh dung lượng và vòng đời dữ liệu.

1.4. Hình dạng của sơ đồ

Nhận xét mở đầu tốt nhất khi đứng trước sơ đồ là nói về **mật độ đường nối**, vì nó cho thấy ngay bản chất hệ thống mà không cần đọc bảng nào.

Bảng 1.5: Các bảng nhận nhiều quan hệ nhất

Bảng	Số quan hệ nhận vào	Ý nghĩa
Người Dùng	21	Trung tâm thứ nhất. Gần như mọi hành vi đều gắn với một người.
Sản Phẩm	14	Trung tâm thứ hai. Gần như mọi hành vi đều hướng tới một món hàng.
Đơn Hàng	5	Trung tâm cấp hai: nơi hai trung tâm trên gặp nhau thành chứng từ.
Phiếu Giảm Giá	2	Nhận từ đơn hàng và từ lượt sử dụng.
Đánh Giá Sản Phẩm	2	Nhận từ tương tác đánh giá và mẫu kiểm duyệt.

Hai bảng đầu chiếm **35 trên tổng số 50 quan hệ**. [THỰC TẾ] Câu mở đầu nên là: "*Toàn bộ hệ thống quy về việc một người làm một điều gì đó với một sản phẩm; phần còn lại của sơ đồ là các cách khác nhau để ghi lại việc đó.*"

PHẦN 2. HỒ SƠ BẢO VỆ CÁC BẢNG HẠNG S

Sáu bảng trong phần này chiếm phần lớn thời gian phản biện. Mỗi bảng được trình bày theo đúng khung ở mục 0.2 để khi bị hỏi chỉ cần chạy tuần tự.

2.1. Bảng `users`

2.1.1. Bảng này đại diện cho cái gì

Trong thực tế: một con người có thể đăng nhập vào hệ thống — khách mua hàng, nhân viên xử lý đơn, hoặc quản trị viên. **Trong phần mềm:** một danh tính có thể xác thực và được phân quyền. **Trong cơ sở dữ liệu:** một bản ghi giữ đúng những gì cần để trả lời hai câu hỏi — *người này có phải chính họ không* và *người này được làm gì*.

Điểm cần nhấn mạnh: bảng users **cố ý không giữ hồ sơ cá nhân**. Ảnh đại diện, giới tính, ngày sinh nằm ở profiles. Lý do ở mục 2.1.4.

2.1.2. Nếu bỏ bảng này thì hỏng gì

- Không đăng nhập được — không còn nơi lưu email và passwordHash.
- Không phân quyền được — 4 vai trò customer < staff < admin < super_admin mất chỗ lưu.
- Toàn bộ 21 quan hệ trở về users mất đích, tức là **21 trên 50 đường nối** trên sơ đồ sụp.
- Không quy được đơn hàng, thanh toán, đánh giá về ai — hệ thống không còn là thương mại điện tử.

Phân loại: **dữ liệu gốc (Master Data)**. Tồn tại độc lập, không phụ thuộc bảng nào khác. Trên sơ đồ, users không có mũi tên nào đi ra.

2.1.3. Phân tích các cột quan trọng

Bảng 2.1: Cấu trúc thật của collection `users` (36 bản ghi)

Cột	Kiểu thật	Có mặt	Vai trò
id	chuỗi	100%	Khoá chính. Dạng u1, u-<mốc thời gian>-<ngẫu nhiên>.
email	chuỗi	100%	Định danh đăng nhập.
passwordHash	chuỗi	97%	Băm bcrypt. 3% thiếu là tài khoản đăng nhập bằng Google.
role	chuỗi	100%	customer staff admin super_admin
status	chuỗi	100%	active locked
joinedAt	số	100%	Mốc thời gian tạo tài khoản.
googleId	chuỗi	3%	Định danh từ nhà cung cấp bên ngoài.
authProviders	mảng	31%	Các cách đăng nhập đã liên kết.
stripeCustomerId	chuỗi	6%	Mã khách trên cổng thanh toán, để lưu thẻ.
resetCodeHash	chuỗi	3%	Băm mã đặt lại mật khẩu, có hạn dùng.

Cột `email` — phân tích sâu

Câu hỏi	Trả lời
---------	---------

Ý nghĩa thực tế	Địa chỉ thư điện tử dùng để đăng nhập và nhận thông báo hệ thống.
Vì sao phải lưu	Là thứ người dùng nhớ được. Không ai nhớ id của mình.
Bỏ được không	Không. Mất luôn khả năng đăng nhập và khôi phục mật khẩu.
Suy ra từ dữ liệu khác được không	Không. Đây là dữ liệu do người dùng cung cấp.
NULL hay NOT NULL	NOT NULL. Thực tế 100% bản ghi đều có. [THỰC TẾ]
Có cần duy nhất không	Có — hai tài khoản cùng email thì đăng nhập không xác định được là ai.
Thực tế có duy nhất không	Chỉ ở tầng ứng dụng. Chỉ mục trong MongoDB là {email: 1} KHÔNG unique. Xem Phần 14, mục Y-01.
Có chỉ mục không	Có, chỉ mục thường trên email. [THỰC TẾ]
Có đổi theo thời gian không	Có thể đổi. Đây chính là lý do không dùng email làm khoá chính.

Cột `passwordHash` — phân tích sâu

Tên cột đã nói rõ nó lưu **giá trị băm**, không lưu mật khẩu. Thuật toán là bcrypt, có muối ngẫu nhiên nhúng sẵn trong chuỗi kết quả nên không cần cột salt riêng. [THỰC TẾ] Giá trị thật có dạng \$2b\$10\$..., trong đó 10 là số vòng lặp.

- **Vì sao không lưu mật khẩu gốc:** nếu cơ sở dữ liệu bị lộ, toàn bộ tài khoản mất ngay, và người dùng thường dùng lại mật khẩu ở nơi khác nên thiệt hại lan sang hệ thống khác.
- **Vì sao bcrypt chứ không phải MD5 hay SHA-256:** hai hàm kia thiết kế để chạy *nhANH*, đúng thứ kẻ tấn công cần khi dò hàng loạt. bcrypt cố tình chậm và chỉnh được độ chậm.
- **Cột này không bao giờ được trả về qua giao diện lập trình.** Có 3% bản ghi không có cột này — đó là tài khoản đăng nhập bằng Google, chưa từng đặt mật khẩu. [THỰC TẾ]

Cột `role` — phân tích sâu

Hiện lưu trực tiếp một chuỗi trong users, không tách bảng roles riêng. [THỰC TẾ] Đây là quyết định có thể bị chất vấn nên cần chuẩn bị kỹ.

GV: Sao không tách bảng roles cho chuẩn?

Vì hệ thống hiện tại có đúng **bốn vai trò xếp thứ bậc**: customer nhỏ hơn staff, nhỏ hơn admin, nhỏ hơn super_admin. Tập giá trị đóng, và **một người chỉ có một vai trò**.

Tách bảng roles chỉ có ích khi một người mang nhiều vai trò cùng lúc, hoặc khi quyền cần cấu hình lại lúc chạy mà không sửa mã. Cả hai điều đó hiện đều không có trong yêu cầu, nên tách bảng sẽ thêm một phép nối vào **mọi** lượt kiểm tra quyền mà không đổi được kết quả nào.

GIẢNG VIÊN SẼ TRUY TIẾP

Nếu giảng viên nói *"nhưng sau này cần thì sao"* — trả lời: khi đó cần bảng roles và bảng bắc cầu user_roles với khoá chính ghép (userId, roleId). Việc chuyển đổi là một lần chuyển dữ liệu, không phải viết lại hệ thống. [ĐỀ XUẤT]

2.1.4. Khoá chính — vì sao là `id`

GV: Email đã là duy nhất rồi, sao không lấy luôn email làm khoá chính?

Vì email là **thuộc tính nghiệp vụ**, còn khoá chính cần là **định danh kỹ thuật**. Ba lý do cụ thể:

Một là email đổi được. Người dùng đổi email thì khoá chính đổi theo, kéo theo phải cập nhật khoá ngoại ở 21 nơi đang trỏ về users. Chỉ cần một chỗ cập nhật sót là dữ liệu mờ côi.

Hai là khoá chính bị đem đi khắp nơi. orders.userId, payments.userId, reviews.userId... đều mang giá trị khoá chính. Nếu đó là email thì địa chỉ thư của khách nằm rải rác trong hàng chục collection — vừa tốn chỗ, vừa là vấn đề riêng tư khi cần xoá dữ liệu cá nhân.

Ba là tách danh tính khỏi dữ liệu nghiệp vụ. Khoá chính nên vô nghĩa để không bao giờ phải đổi vì lý do nghiệp vụ.

GIẢNG VIÊN SẼ TRUY TIẾP

Giảng viên có thể hỏi ngược: "*Thêm cột `id` là dư thừa dữ liệu?*" → Không. Dư thừa là khi **cùng một sự thật** được lưu hai chỗ. id và email là hai sự thật khác nhau: một là định danh nội bộ, một là địa chỉ liên lạc. Chi phí thêm vài byte đổi lấy việc không bao giờ phải cập nhật dây chuyền.

Vì sao chuỗi chứ không phải số tự tăng: MongoDB không có cơ chế tự tăng dựng sẵn, và khoá dạng chuỗi cho phép sinh mã ở phía ứng dụng mà không cần hỏi cơ sở dữ liệu — cần thiết khi muốn chạy nhiều bản sao máy chủ. Đánh đổi là khoá dài hơn và không sắp xếp theo thứ tự tạo được. [THỰC TẾ]

2.1.5. Tình huống thực tế với `users`

GV: Người dùng xoá tài khoản thì đơn hàng cũ ra sao?

Trước hết em xin nói rõ hiện trạng: **hệ thống hiện không có chức năng xoá tài khoản**. Không có điểm cuối nào cho phép xoá một bản ghi users. [THỰC TẾ] Cái có là cột status với hai giá trị active và locked — tức là **khoá**, không phải xoá.

Và đó là lựa chọn đúng cho một hệ thống thương mại điện tử. Nếu xoá thật bản ghi người dùng thì: đơn hàng cũ mất chủ, không đối soát được với cổng thanh toán; hoá đơn mất thông tin người mua, sai về kế toán; và nếu có tranh chấp thì không còn gì để truy vết.

Cách xử lý đúng khi người dùng yêu cầu xoá dữ liệu cá nhân là **ẩn danh hoá** chứ không phải xoá: giữ nguyên bản ghi và khoá chính, nhưng thay tên, email, số điện thoại bằng giá trị vô danh. Đơn hàng vẫn nổi được, mà dữ liệu cá nhân thì không còn. Đây là phần hệ thống chưa làm. [ĐỀ XUẤT]

GIẢNG VIÊN SẼ TRUY TIẾP

Câu này đang kiểm tra cùng lúc sáu thứ: khoá ngoại, chiến lược xoá, lưu trữ dữ liệu, kiểm toán, lịch sử nghiệp vụ và quyền riêng tư. Trả lời được đầy đủ là ghi điểm rất mạnh.

GV: Người dùng đổi email thì các khoá ngoại có bị ảnh hưởng không?

Không, và đây chính là phần thưởng của việc chọn id làm khoá chính. Các bảng con đều trỏ về `users.id`, mà id thì không đổi khi email đổi. Toàn bộ 21 quan hệ vẫn nguyên vẹn.

Nếu ngày xưa chọn email làm khoá chính thì một lần đổi email sẽ phải cập nhật dây chuyền qua tất cả bảng con — đúng thứ mà cơ chế cập nhật dây chuyền sinh ra để xử lý, nhưng MongoDB không có cơ chế đó, nên sẽ phải tự viết và rất dễ sót.

GV: Một triệu người dùng thì đăng nhập có chậm không?

Đăng nhập là một lượt tìm theo email. Nếu quét tuần tự thì với một triệu bản ghi sẽ phải đọc trung bình năm trăm nghìn bản ghi cho mỗi lần đăng nhập — không chấp nhận được.

Hệ thống hiện đã có chỉ mục trên `users.email`. [THỰC TẾ] Với chỉ mục dạng cây, chi phí tìm giảm từ tuyến tính xuống lôgarit, tức khoảng hai mươi bước thay vì năm trăm nghìn.

Nhưng em xin nói thêm một điểm hạn chế ở quy mô đó: toàn bộ trạng thái hệ thống hiện được giữ trong bộ nhớ của một tiến trình. Một triệu người dùng thì không nạp hết vào bộ nhớ được nữa, và phải chuyển sang truy vấn thẳng vào cơ sở dữ liệu theo từng yêu cầu. Đó là thay đổi kiến trúc, không phải thêm chỉ mục. [ĐỀ XUẤT]

ĐIỂM YẾU PHẢI TỰ NÊU Ở BẢNG `USERS`

Chỉ mục email **không** đặt duy nhất ở tầng cơ sở dữ liệu. Việc chặn trùng email chỉ nằm trong mã đăng ký, trả lỗi 409. [THỰC TẾ]

Cách trả lời trung thực: *"Hiện tại ràng buộc duy nhất chỉ có ở tầng ứng dụng. Trong kiến trúc hiện tại điều đó vẫn an toàn vì mọi lượt ghi chạy đồng bộ trên một luồng nên không có hai yêu cầu chen vào giữa bước kiểm tra và bước ghi. Nhưng ngay khi chạy nhiều bản sao máy chủ thì lỗ hổng này là thật, và cách sửa đúng là đặt chỉ mục duy nhất ở cơ sở dữ liệu."*

2.2. Bảng `products` và `product\_variants`

Hai bảng này phải trình bày cùng nhau, vì phần lớn câu hỏi khó nằm ở chỗ **ranh giới** giữa chúng.

2.2.1. Đại diện cho cái gì

`products` đại diện cho một **mặt hàng** theo nghĩa mà khách hàng nghĩ: "áo choàng Haori đáng dài". Đó là thứ có tên, có ảnh, có mô tả, có danh mục, và là thứ khách tìm kiếm.

`product_variants` đại diện cho một **thứ có thể lấy khỏi kho**: đúng cái áo đó, màu Sumi, cỡ S, mã hàng CARDIGA-SUM-S. Khách không mua "áo Haori", khách mua một biến thể cụ thể.

CÂU CHỐT CHO PHẦN NÀY

Khách hàng duyệt theo sản phẩm nhưng kho vận hành theo biến thể. Hai góc nhìn khác nhau nên phải là hai bảng. Nếu gộp một bảng thì hoặc là mất khả năng bán nhiều màu nhiều cỡ, hoặc là phải nhân bản toàn bộ mô tả và ảnh cho từng tổ hợp.

2.2.2. Vì sao tồn kho nằm ở `product\_variants`

Đây là câu hỏi hay gặp nhất ở cụm sản phẩm.

GV: Vì sao stock không để ở `products` cho đơn giản?

Vì **hết cỡ L không có nghĩa là hết áo**. Nếu tồn kho nằm ở `products` thì hệ thống chỉ biết "còn 33 cái" mà không biết còn cỡ nào, màu nào. Khách đặt cỡ XXL trong khi kho chỉ còn cỡ S thì đơn vẫn được nhận, và đến lúc đóng gói mới phát hiện — đó là lỗi nghiệp vụ nghiêm trọng.

Trong dữ liệu thật, 35 sản phẩm sinh ra **317 biến thể**, trung bình chín biến thể một sản phẩm. [THỰC TẾ] Mỗi biến thể có tồn kho riêng.

Nguyên tắc chung: **tồn kho phải nằm ở đúng mức mà người ta lấy hàng ra khỏi kệ**.

GIẢNG VIÊN SẼ TRUY TIẾP

Truy tiếp thường là: "Vây trang danh sách hiển thị còn hàng hay hết hàng dựa vào đâu?" → Cộng tổng tồn của tất cả biến thể; bằng không thì mới coi là hết hàng. Logic này nằm ở `lib/inventory.js`. [THỰC TẾ]

2.2.3. Giá nằm ở đâu — bài toán hai tầng giá

Cột `price` xuất hiện ở **ba** nơi và mỗi nơi mang một nghĩa khác nhau. Phải phân biệt được ba nghĩa này, vì đây là câu hỏi phân loại sinh viên rõ nhất.

Bảng 2.2: Ba cột giá và ý nghĩa khác nhau của chúng

Vị trí	Có mặt	Nghĩa	Đổi được không
<code>products.price</code>	100%	Giá niêm yết hiện tại của mặt hàng	Có. Đổi bất cứ lúc nào.
<code>product_variants.price</code>	1%	Giá riêng của một biến thể, ghi đề giá	Có. Thường để

		chung	trống.
order_items.price	100%	Giá đã chốt tại thời điểm khách đặt	Không bao giờ.

Quy tắc tính giá thật trong mã nguồn: *nếu biến thể có giá riêng thì lấy giá đó; nếu không thì lấy giá sản phẩm cộng phụ phí theo kích cỡ*. Toàn bộ phép tính chạy ở máy chủ, số tiền do ứng dụng gửi lên bị bỏ qua hoàn toàn. [THỰC TẾ]

GV: Nếu giá sản phẩm đổi từ 100.000 lên 150.000 thì đơn hàng cũ hiển thị giá nào?

Vẫn là 100.000. Vì order_items lưu bản chụp giá tại thời điểm đặt, chứ không nối sang products để lấy giá hiện tại. [THỰC TẾ]

Nếu thiết kế theo hướng nối sang products mỗi lần hiển thị thì mọi hoá đơn cũ sẽ tự đổi giá mỗi khi cửa hàng chỉnh giá. Khách mở lại đơn tháng trước thấy số tiền khác với số đã trả — sai về kế toán, và không đối soát được với công thanh toán.

GIẢNG VIÊN SẼ TRUY TIẾP

Truy tiếp gần như chắc chắn: *"Lưu giá ở hai chỗ như vậy có phải dư thừa dữ liệu, có vi phạm chuẩn 3 không?"* → Xem mục 8.1, đây là câu quan trọng nhất của cả cụm sản phẩm.

2.2.4. Vì sao `categoryId` chứ không lưu thẳng tên danh mục

GV: Sao không lưu luôn chuỗi "Áo truyền thống" vào products cho khỏi phải nối bảng?

Vì tên danh mục là **một sự thật dùng chung**, không phải thuộc tính riêng của từng sản phẩm. Nếu chép tên vào mỗi sản phẩm thì khi đổi tên danh mục sẽ phải cập nhật lại toàn bộ sản phẩm thuộc danh mục đó. Sót một bản ghi là dữ liệu mâu thuẫn — đây gọi là bất thường khi cập nhật.

Ngoài ra còn hai vấn đề nữa. Gỡ sai chính tả sẽ tạo ra một danh mục ma mà không ai biết. Và nếu xoá sản phẩm cuối cùng của một danh mục thì danh mục đó biến mất khỏi hệ thống luôn, dù cửa hàng vẫn muốn giữ — đây là bất thường khi xoá.

Với categoryId, đổi tên danh mục là sửa **một** bản ghi.

GIẢNG VIÊN SẼ TRUY TIẾP

Truy tiếp: *"Một sản phẩm thuộc nhiều danh mục thì sao?"* → Thiết kế hiện tại **không hỗ trợ**, vì products.categoryId là một giá trị đơn. Muốn hỗ trợ phải thêm bảng bắc cầu product_categories với khoá chính ghép (productId, categoryId), và bỏ cột categoryId khỏi products. [ĐỀ XUẤT] Xem Phần 12.

2.2.5. Ràng buộc duy nhất thật trên cụm sản phẩm

Bảng 2.3: Chỉ mục thật trên cụm sản phẩm [THỰC TẾ]

Chỉ mục	Duy nhất	Chặn điều gì
products(slug)	Có	Hai sản phẩm cùng đường dẫn — khách mở nhầm hàng.
products(categoryId)	Không	Chỉ để lọc theo danh mục cho nhanh.
product_variants(sku)	Có	Hai biến thể cùng mã hàng — kho không phân biệt

		được.
product_variants(productId, colorName, size)	Có	Hai dòng cùng một tổ hợp màu–cỡ. Nếu trùng thì tồn kho tách làm hai, trừ một dòng còn dòng kia vẫn báo còn hàng.
product_details(productId)	Có	Một sản phẩm có hai bản mô tả chi tiết.
product_media(productId, position)	Không	Chỉ để lấy ảnh theo đúng thứ tự.

Ràng buộc ghép (productId, colorName, size) là ràng buộc quan trọng nhất của cả cụm. Nó mã hoá đúng một quy tắc nghiệp vụ: **trong một sản phẩm, mỗi tổ hợp màu và cỡ chỉ tồn tại một lần.**

2.3. Bảng `orders` và `order\_items`

2.3.1. Vì sao phải tách hai bảng

Đây là câu hỏi kinh điển nhất trong mọi buổi bảo vệ có đơn hàng. Câu trả lời phải đi từ bản số quan hệ chứ không phải từ thói quen.

Về nghiệp vụ, **một đơn có nhiều sản phẩm, và một sản phẩm xuất hiện trong nhiều đơn**. Đó là quan hệ nhiều–nhiều. Mô hình quan hệ không biểu diễn trực tiếp được nhiều–nhiều, nên phải có một bảng ở giữa:

orders	1 : N	order_items
products	1 : N	order_items
⇒ orders	N : N	products

Nhưng `order_items` **không chỉ là bảng nối**. Nếu chỉ nối thì nó chỉ cần hai khoá ngoại. Thực tế nó còn giữ số lượng, giá và bản chụp thông tin hàng — những dữ liệu **không thuộc về đơn hàng cũng không thuộc về sản phẩm, mà thuộc về đúng dòng hàng đó**.

GV: Sao không lưu luôn một mảng sản phẩm trong `orders` cho gọn?

Ba lý do.

Một là vi phạm chuẩn 1. Một ô trong bảng quan hệ phải chứa một giá trị nguyên tử. Lưu "p1,p5,p8" biến một ô thành một danh sách, và ngay lập tức mất khả năng truy vấn: không lọc được đơn nào chứa sản phẩm p5 mà không phải quét chuỗi.

Hai là không có chỗ cho dữ liệu của dòng hàng. Số lượng, giá lúc mua, màu và cỡ đã chọn — chúng thuộc về cặp *đơn này với sản phẩm này*, không nhét vào mảng được nếu không lồng thêm một tầng cấu trúc.

Ba là không thống kê được. Câu hỏi "sản phẩm nào bán chạy nhất tháng" trở thành quét toàn bộ đơn rồi tách chuỗi, thay vì một phép gộp trên `order_items`.

GIẢNG VIÊN SẼ TRUY TIẾP

Truy tiếp: "*Nhưng MongoDB là cơ sở dữ liệu tài liệu, nhúng mảng là chuyện bình thường mà?*" → Đúng, và hệ thống **có** nhúng: mỗi bản ghi `orders` mang một mảng `items`. Nhưng đồng thời vẫn có collection `order_items` riêng với 154 bản ghi và chỉ mục trên `orderId` và `productId`. [THỰC TẾ] Nhúng để đọc một đơn chỉ tốn một lượt truy vấn; tách để thống kê theo sản phẩm. Đây là đánh đổi có chủ ý, không phải trùng lặp do quên.

2.3.2. `order\_items` lưu gì ngoài hai khoá ngoại

Bảng 2.4: Cấu trúc thật của `order\_items` (154 bản ghi) [THỰC TẾ]

Cột	Có mặt	Vì sao có mặt
<code>orderId</code>	100%	Khoá ngoại về <code>orders</code> . Dòng hàng không tồn tại độc lập.
<code>productId</code>	100%	Khoá ngoại về <code>products</code> . Để còn truy được về mặt hàng gốc.

qty	100%	Số lượng của riêng dòng này.
price	100%	Bản chụp giá tại thời điểm đặt. Không bao giờ đổi.
productName	100%	Bản chụp tên hàng. Sản phẩm đổi tên thì đơn cũ vẫn đúng.
productSlug	100%	Bản chụp đường dẫn, để mở lại trang sản phẩm.
colorName colorHex	100%	Bản chụp màu đã chọn, kèm mã màu để hiển thị đúng.
size	100%	Bản chụp kích cỡ đã chọn.
position	100%	Thứ tự dòng trong đơn, để in hoá đơn ổn định.

ĐÂY LÀ ĐIỂM MẠNH NHẤT CỦA THIẾT KẾ — NÊN CHỦ ĐỘNG KHOE

`order_items` chụp lại **năm** thuộc tính của hàng tại thời điểm mua: tên, đường dẫn, màu, cỡ và giá. Nghĩa là một đơn hàng cũ vẫn hiển thị đầy đủ và chính xác kể cả khi sản phẩm đã đổi tên, đổi giá, đổi màu, hoặc **đã bị xoá khỏi hệ thống**.

Rất nhiều đồ án chỉ lưu `productId` rồi nối sang `products` khi hiển thị, và không trả lời được câu hỏi về đổi giá. Ở đây nên nói trước khi bị hỏi.

2.3.3. Trạng thái đơn hàng

Cột `orders.status` có bảy giá trị thật trong dữ liệu. [THỰC TẾ]

Bảng 2.5: Bảy trạng thái đơn hàng

Trạng thái	Nghĩa	Ai chuyển vào
pending	Đơn vừa tạo, chờ cửa hàng xác nhận	Hệ thống, khi khách đặt
pending_payment	Chờ khách hoàn tất thanh toán trực tuyến	Hệ thống
confirmed	Cửa hàng đã xác nhận	Nhân viên
shipping	Đã giao cho đơn vị vận chuyển	Nhân viên
completed	Khách đã nhận hàng	Khách, hoặc hệ thống tự chốt sau thời hạn
cancelled	Đã huỷ	Khách hoặc nhân viên
returned	Đã trả hàng và hoàn tiền	Nhân viên

GV: Có chuyển ngược từ `completed` về `pending` được không? Có mấy trạng thái không?

Về nghiệp vụ thì **không được**, vì đó là đi ngược thời gian: hàng đã giao rồi không thể quay lại trạng thái chờ xác nhận. Đường đi hợp lệ là `pending` → `confirmed` → `shipping` → `completed`, và có thể rẽ sang `cancelled` ở các bước đầu, hoặc sang `returned` sau khi đã hoàn tất.

Về cài đặt, em xin nói trung thực: hệ thống **có** lưu vết mọi lần đổi trạng thái trong cột `history` — một mảng các cặp trạng thái và mốc thời gian [THỰC TẾ] — nên luôn dựng lại được đường đi của đơn. Nhưng **chưa có một bảng quy tắc chuyển trạng thái tập trung** chặn mọi bước nhảy sai. Việc kiểm tra nằm rải trong từng điểm cuối.

Nếu làm tiếp, em sẽ khai báo một bảng chuyển trạng thái hợp lệ ở một chỗ, và mọi điểm cuối đối trạng thái đều phải đi qua đó. [ĐỀ XUẤT]

GIẢNG VIÊN SẼ TRUY TIẾP

"Nếu ứng dụng gửi thẳng `status` tùy ý lên thì sao?" → Điểm cuối đối trạng thái yêu cầu vai trò từ staff trở lên; khách không gọi được. Nhưng đây đúng là lý do vì sao cần bảng quy tắc tập trung thay vì tin vào việc từng điểm cuối tự kiểm tra.

2.4. Bảng `payments`

2.4.1. Vì sao thanh toán không nằm trong `orders`

GV: Sao không để mấy cột paymentStatus, paidAt ngay trong orders cho gọn?

Vì một đơn hàng có thể phát sinh nhiều lượt giao dịch, còn một cột thì chỉ ghi được một lần.

Ví dụ thực tế: khách trả tiền lần một bằng thẻ nhưng ngân hàng từ chối; lần hai đổi sang ví điện tử thì thành công; sau đó khách trả lại một món và cửa hàng hoàn một phần; rồi khách trả nốt và hoàn phần còn lại. Đó là bốn bản ghi giao dịch cho **một** đơn hàng. Với thiết kế một cột thì ba bản ghi đầu bị ghi đè mất, và không đối soát được với cổng thanh toán.

Ngoài ra bản ghi giao dịch còn mang dữ liệu riêng mà đơn hàng không có chỗ chứa: mã giao dịch phía cổng, mã biên nhận, bốn số cuối của thẻ, mã ngân hàng, lý do thất bại.

GIẢNG VIÊN SẼ TRUY TIẾP

"Nhưng dữ liệu hiện tại mỗi đơn chỉ có đúng một thanh toán mà?" → Đúng, hiện là 94 đơn với 94 giao dịch, tỉ lệ một-một. [THỰC TẾ] Nhưng đó là **dữ liệu hiện tại**, không phải **ràng buộc thiết kế**. Chỉ mục trên payments.orderId không đặt duy nhất, nên mô hình sẵn sàng cho nhiều giao dịch một đơn. Nếu đặt quan hệ một-một thì tình huống thanh toán thất bại rồi thử lại sẽ không ghi được.

2.4.2. Chống thanh toán trùng

Đây là phần đáng khoe nhất của cụm thanh toán, vì nó xử lý đúng vấn đề mà hệ thống thật gặp phải: cổng thanh toán **gửi lại thông báo nhiều lần** khi không nhận được phản hồi.

Bảng 2.6: Bốn lớp chống xử lý trùng [THỰC TẾ]

Lớp bảo vệ	Cột thật	Chặn tình huống
Chống tạo đơn trùng	orders.clientRequestId	Khách bấm đặt hàng hai lần, hoặc mạng chập nên ứng dụng gửi lại. Cùng một mã yêu cầu thì trả về đơn đã tạo, không tạo đơn thứ hai.
Chống ghi nhận trả tiền trùng	payments.paidAt và cờ đã trả	Cổng thanh toán gửi lại thông báo thành công lần thứ hai, thứ ba. Mốc thời gian chỉ được đặt một lần.
Chống hoàn kho trùng	orders.stockRestoredAt	Đơn bị hủy hai lần do thao tác lặp. Nếu mốc này đã có thì bỏ qua, không cộng kho lần nữa.
Đối chiếu mã giao dịch	payments.transactionCode	Cùng một mã giao dịch từ cổng thì không tạo bản ghi mới.

GV: Nếu cổng thanh toán báo thành công nhưng máy chủ của em bị lỗi ngay lúc đó thì sao?

Đây là tình huống nguy hiểm nhất trong thương mại điện tử: **tiền đã trừ của khách nhưng hệ thống không biết**.

Hệ thống xử lý theo hai lớp. Lớp thứ nhất là cổng thanh toán **tự gửi lại** thông báo nhiều lần khi không nhận được xác nhận; nhờ các cờ chống trùng ở bảng trên nên gửi lại bao nhiêu lần cũng chỉ ghi nhận một lần.

Lớp thứ hai là **đối soát chủ động**: có điểm cuối cho phép quản trị viên yêu cầu hệ thống hỏi ngược lại cổng thanh toán về trạng thái thật của một giao dịch, rồi cập nhật theo câu trả lời của cổng. [THỰC TẾ] Điểm cuối này yêu cầu quyền quản trị.

Nguyên tắc nền: **cổng thanh toán là nguồn sự thật về tiền**, không phải cơ sở dữ liệu của em. Khi hai bên lệch nhau thì tin cổng.

GIẢNG VIÊN SẼ TRUY TIẾP

"Thế nếu ghi nhận thanh toán thành công nhưng cập nhật đơn hàng lại hỏng?" → Xem Phần 9. Đây là câu hỏi về tính nguyên tử và cần trả lời rất cẩn thận vì hệ thống **không dùng giao dịch của MongoDB**.

PHẦN 3. HỒ SƠ BẢO VỆ CÁC BẢNG HẠNG A

3.1. Bảng `reviews`

reviews là nơi kiểm tra hiểu biết về **ràng buộc nghiệp vụ**, vì đánh giá sản phẩm có nhiều quy tắc mà người mới thiết kế hay bỏ sót.

Bảng 3.1: Cấu trúc thật của `reviews` (31 bản ghi)

Cột	Có mặt	Vai trò
productId userId	100%	Hai khoá ngoại tạo nên bản chất của đánh giá.
orderId orderCode	100%	Bằng chứng đã mua. Đây là điểm mạnh của thiết kế.
userName	100%	Bản chụp tên người viết tại thời điểm đánh giá.
rating	100%	Điểm số. Giá trị thật chỉ có 3, 4, 5.
status	100%	Kết quả kiểm duyệt.
moderation	100%	Chi tiết quyết định kiểm duyệt kèm điểm số.

GV: Người chưa mua có viết đánh giá được không?

Không. Cột orderId là bắt buộc và điểm cuối tạo đánh giá kiểm tra rằng người viết phải có một đơn hàng chứa đúng sản phẩm đó. [THỰC TẾ] Đây là ràng buộc chống đánh giá giả — thứ mà một sàn thương mại điện tử thật bắt buộc phải có, nếu không thì đối thủ có thể tạo tài khoản ảo dìm điểm sản phẩm.

Trên sơ đồ, điều này nhìn thấy được: Đánh Giá Sản Phẩm có đường nối tới Đơn Hàng, không chỉ tới Sản Phẩm. Đó là dây mà nhiều đồ án không có.

GV: Một người đánh giá cùng một sản phẩm hai lần được không?

Về nghiệp vụ thì không nên, và hệ thống **có chặn**: điểm cuối tạo đánh giá kiểm tra xem đã tồn tại bản ghi cùng userId và productId chưa, nếu có thì trả lỗi 409. [THỰC TẾ]

Nhưng em xin nói rõ giới hạn: việc chặn này **chỉ nằm ở tầng ứng dụng**. Chỉ mục trên collection reviews gồm productId, userId, orderId nhưng **không có chỉ mục duy nhất trên cặp (userId, productId)**. [THỰC TẾ]

Thiết kế đúng là thêm ràng buộc duy nhất trên cặp đó ở cơ sở dữ liệu. [ĐỀ XUẤT] Đáng chú ý là hệ thống **đã làm đúng điều này ở các bảng khác** — wishlist_items có ràng buộc duy nhất trên (userId, productId), review_reactions có trên (reviewId, userId). Nên đây là chỗ áp dụng thiếu nhất quán chứ không phải không biết.

GIẢNG VIÊN SẼ TRUY TIẾP

"Ứng dụng kiểm tra rồi thì cơ sở dữ liệu kiểm tra làm gì nữa?" → Xem Phần 10.2. Đây là câu hỏi về tranh chấp đồng thời và phải trả lời bằng ví dụ cụ thể, không nói chung chung.

GV: Điểm trung bình sao có nên lưu sẵn trong products không?

Có hai phương án và mỗi phương án đúng trong một bối cảnh.

Tính lại mỗi lần đọc — luôn chính xác tuyệt đối, không bao giờ lệch, nhưng mỗi lần mở trang sản phẩm phải quét toàn bộ đánh giá của sản phẩm đó.

Lưu sẵn trong `products` — đọc rất nhanh, nhưng phải cập nhật lại mỗi khi có đánh giá mới, sửa, xoá hoặc đổi trạng thái kiểm duyệt. Sót một đường là số hiển thị sai vĩnh viễn mà không ai phát hiện.

Ở quy mô hiện tại — 31 đánh giá trên 35 sản phẩm — tính lại mỗi lần đọc là gần như miễn phí, nên chọn phương án thứ nhất là đúng. Chỉ khi số đánh giá lên hàng chục nghìn mỗi sản phẩm thì mới đáng đánh đổi tính chính xác lấy tốc độ. [SUY LUẬN]

3.2. Bảng `wishlist\_items` và `cart\_items`

Hai bảng bắc cầu này là nơi thể hiện rõ nhất việc hiểu **ràng buộc duy nhất ghép**.

Bảng 3.2: Ràng buộc duy nhất ghép trên hai bảng bắc cầu [THỰC TẾ]

Bảng	Ràng buộc duy nhất thật	Ý nghĩa nghiệp vụ
wishlist_items	(userId, productId)	Một người chỉ thích một sản phẩm một lần. Bấm tìm hai lần không tạo hai bản ghi.
cart_items	(userId, productId, color, size)	Bốn cột , vì cùng một áo nhưng khác màu hoặc khác cỡ là hai dòng giỏ hàng hợp lệ. Chỉ khi trùng cả bốn mới là thêm trùng.

CHI TIẾT GHI ĐIỂM RẤT MẠNH

Việc cart_items dùng ràng buộc **bốn cột** thay vì hai cột cho thấy người thiết kế đã nghĩ tới đúng nghiệp vụ. Nếu chỉ đặt (userId, productId) thì khách không thể bỏ vào giỏ hai cỡ khác nhau của cùng một chiếc áo — một lỗi nghiệp vụ nghiêm trọng trong bán thời trang. Nên chủ động nêu chi tiết này.

GV: Sao không lưu danh sách sản phẩm yêu thích thành một mảng ngay trong users?

Bốn lý do cụ thể.

Không đặt được ràng buộc. Cơ sở dữ liệu không ngăn được việc cùng một mã sản phẩm xuất hiện hai lần trong mảng.

Không lưu được thời điểm. Bảng riêng có cột createdAt [THỰC TẾ], nhờ đó biết được khách thích món này từ bao giờ — dữ liệu dùng để phân tích hành vi và gợi ý.

Truy vấn ngược rất đắt. Câu hỏi "sản phẩm nào được thích nhiều nhất" trở thành quét toàn bộ người dùng rồi mở từng mảng.

Ghi đồng thời hỏng dữ liệu. Hai thiết bị cùng thêm yêu thích thì cả hai đọc mảng cũ rồi cùng ghi đè — một trong hai lượt thêm biến mất.

3.3. Bảng `vouchers` và `voucher\_redemptions`

Cụm này giải thích vì sao **lượt sử dụng** phải là một bảng riêng chứ không phải một bộ đếm.

GV: Sao không để một cột `usedCount` trong vouchers cho nhanh?

Vì một bộ đếm chỉ trả lời được câu "đã dùng bao nhiêu lần", còn nghiệp vụ cần trả lời thêm ba câu nữa: **ai** đã dùng, dùng cho **đơn nào**, và dùng **lúc nào**.

Không có ba câu đó thì không chặn được việc một người dùng đi dùng lại phiếu giới hạn mỗi người một lần, không đối soát được khi khách khiếu nại đã dùng phiếu mà không được giảm, và không thống kê được hiệu quả của chiến dịch khuyến mãi.

Bảng `voucher_redemptions` có chỉ mục trên cả `voucherId`, `userId` và `orderId` [THỰC TẾ] — đúng ba chiều tra cứu mà nghiệp vụ cần.

Trên sơ đồ, Phiếu Giảm Giá nổi tới Người Dùng bằng **hai đường** chứ không phải một. Một đường là người được cấp phiếu, đường kia là người tạo phiếu — cần thiết vì quản trị viên có thể cấp phiếu bù cho khách khi có sự cố, mà đó là thao tác phát sinh giá trị tiền nên phải truy được trách nhiệm. Nhìn sơ đồ dễ tưởng vẽ nhầm, nên **chủ động giải thích trước khi bị hỏi**.

3.4. Bảng ``return_requests``

Bảng này có **ba** khoá ngoại, và ba khoá đó chính là ba câu hỏi bắt buộc phải trả lời khi xử lý một yêu cầu trả hàng.

Bảng 3.3: Ba khoá ngoại của ``return_requests`` [THỰC TẾ]

Khoá ngoại	Trả lời câu hỏi	Bắt buộc
<code>userId</code> → <code>users</code>	Ai yêu cầu trả hàng?	Có
<code>orderId</code> → <code>orders</code>	Trả cho đơn nào?	Có
<code>paymentId</code> → <code>payments</code>	Hoàn tiền qua giao dịch nào?	Không — đơn trả tiền mặt thì không có giao dịch trực tuyến

Việc `paymentId` được phép rỗng là **có chủ ý**, không phải lỏng lẻo: đơn thanh toán khi nhận hàng không phát sinh giao dịch trực tuyến nào để hoàn. Hàm kiểm tra toàn vẹn đánh dấu khoá này là tùy chọn, còn hai khoá kia là bắt buộc. [THỰC TẾ]

3.5. Bảng ``addresses``

GV: Sao địa chỉ phải tách bảng, không để mấy cột ngay trong users?

Vì **một người có nhiều địa chỉ**: nhà riêng, công ty, nhà bố mẹ. Nếu để cột trong `users` thì mỗi người chỉ có đúng một địa chỉ, và mỗi lần giao chỗ khác lại phải sửa đề lên địa chỉ cũ.

Nghiêm trọng hơn là **sửa đề làm sai lịch sử**: đơn hàng tháng trước giao về công ty, tháng này khách đổi địa chỉ mặc định thành nhà riêng, thì đơn cũ hiển thị sai nơi đã giao.

Đó cũng là lý do `orders` **không** nổi tới `addresses` để lấy địa chỉ khi hiển thị, mà chụp lại địa chỉ vào chính bản ghi đơn — cột `address` dạng chuỗi và `addressDetails` dạng cấu trúc. [THỰC TẾ] Cùng một nguyên tắc với giá: **chứng từ phải đóng băng dữ liệu tại thời điểm phát sinh**.

PHẦN 4. PHÂN TÍCH ĐƯỜNG NỐI

Hội đồng rất hay chỉ vào một đường nối thay vì một bảng, vì đường nối kiểm tra được nhiều kiến thức hơn. Phần này phân tích các đường quan trọng theo đúng khung ở mục 0.2.

4.1. Nguyên tắc chung: khoá ngoại nằm ở phía nào

QUY TẮC PHẢI THUỘC LÒNG

Trong quan hệ **một–nhiều**, khoá ngoại luôn nằm ở phía **nhiều**.

Lý do: mỗi bản ghi phía nhiều chỉ trỏ về **một** bản ghi phía một, nên chỉ cần một ô để chứa. Nếu đặt ngược lại, bản ghi phía một sẽ phải chứa **danh sách** — mà một ô chứa danh sách là vi phạm chuẩn 1.

4.2. `users` (1) → (N) `orders`

Bước	Nội dung
Nghịệp vụ	Một khách hàng đặt được nhiều đơn. Một đơn chỉ thuộc về một khách hàng.
Bản số	users 1 : N orders
Vì sao không phải 1:1	Vì như vậy mỗi khách chỉ mua được đúng một lần trong đời. Dữ liệu thật có khách nhiều đơn.
Vì sao không phải N:N	Vì một đơn không thể thuộc về hai người cùng lúc — không biết ai trả tiền, giao cho ai, ai được trả hàng.
Khoá ngoại	orders.userId → users.id [THỰC TẾ]
Vì sao nằm ở orders	orders là phía nhiều. Mỗi đơn chỉ cần một ô để ghi chủ đơn.
Nếu đặt ngược lại	Nếu users giữ orderId thì mỗi người chỉ lưu được một đơn, hoặc phải lưu mảng mã đơn — vi phạm chuẩn 1 và không truy vấn ngược được.
Toàn vẹn	Hàm relationshipErrors() bắt buộc orders.userId phải tồn tại trong users. [THỰC TẾ]
Chỉ mục	orders(userId, createdAt) — chỉ mục ghép, phục vụ đúng truy vấn "đơn của tôi, mới nhất trước". [THỰC TẾ]
Chiến lược xoá	Không xoá người dùng. Chỉ khoá bằng status. Xem mục 2.1.5.

Ngoại lệ cần biết: trong dữ liệu thật, orders.userId có mặt ở 99% bản ghi chứ không phải 100%. [THỰC TẾ] Hàm kiểm tra toàn vẹn bỏ qua khi giá trị là guest — tức là thiết kế có chừa chỗ cho đơn khách vắng lai không đăng nhập. Nếu bị hỏi "có hỗ trợ mua không cần đăng nhập không", đây là câu trả lời.

4.3. `orders` (1) → (N) `order\_items` (N) → (1) `products`

Đây là cặp đường nối quan trọng nhất sơ đồ. Cách trình bày tốt nhất là đi từ nhiều–nhiều.

Bước	Nội dung
Nghịệp vụ	Một đơn có nhiều dòng hàng. Một sản phẩm nằm trong nhiều đơn khác nhau.
Bản số gốc	orders N : N products — mô hình quan hệ không biểu diễn trực tiếp được.
Cách giải	Tách thành hai quan hệ một–nhiều qua bảng order_items ở giữa.
Khoá ngoại	order_items.orderId và order_items.productId [THỰC TẾ]
Không chỉ là bảng nối	Còn giữ qty, price, productName, colorName, size — dữ liệu thuộc về dòng hàng , không thuộc đơn cũng không thuộc sản phẩm.
Chỉ mục	order_items(orderId) để mở một đơn; order_items(productId) để thống kê theo sản phẩm. [THỰC TẾ]
Xoá đơn	Không xoá. Đơn huỷ chuyển status sang cancelled, dữ liệu vẫn còn.
Xoá sản phẩm	Xem mục 11.2 — đây là chỗ có phát hiện quan trọng.

4.4. `orders` (1) → (N) `payments`

Đường nối này hay bị hiểu nhầm là một–một vì dữ liệu hiện tại đang là một–một.

Bước	Nội dung
Nghịệp vụ	Một đơn có thể phát sinh nhiều lượt giao dịch: thử lại sau khi thất bại, hoàn tiền từng phần.
Bản số thiết kế	orders 1 : N payments
Bản số trong dữ liệu hiện tại	1 : 1 — 94 đơn, 94 giao dịch. [THỰC TẾ]
Bảng chứng thiết kế là 1:N	Chỉ mục payments(orderId) không đặt duy nhất. Nếu chủ ý một–một thì phải đặt duy nhất. [THỰC TẾ]
Vì sao không gộp vào orders	Xem mục 2.4.1.
Toàn vẹn	payments.orderId bắt buộc tồn tại; payments.userId là tùy chọn. [THỰC TẾ]

4.5. `products` (1) → (N) `product\_variants` (N) → (1) `Màu Sắc` / `Kích Thước`

Trên sơ đồ, Biến Thể Sản Phẩm có **ba** đường đi ra: về Sản Phẩm, về Màu Sắc và về Kích Thước. Đây là dạng bảng nối ba chiều.

CHỖ DỄ BỊ BẮT BÊ — CHUẨN BỊ TRƯỚC

Trên sơ đồ có hai bảng Màu Sắc và Kích Thước với khoá ngoại từ Biến Thể. Trong MongoDB **không có hai collection đó**; product_variants lưu thẳng colorName, colorHex và size. [THỰC TẾ]

Cách trả lời: "Trên mô hình logic em tách hai bảng tra cứu cho đúng chuẩn. Khi cài đặt em cố ý nhúng vào biến thể, vì hai tập giá trị này đóng, hiếm đối, và không có màn hình nào truy vấn chúng độc lập. Đánh đổi là nếu muốn đổi tên một màu trên toàn hệ thống thì phải cập nhật nhiều bản ghi thay vì một."

4.6. Bảng tra nhanh toàn bộ quan hệ

Mỗi đường được cấp mã R01–R50, ghi rõ field sở hữu tham chiếu, bản số nghiệp vụ và hành vi xoá. Với MongoDB, đây là **logical FK**; chỉ những quan hệ được `relationshipErrors()` phủ mới có enforcement ở tầng ứng dụng.

Bảng 4.2: Quan hệ R01–R10 [THỰC TẾ + CHÍNH SÁCH]

Mã	Bảng con · logical FK	Bảng cha	Bản số	Nghĩa & xoá
R01	Sản Phẩm products.categoryId	Danh Mục Sản Phẩm	category 1 → N products	Một danh mục gom nhiều sản phẩm. Xoá: RESTRICT/reassign khi còn product.
R02	Chi Tiết Sản Phẩm product_details.productId	Sản Phẩm	product 1 → 0..1 detail	Một product có tối đa một khối chi tiết. Xoá: CASCADE phần chi tiết khi product được xoá hợp lệ.
R03	Biến Thể Sản Phẩm product_variants.productId	Sản Phẩm	product 1 → N variants	Một product có nhiều tổ hợp màu–cỡ. Xoá: CASCADE variant; order history dùng snapshot.
R04	Biến Thể Sản Phẩm product_variants.colorName/ colorHex	Màu Sắc	color 1 → N variants (logic)	Nhiều variant dùng cùng khái niệm màu. Xoá: Hiện nhúng; không có parent vật lý để xoá.
R05	Biến Thể Sản Phẩm product_variants.size	Kích Thước	size 1 → N variants (logic)	Nhiều variant dùng cùng kích cỡ. Xoá: Hiện nhúng; không có parent vật lý để xoá.
R06	Hình Ảnh legacy_image.productId	Sản Phẩm	product 1 → N images (logic)	Ảnh cũ thuộc một product. Xoá: Thực thể trùng; phải gộp vào product_media.
R07	Mô Tả Sản Phẩm Tạo Bởi AI ai_descriptions.productId	Sản Phẩm	product 1 → N generations	Một product có thể sinh mô tả nhiều lần. Xoá: Archive/cascade theo retention.
R08	Chi Tiết Đơn Hàng order_items.orderId	Đơn Hàng	order 1 → N items	Một đơn gồm nhiều dòng hàng. Xoá: Không hard-delete chứng từ.
R09	Chi Tiết Đơn Hàng order_items.productId	Sản Phẩm	product 1 → N sold lines	Một product xuất hiện ở nhiều đơn. Xoá: Không cascade lịch sử; snapshot giữ hiển thị.
R10	Đơn Hàng orders.userId	Người Dùng	user 1 → N orders	Một user đặt nhiều đơn; guest là ngoại lệ logic. Xoá: Khóa/ẩn danh user, giữ order.

Bảng 4.3: Quan hệ R11–R20 [THỰC TẾ + CHÍNH SÁCH]

Mã	Bảng con · logical FK	Bảng cha	Bản số	Nghĩa & xoá
R11	Đơn Hàng orders.voucherId	Phiếu Giảm Giá	voucher 1 → 0..N orders	Voucher có thể áp cho nhiều đơn; order có tối đa một voucher record. Xoá: Archive voucher, giữ snapshot giảm trong order.
R12	Thanh Toán payments.orderId	Đơn Hàng	order 1 → N payment attempts	Một đơn có thể retry thanh toán. Xoá: Giữ payment khi order huỷ.
R13	Thanh Toán payments.userId	Người Dùng	user 1 → N payments	Một user phát sinh nhiều giao dịch. Xoá: Ẩn danh user, giữ ledger.
R14	Yêu Cầu Trả Hàng return_requests.orderId	Đơn Hàng	order 1 → N requests	Một order có thể trả theo nhiều đợt/item. Xoá: Giữ request và order.
R15	Yêu Cầu Trả Hàng return_requests.userId	Người Dùng	user 1 → N requests	User tạo nhiều yêu cầu hậu mãi.

				Xoá: Ẩn danh, không phá audit.
R16	Yêu Cầu Trả Hàng return_requests.paymentId	Thanh Toán	payment 1 → 0..N requests	Refund online trả đúng payment; COD có thể rỗng. Xoá: Giữ payment/request.
R17	Địa Chỉ addresses.userId	Người Dùng	user 1 → N addresses	Một user lưu nhiều địa chỉ. Xoá: Có thể cascade số địa chỉ, không chạm order snapshot.
R18	Hồ`Sơ Người Dùng profiles.userId	Người Dùng	user 1 → 0..1 profile	Một user có tối đa một profile hiện tại. Xoá: Cascade/anonymize profile.
R19	Thông Báo notifications.userId	Người Dùng	user 1 → N notifications	Một user nhận nhiều thông báo. Xoá: TTL/anonymize theo retention.
R20	Chi Tiết Giỏ Hàng cart_items.userId	Người Dùng	user 1 → N cart lines	Một user có nhiều dòng giỏ. Xoá: CASCADE/clear khi user xoá.

Bảng 4.4: Quan hệ R21–R30 [THỰC TẾ + CHÍNH SÁCH]

Mã	Bảng con · logical FK	Bảng cha	Bản số	Nghĩa & xoá
R21	Chi Tiết Giỏ Hàng cart_items.productId	Sản Phẩm	product 1 → N cart lines	Một product nằm trong nhiều giỏ. Xoá: Xoá/ẩn cart line khi product unavailable.
R22	Danh Sách Sản Phẩm Yêu Thích wishlist_items.userId	Người Dùng	user 1 → N wishlist rows	Một user thích nhiều product. Xoá: CASCADE mapping.
R23	Danh Sách Sản Phẩm Yêu Thích wishlist_items.productId	Sản Phẩm	product 1 → N wishlist rows	Một product được nhiều user thích. Xoá: CASCADE/ẩn mapping khi archive.
R24	Mục Tiêu Tiết Kiệm goals.userId	Người Dùng	user 1 → N goals	Một user có nhiều mục tiêu. Xoá: Archive/anonymize theo ledger.
R25	Mục Tiêu Tiết Kiệm goals.productId	Sản Phẩm	product 1 → N goals	Một product là đích của nhiều goal. Xoá: Giữ snapshot/đóng goal khi product mất.
R26	Lịch Su`Tìm Kiếm search_logs.userId	Người Dùng	user 1 → N search events	Một user phát sinh nhiều lượt tìm. Xoá: TTL/anonymize.
R27	Tin Nhắn chats.userId	Người Dùng	user 1 → N messages	Một user có nhiều message. Xoá: Retention/anonymize.
R28	Tin Nhắn chats.productId	Sản Phẩm	product 1 → 0..N contextual messages	Message có thể hỏi về một product. Xoá: Giữ text, cho FK logic nullable.
R29	Tương Tác Người Dùng interactions.userId	Người Dùng	user 1 → N events	Một user tạo nhiều event hành vi. Xoá: TTL/anonymize.
R30	Tương Tác Người Dùng interactions.productId	Sản Phẩm	product 1 → N events	Một product nhận nhiều event. Xoá: Giữ aggregate, dọn raw event.

Bảng 4.5: Quan hệ R31–R40 [THỰC TẾ + CHÍNH SÁCH]

Mã	Bảng con · logical FK	Bảng cha	Bản số	Nghĩa & xoá
R31	Lịch Su`Thử`Đồ` tryon_history.productId	Sản Phẩm	product 1 → N try-ons	Một product được thử nhiều lần. Xoá: Retention; không cascade mù asset.
R32	Lịch Su`Thử`Đồ` tryon_history.userId	Người Dùng	user 1 → N try-ons	Một user thử nhiều product/lần. Xoá: Xoá cả metadata và asset theo yêu cầu.
R33	Phiếu Giảm Giá	Người Dùng	user 1 → N issued	Một staff/admin cấp nhiều voucher.

	vouchers.ownerId		vouchers	Xoá: Giữ issuer audit, có thể snapshot tên.
R34	Phiếu Giảm Giá vouchers.userId	Người Dùng	user 1 → 0..N assigned vouchers	Voucher có thể cấp riêng một user. Xoá: Ẩn danh/giữ assignment theo policy.
R35	Lượt Sử Dụng Phiếu voucher_redemptions.voucherId	Phiếu Giảm Giá	voucher 1 → N redemptions	Một voucher có nhiều lượt dùng. Xoá: Không cascade ledger.
R36	Lượt Sử Dụng Phiếu voucher_redemptions.userId	Người Dùng	user 1 → N redemptions	Một user có nhiều lượt dùng voucher. Xoá: Ẩn danh, giữ audit.
R37	Lượt Sử Dụng Phiếu voucher_redemptions.orderId	Đơn Hàng	order 1 → 0..N redemptions	Lượt dùng phải đối soát về order. Xoá: Không xoá ledger.
R38	Thành Viên VIP vip_memberships.userId	Người Dùng	user 1 → N membership periods	User có nhiều kỳ VIP theo thời gian. Xoá: Expire/archive, không hard-delete.
R39	Thành Viên VIP vip_memberships.discountRuleId	Quy Tắc Giảm Giá	rule 1 → N memberships	Nhiều membership dùng cùng rule. Xoá: Archive/version rule.
R40	Đánh Giá Sản Phẩm reviews.productId	Sản Phẩm	product 1 → N reviews	Một product có nhiều review. Xoá: Ẩn review khi product archive; giữ audit.

Bảng 4.6: Quan hệ R41–R50 [THỰC TẾ + CHÍNH SÁCH]

Mã	Bảng con · logical FK	Bảng cha	Bản số	Nghĩa & xoá
R41	Đánh Giá Sản Phẩm reviews.userId	Người Dùng	user 1 → N reviews	Một user review nhiều product. Xoá: Ẩn danh tên snapshot khi cần.
R42	Đánh Giá Sản Phẩm reviews.orderId	Đơn Hàng	order 1 → 0..N reviews	Order hoàn tất chứng minh verified purchase. Xoá: Giữ link chứng từ.
R43	Tương Tác Đánh Giá review_reactions.reviewId	Đánh Giá Sản Phẩm	review 1 → N reactions	Một review nhận nhiều phản ứng. Xoá: CASCADE reaction nếu review bị xoá thật.
R44	Tương Tác Đánh Giá review_reactions.userId	Người Dùng	user 1 → N reactions	Một user phản ứng nhiều review. Xoá: CASCADE/anonymize mapping.
R45	Mẫu Kiểm Duyệt moderation_samples.reviewId	Đánh Giá Sản Phẩm	review 1 → 0..N samples	Mẫu học truy về review nguồn. Xoá: Retention/anonymize.
R46	Đánh Giá Địa Điểm Nhật Bản japan_spot_reviews.userId	Người Dùng	user 1 → N spot reviews	Một user đánh giá nhiều địa điểm. Xoá: Ẩn danh/moderate.
R47	Hình Ảnh Giao Diện Sản Phẩm product_media.productId	Sản Phẩm	product 1 → N media	Một product có nhiều ảnh/video. Xoá: CASCADE metadata, dọn Cloudinary có bù trừ.
R48	Thẻ Địa Danh flagcards.recommendedProductIds[]	Sản Phẩm	flagcard N ↔ N products (array)	Một card gợi ý nhiều product và ngược lại. Xoá: Gỡ ID khi product archive; không xoá card award.
R49	Bộ Sưu Tập Thẻ Của Người Dùng flagcard_collections.userId	Người Dùng	user 1 → 0..N collections	Collection ghi tiến trình/award của user. Xoá: Ẩn danh hoặc giữ audit reward.
R50	Bộ Sưu Tập Thẻ Của Người Dùng flagcard_collections.cardIds[]	Thẻ Địa Danh	collection N ↔ N flagcards (array)	Một collection chứa nhiều card; card thuộc nhiều user. Xoá: Không cascade card khỏi lịch sử award.

Khung nói khi bị chỉ vào đây: **hai thực thể** → **ng nghiệp vụ** → **cardinality** → **field logical FK** → **field nằm ở bên nào và vì sao** → **enforcement thật** → **hành vi xoá** → **ví dụ**. Đừng gọi các field trên là database-native foreign key constraint vì MongoDB không tự tạo ràng buộc đó.

PHẦN 5. DỮ LIỆU CHỤP LẠI VÀ CHUẨN HOÁ

5.1. Câu hỏi khó nhất về chuẩn hoá

GV: products có price, order_items cũng có price. Lưu hai chỗ như vậy là dư thừa, vi phạm chuẩn 3 phải không?

Dạ không, và em xin phân biệt hai khái niệm hay bị nhầm.

Dư thừa là khi cùng **một sự thật** được lưu ở hai nơi, nên sửa một nơi mà quên nơi kia thì dữ liệu mâu thuẫn. Ví dụ chép tên danh mục vào từng sản phẩm: chỉ có một sự thật "danh mục này tên gì", chép ra nhiều bản là dư thừa thật.

Bản chụp lịch sử là khi hai nơi lưu **hai sự thật khác nhau tại hai thời điểm khác nhau**. `products.price` là *giá bán hôm nay*. `order_items.price` là *giá khách đã trả ngày mùng bốn tháng bảy*. Hai con số này **không cần bằng nhau**, và việc chúng khác nhau không phải là mâu thuẫn — đó là lịch sử.

Chuẩn 3 nói rằng một cột không được phụ thuộc hàm vào cột không phải khoá. `order_items.price` không suy ra được từ bất cứ cột nào trong bảng đó, cũng không suy ra được từ `products` — vì nó phụ thuộc vào **thời điểm**, mà thời điểm đó đã trôi qua. Nên nó không vi phạm chuẩn 3.

GIẢNG VIÊN SẼ TRUY TIẾP

"Vậy làm sao phân biệt lúc nào là dư thừa lúc nào là chụp lịch sử?" → Hỏi một câu: "**Nếu bản gốc đổi, giá trị ở đây có phải đổi theo không?**" Nếu **có** thì đó là dư thừa và phải chuẩn hoá. Nếu **không** thì đó là bản chụp và phải giữ.

5.2. Toàn bộ dữ liệu được chụp lại trong hệ thống

Bảng 5.1: Bảy nhóm dữ liệu chụp lại có chủ ý [THỰC TẾ]

Nơi chụp	Chụp cái gì	Bảo vệ điều gì
<code>order_items.price</code>	Giá tại thời điểm đặt	Hoá đơn cũ không đổi số tiền khi cửa hàng chỉnh giá.
<code>order_items.productName</code>	Tên hàng tại thời điểm đặt	Đơn cũ vẫn đúng tên khi sản phẩm đổi tên.
<code>order_items.colorName</code> <code>colorHex</code> <code>size</code>	Màu và cỡ đã chọn	Biến thể bị xoá thì đơn vẫn hiển thị đủ.
<code>orders.customer</code>	Tên và số điện thoại người mua	Khách đổi tên thì đơn cũ vẫn ghi tên lúc mua.
<code>orders.address</code> <code>addressDetails</code>	Địa chỉ giao tại thời điểm đặt	Khách đổi địa chỉ mặc định thì đơn cũ vẫn đúng nơi đã giao.
<code>reviews.userName</code>	Tên người viết đánh giá	Đổi tên hiển thị không làm sai lịch sử đánh giá.
<code>payments.orderCode</code>	Mã đơn	Đối soát với cổng thanh toán không cần nối bảng.

Nguyên tắc chung của cả bảng trên: **dữ liệu gốc được chuẩn hoá, chứng từ thì đóng băng**. Đây là câu tổng kết nên nói ra khi bị hỏi về chuẩn hoá.

5.3. Bốn loại bất thường và cách hệ thống tránh

Bảng 5.2: Bốn bất thường kinh điển và cách thiết kế hiện tại xử lý

Loại bất thường	Ví dụ nếu thiết kế sai	Hệ thống tránh bằng cách nào
Khi thêm	Không tạo được danh mục mới nếu chưa có sản phẩm nào thuộc danh mục đó.	<code>categories</code> là bảng độc lập, tồn tại không cần sản phẩm.
Khi sửa	Đổi tên danh mục phải sửa hàng nghìn dòng sản phẩm, sót một dòng là mâu thuẫn.	<code>products.categoryId</code> trỏ về <code>categories</code> , đổi tên chỉ sửa một bản ghi.
Khi xoá	Xoá sản phẩm cuối của một danh mục làm mất luôn danh mục đó.	Danh mục nằm ở bảng riêng, không sống nhờ sản phẩm.
Vi phạm nguyên tử	Lưu "p1, p5, p8" vào một ô — không lọc, không thống kê, không ràng buộc được.	Dòng hàng tách thành bản ghi riêng trong <code>order_items</code> .

PHẦN 6. ĐỒNG THỜI, GIAO DỊCH VÀ TÍNH NGUYÊN TỬ

Đây là phần dễ mất điểm nhất nếu trả lời theo sách giáo khoa, và ghi điểm rất mạnh nếu trả lời đúng cơ chế thật của hệ thống.

6.1. Bài toán hai người cùng mua sản phẩm cuối cùng

Tình huống: biển thể còn đúng **một** cái. Khách A và khách B cùng bấm đặt hàng.

Bảng 6.1: Tranh chấp đồng thời nếu không có cơ chế bảo vệ

Thời điểm	Khách A	Khách B	Tồn kho
t1	Đọc tồn = 1 → đủ		1
t2		Đọc tồn = 1 → đủ	1
t3	Trừ 1 → ghi 0		0
t4		Trừ 1 → ghi 0	0 nhưng đã bán 2 cái

GV: Hệ thống của em có bị lỗi này không?

Trong phạm vi một tiến trình hiện tại thì bài kiểm thử không bị oversell; nếu chạy nhiều tiến trình thì chưa an toàn. Cơ chế hiện tại không phải giao dịch MongoDB.

Toàn bộ trạng thái hệ thống nằm trong một đối tượng trong bộ nhớ, và mọi hàm sửa đổi chạy **đồng bộ trên một luồng**. Bước kiểm tra tồn kho và bước trừ kho nằm trong cùng một hàm sửa đổi, không có `await` ở giữa. Vì Node chạy một luồng nên không yêu cầu nào chen vào được giữa hai bước đó. [THỰC TẾ]

Nhóm có unit test: **50 lượt gọi Promise trên tồn kho 5**, chạy với store file-mode trong một process; kết quả đúng 5 lượt thành công, 45 lượt bị từ chối và tồn kho về 0. [THỰC TẾ] Test này chứng minh cơ chế synchronous mutator, không mô phỏng hai process/pod.

Nhưng em xin nêu thẳng giới hạn: tính chất này **chỉ đúng trong một tiến trình**. Nếu chạy hai bản sao máy chủ thì mỗi bản có một bản trạng thái riêng trong bộ nhớ, và lỗi ở bảng trên quay lại ngay.

GIẢNG VIÊN SẼ TRUY TIẾP

"*Thế muốn chạy nhiều máy chủ thì phải làm gì?*" → Có ba hướng: cập nhật có điều kiện (`findOneAndUpdate` với điều kiện `stock >= qty`, để chính cơ sở dữ liệu quyết định ai thắng), khoá lạc quan bằng cột phiên bản, hoặc dùng giao dịch thật của MongoDB. Hướng thứ nhất rẻ nhất và đủ cho bài toán tồn kho. [ĐỀ XUẤT]

6.2. Hệ thống có dùng giao dịch không

TRẢ LỜI TRUNG THỰC — ĐÂY LÀ CÂU HAY BỊ HỎI BẮY

Không. Tìm toàn bộ mã nguồn máy chủ không có lời gọi `startSession` hay `withTransaction`

nào. [THỰC TẾ]

Không được trả lời "có" cho câu này. Nếu giảng viên yêu cầu chỉ ra chỗ dùng giao dịch mà không có thì mất điểm nặng hơn nhiều so với việc thừa nhận ngay từ đầu và giải thích được cơ chế thay thế.

Cách trình bày đúng là đối chiếu bốn tính chất của giao dịch với cơ chế thật:

Bảng 6.2: Bốn tính chất giao dịch và cách hệ thống đạt được

Tính chất	Hệ thống đạt được bằng cách nào	Mức độ
Nguyên tử	Hàm sửa đổi chạy đồng bộ trên bản sao trạng thái; nếu ném lỗi giữa chừng thì bản sao bị bỏ, trạng thái gốc không đổi. Có kiểm thử. [THỰC TẾ]	Đạt trong một tiến trình
Nhất quán	Hàm <code>relationshipErrors()</code> chạy trước <code>write()</code> ở Mongo mode; nhánh <code>writeFile()</code> JSON không gọi. [THỰC TẾ]	Một phần — enforcement khác nhau theo mode
Cô lập	Một luồng nên không có hai lượt ghi đan xen. Không cần mức cô lập vì không có song song thật.	Đạt trong một tiến trình
Bền vững	Response có thể trả sau khi state mới chỉ ở RAM. Timer debounce chờ 40 ms yên lặng rồi mới bắt đầu persist Mongo; write liên tục còn reset timer.	Chưa có durable-before-ack

Đây không chỉ là một "cửa sổ đúng 40 ms". Timer bị reset mỗi lần ghi và persist còn chạy bất đồng bộ, nên crash có thể làm mất **toàn bộ thay đổi chưa flush kể từ lần persist thành công cuối**, không có cơ sở để khẳng định chỉ mất một write. [THỰC TẾ]

6.3. Nếu một bước trong luồng đặt hàng thất bại

Luồng đặt hàng gồm năm việc: tạo đơn, tạo dòng hàng, trừ kho, tạo bản ghi thanh toán, cập nhật điểm thành viên.

GV: Nếu bước bốn thất bại thì bốn bước trước có bị hoàn lại không?

Có, với điều kiện cả năm bước nằm trong **cùng một hàm sửa đổi**. Cách hoạt động: hệ thống nhân bản trạng thái, chạy cả năm bước trên bản sao, rồi mới thay thế trạng thái gốc bằng bản sao. Nếu bước bốn ném lỗi thì bản sao bị bỏ đi và trạng thái gốc **chưa hề bị chạm vào** — tương đương một lần quay lui. [THỰC TẾ] Nhóm có kiểm thử khẳng định điều này.

Nhưng em xin nêu một tình huống thật mà cơ chế trên **không** che được: khi việc thất bại không nằm bên trong hàm sửa đổi mà nằm ở **cổng thanh toán bên ngoài**. Ví dụ đơn đã tạo, kho đã trừ, rồi khách bỏ ngang không trả tiền.

Kho được trừ ngay khi tạo đơn chờ thanh toán. Bản code hiện tại **đã** gọi `restockCancelledOrder()` khi Stripe/VNPay được xác nhận failed/cancelled/expired; cờ `order.stockRestoredAt` làm thao tác hoàn kho idempotent. [THỰC TẾ]

Giới hạn còn lại: một order cứ treo pending vì callback không tới và không ai chạy reconcile thì hàng vẫn bị giữ. Các endpoint reconcile có thật, nhưng không thấy scheduler thanh toán tự động trong server. Không được nói mọi failure path đã được giải quyết.

GIẢNG VIÊN SẼ TRUY TIẾP

"Vậy sửa phần *pending treo thế nào?*" → Chạy worker định kỳ hỏi provider, đặt timeout cho reservation, rồi gọi cùng hàm hoàn kho idempotent. [ĐỀ XUẤT]

PHẦN 7. TOÀN VỆN THAM CHIẾU VÀ CHIẾN LƯỢC XOÁ

7.1. MongoDB không có khoá ngoại — vậy 50 quan hệ là gì

GV: MongoDB không thực thi khoá ngoại, vậy các đường nối trên sơ đồ có ý nghĩa gì?

Sơ đồ là **mô hình logic**, không phải ảnh chụp cấu trúc vật lý. Đúng là MongoDB không có ràng buộc khoá ngoại dựng sẵn.

Ràng buộc được thực thi ở tầng ứng dụng bằng hàm `relationshipErrors()`. Hàm này quét toàn bộ trạng thái trước mỗi lượt ghi; nếu phát hiện tham chiếu trỏ tới bản ghi không tồn tại thì ném lỗi mã 409 và **huỷ toàn bộ lượt ghi**, không ghi một phần. [THỰC TẾ]

GIỚI HẠN THẬT CỦA CƠ CHẾ NÀY — PHẢI TỰ NÊU

Kiểm tra toàn vẹn **chỉ chạy trên đường ghi vào MongoDB**. Trong `lib/store.js`, hàm `write()` gọi `assertValid()` khi đang dùng MongoDB, nhưng nhánh ghi xuống tệp JSON dự phòng thì **không gọi**. [THỰC TẾ]

Nghĩa là khi MongoDB gặp sự cố và hệ thống rơi về tệp cục bộ, các lượt ghi trong giai đoạn đó không được kiểm tra toàn vẹn.

Ngoài ra hàm chỉ phủ khoảng **chín** nhóm bảng con: sản phẩm, đơn hàng, dòng hàng, thanh toán, yêu cầu trả hàng, đánh giá, tương tác đánh giá, thành viên VIP, và nhóm địa chỉ – giỏ – yêu thích – hồ sơ – mục tiêu. Các bảng còn lại như `notifications`, `interactions`, `tryon_history`, `voucher_redemptions` **không** được kiểm tra. [THỰC TẾ]

Cách trả lời: "Cơ chế này phủ các quan hệ có giá trị nghiệp vụ cao nhất, chưa phủ hết. Đó là lựa chọn ưu tiên chứ không phải quên, nhưng em thừa nhận là chưa đầy đủ."

7.2. Sản phẩm bị xoá thì đơn hàng cũ ra sao

Câu này rất hay bị hỏi và có một phát hiện đáng nói.

Bảng 7.1: Hành vi thật khi xoá sản phẩm

Câu hỏi	Kết quả thật
Có điểm cuối xoá sản phẩm không?	Có — <code>DELETE /api/products/:id</code> , yêu cầu quyền quản trị. [THỰC TẾ]
Là xoá mềm hay xoá thật?	Xoá thật . Bản ghi bị loại khỏi mảng sản phẩm, không có cột <code>deletedAt</code> . [THỰC TẾ]
Đơn hàng cũ có hỏng không?	Không . <code>order_items</code> đã chụp tên, giá, màu, cỡ nên vẫn hiển thị đủ.
Còn mở được trang sản phẩm từ đơn cũ không?	Không — liên kết dẫn tới sản phẩm không còn tồn tại.
Hàm toàn vẹn có chặn không?	Có chặn khi đang dùng MongoDB , vì hàm bắt buộc <code>productId</code> trong dòng hàng phải tồn tại. Khi chạy trên tệp dự phòng thì không chặn. [THỰC TẾ]

Cách trả lời gọn: "Đơn hàng cũ không hỏng, vì dòng hàng đã chụp lại toàn bộ thông tin cần để hiển thị. Cái mất là liên kết dẫn về trang sản phẩm. Về mặt cơ chế, hàm kiểm tra toàn vẹn của em tạo ra hiệu ứng tương đương *RESTRICT* — không cho xoá sản phẩm đã từng được đặt — nhưng em xin nói rõ đây là hệ quả của kiểm tra toàn vẹn chứ nhóm chưa khai báo nó thành một chính sách xoá tường minh cho từng bảng."

7.3. Bảng chiến lược xoá cho từng quan hệ

Bảng 7.2: Chiến lược xoá theo từng cặp quan hệ

Bản ghi cha	Bản ghi con	Hành vi hiện tại	Có đúng không
users	orders payments reviews	Không có chức năng xoá người dùng; chỉ khoá bằng status.	Đúng. Chứng từ phải giữ. Nên bổ sung ẩn danh hoá. [ĐỀ XUẤT]
products	order_items	Xoá thật, nhưng bị hàm toàn vẹn chặn nếu đã có đơn.	Đúng về kết quả, chưa tường minh về chính sách.
products	product_variants product_media	Con sống chết theo cha — đúng dạng dữ liệu chi tiết.	Đúng. Tương đương CASCADE.
orders	order_items payments	Đơn không bị xoá; huỷ thì chuyển status sang cancelled.	Đúng. Dữ liệu giao dịch không xoá thật.
users	wishlist_items cart_items	Xoá thoải mái, không mất thông tin gốc.	Đúng. Dữ liệu bắc cầu.
reviews	review_reactions	Ràng buộc duy nhất (reviewId, userId); đánh giá mất thì phản hồi vô nghĩa.	Đúng, nên xoá theo cha. [SUY LUẬN]

VÌ SAO KHÔNG DÙNG CASCADE CHO MỌI THỨ

CASCADE nghe tiện nhưng rất nguy hiểm với dữ liệu giao dịch. Nếu đặt xoá dây chuyền từ users thì một lệnh xoá nhằm một tài khoản sẽ kéo theo mất hết đơn hàng, thanh toán và đánh giá của người đó — không khôi phục được và sai về kế toán.

Quy tắc: **CASCADE chỉ dùng cho dữ liệu chi tiết không tồn tại độc lập** (biến thể, ảnh sản phẩm). Với dữ liệu giao dịch thì dùng *RESTRICT* hoặc không cho xoá.

PHẦN 8. RÀNG BUỘC DUY NHẤT VÀ CHỈ MỤC

8.1. Toàn bộ ràng buộc duy nhất thật trong hệ thống

Bảng 8.1: Mười hai ràng buộc duy nhất thật [THỰC TẾ]

Collection	Ràng buộc duy nhất	Chặn điều gì
users	id	Hai người cùng định danh.
products	id · slug	Hai sản phẩm cùng đường dẫn.
product_variants	sku · (productId, colorName, size)	Tồn kho bị tách đôi cho cùng một tổ hợp.
product_details	productId	Một sản phẩm có hai bản mô tả.
orders	id · code	Hai đơn cùng mã — khách mở nhầm đơn người khác.
cart_items	(userId, productId, color, size)	Thêm trùng đúng một tổ hợp vào giỏ.
wishlist_items	(userId, productId)	Thích một sản phẩm hai lần.
review_reactions	(reviewId, userId)	Một người bấm hữu ích hai lần.
vouchers	id · code	Hai phiếu cùng mã.
discount_rules	id · code	Hai quy tắc giảm giá cùng mã.
profiles	userId	Một người có hai hồ sơ.
push_tokens	token	Một thiết bị đăng ký hai lần, nhận thông báo nhân đôi.

8.2. Vì sao cơ sở dữ liệu vẫn cần ràng buộc dù ứng dụng đã kiểm tra

GV: Ứng dụng đã kiểm tra trùng rồi, đặt ràng buộc ở cơ sở dữ liệu làm gì cho chậm?

Vì giữa lúc **đọc để kiểm tra** và lúc **ghi** luôn có một khoảng trống thời gian, và trong khoảng đó một yêu cầu khác có thể chen vào.

Ví dụ cụ thể với đăng ký tài khoản: yêu cầu A đọc và thấy chưa có ai dùng email này; yêu cầu B cũng đọc và cũng thấy chưa có; A ghi thành công; B ghi cũng thành công. Kết quả là hai tài khoản cùng email, và từ đó trở đi việc đăng nhập không xác định được là ai.

Kiểm tra ở ứng dụng là để **báo lỗi thân thiện cho người dùng**. Ràng buộc ở cơ sở dữ liệu là để **bảo đảm dữ liệu không bao giờ sai**. Hai việc khác nhau, cần cả hai.

Về chi phí: ràng buộc duy nhất được cài đặt bằng một chỉ mục, mà chỉ mục đó thường vẫn cần cho việc tra cứu. Nên chi phí gần như bằng không.

GIẢNG VIÊN SẼ TRUY TIẾP

Nếu giảng viên hỏi "*hệ thống em có bị lỗi đó không*" — trả lời: **hiện tại thì không**, vì mọi lượt ghi chạy đồng bộ trên một luồng nên không có hai lượt chen nhau. Nhưng đó là do kiến trúc chứ không do ràng buộc, nên khi chạy nhiều bản sao máy chủ thì lỗi hổng thành thật. Xem điểm yếu Y-01 và Y-02 ở Phần 12.

8.3. Chỉ mục và truy vấn tương ứng

Chỉ mục không được đặt theo cảm tính. Mỗi chỉ mục phải trả lời được câu hỏi "truy vấn nào dùng nó".

Bảng 8.2: Chỉ mục và truy vấn tương ứng [THỰC TẾ]

Chỉ mục thật	Truy vấn nó phục vụ
users(email)	Đăng nhập — tìm tài khoản theo email.
orders(userId, createdAt)	Màn hình "Đơn hàng của tôi", sắp xếp mới nhất trước. Chỉ mục ghép nên vừa lọc vừa sắp xếp được trong một lần đọc.
order_items(orderId)	Mở chi tiết một đơn — lấy các dòng hàng của đơn đó.
order_items(productId)	Thống kê "sản phẩm bán chạy" — gộp theo sản phẩm.
payments(orderId) · payments(userId)	Tra giao dịch theo đơn, và lịch sử thanh toán của một người.
products(categoryId)	Duyệt sản phẩm theo danh mục.
product_variants(productId)	Lấy toàn bộ biến thể khi mở trang sản phẩm.
reviews(productId)	Hiển thị đánh giá trên trang sản phẩm.
return_requests(orderId)	Kiểm tra một đơn đã có yêu cầu trả hàng chưa.
vip_memberships(userId, status)	Kiểm tra một người có đang là thành viên VIP hiệu lực không.

GV: Chỉ mục càng nhiều càng nhanh đúng không?

Không. Chỉ mục làm **đọc nhanh hơn** nhưng làm **ghi chậm hơn**, vì mỗi lần thêm, sửa hoặc xoá một bản ghi thì tất cả chỉ mục của bảng đó đều phải cập nhật theo. Chỉ mục cũng chiếm thêm dung lượng đĩa và bộ nhớ.

Với bảng ghi nhiều đọc ít — như interactions với 675 bản ghi hành vi — đặt nhiều chỉ mục là lỗ vốn. Đúng như vậy, các bảng nhật ký trong hệ thống hiện **không** có chỉ mục ngoài khoá chính. [THỰC TẾ]

Nguyên tắc: chỉ đặt chỉ mục cho truy vấn **thật sự chạy thường xuyên**, và phải chỉ ra được truy vấn đó là gì.

PHẦN 9. BẢO MẬT Ở TẦNG DỮ LIỆU

Bảng 9.1: Bảo mật ở tầng dữ liệu

Câu hỏi thường gặp	Hiện trạng thật
Có lưu mật khẩu dạng rõ không?	Không. Chỉ có cột passwordHash, băm bằng bcrypt 10 vòng. [THỰC TẾ]
Có cột muối riêng không?	Không cần — bcrypt nhúng muối ngẫu nhiên ngay trong chuỗi kết quả.
Có lưu số thẻ ngân hàng không?	Không. Chỉ lưu bốn số cuối, thương hiệu thẻ và mã khách phía cổng thanh toán. [THỰC TẾ]
Có lưu mã bảo mật thẻ không?	Không, và không bao giờ được lưu — vi phạm chuẩn an toàn thẻ.
Giao diện lập trình có trả về mã băm mật khẩu không?	Không. Có hàm loại bỏ trường nhạy cảm trước khi trả về. [THỰC TẾ]
Phân quyền ở đâu?	Bốn vai trò xếp thứ bậc, kiểm tra bằng bộ lọc ở tầng máy chủ trước khi vào điểm cuối. [THỰC TẾ]
Mã đặt lại mật khẩu lưu thế nào?	Băm và có hạn dùng — resetCodeHash kèm resetCodeExpiresAt. [THỰC TẾ]

ĐIỂM NÊN CHỦ ĐỘNG NÊU

Trong đợt rà soát, nhóm tìm ra **chín điểm cuối thiếu kiểm soát truy cập** và đã vá. Nghiêm trọng nhất là điểm cuối tra cứu giao dịch: mã giao dịch có dạng đếm tăng dần nên đoán được, mà nó trả về cả họ tên, số điện thoại và địa chỉ của khách — người gọi ẩn danh cũng xem được. [THỰC TẾ]

Bài học rút ra là **ấn nút trên giao diện không phải là phân quyền**. Nhóm đã bổ sung 10 kiểm thử tự động khoá lại toàn bộ quy tắc truy cập.

PHẦN 10. NẾU HỆ THỐNG CÓ MỘT TRIỆU NGƯỜI DÙNG

Câu này không được trả lời chung chung kiểu "nâng cấp máy chủ". Phải nhìn vào đúng bảng nào phình nhanh và đúng truy vấn nào gây trước.

Bảng 10.1: Bảng nào phình nhanh và gây ở đâu

Bảng	Tốc độ phình	Gây ở đâu trước	Cách xử lý
interactions	Rất nhanh — mỗi lượt xem một bản ghi	Quét để gợi ý sẽ chậm dần	Gộp sẵn theo ngày; dọn bản ghi cũ hơn 90 ngày. [ĐỀ XUẤT]
orders order_items	Theo doanh số	Màn hình "đơn của tôi" nếu thiếu chỉ mục	Đã có orders (userId, createdAt). [THỰC TẾ]
notifications	Nhanh — mỗi sự kiện một bản ghi	Danh sách thông báo	Xoá bản ghi đã đọc quá hạn. [ĐỀ XUẤT]
users	Chậm hơn	Đăng nhập nếu thiếu chỉ mục email	Đã có chỉ mục. [THỰC TẾ]
tryon_history chats	Theo mức dùng tính năng AI	Dung lượng ảnh và hội thoại	Chuyển ảnh sang kho đối tượng, chỉ giữ đường dẫn. [ĐỀ XUẤT]

NÚT THẮT LỚN NHẤT — PHẢI NÊU TRƯỚC KHI BỊ HỎI

Vấn đề ở quy mô một triệu người dùng **không phải chỉ mục**, mà là kiến trúc: toàn bộ trạng thái hệ thống hiện được giữ trong bộ nhớ của **một** tiến trình. [THỰC TẾ]

Với một triệu người dùng, dữ liệu không nạp hết vào bộ nhớ được nữa. Bắt buộc phải chuyển sang truy vấn thẳng vào cơ sở dữ liệu theo từng yêu cầu, và khi đó mất luôn tính nguyên tử nhờ một luồng — nên phải chuyển sang cập nhật có điều kiện hoặc giao dịch thật.

Đây là thay đổi **kiến trúc**, không phải tinh chỉnh. Nói được điều này cho thấy hiểu giới hạn thật của hệ thống mình.

GV: Phân trang với bảng lớn thì làm sao?

Cách thông thường là bỏ qua N bản ghi rồi lấy tiếp — nhưng cách này chậm dần theo số trang, vì để lấy trang thứ mười nghìn thì cơ sở dữ liệu vẫn phải đếm qua toàn bộ bản ghi phía trước rồi vớt đi.

Cách đúng cho bảng lớn là **phân trang theo con trỏ**: thay vì hỏi "trang thứ 10000", hỏi "các bản ghi có createdAt nhỏ hơn giá trị cuối của trang trước". Kết hợp với chỉ mục trên cột đó thì chi phí không đổi dù ở trang nào.

Ở quy mô đồ án, cách thông thường vẫn đủ và đơn giản hơn nhiều. [SUY LUẬN]

PHẦN 11. NẾU GIẢNG VIÊN ĐỔI YÊU CẦU TẠI CHỖ

Dạng câu hỏi này kiểm tra khả năng thiết kế, không phải khả năng nhớ. Trả lời theo ba bước: **thiết kế hiện tại có đáp ứng không** → **cần thêm gì** → **khoá chính và bản số ra sao**.

Bảng 11.1: Tám tình huống đổi yêu cầu

Yêu cầu mới	Hiện tại có đáp ứng	Cần thêm gì
Một sản phẩm thuộc nhiều danh mục	Không — <code>products.categoryId</code> là giá trị đơn	Bảng <code>product_categories</code> , khoá chính ghép (<code>productId</code> , <code>categoryId</code>); bỏ cột <code>categoryId</code> .
Một người có nhiều vai trò	Không — <code>users.role</code> là chuỗi đơn	Bảng <code>roles</code> và bảng bắc cầu <code>user_roles</code> khoá chính ghép (<code>userId</code> , <code>roleId</code>).
Một đơn có nhiều lượt thanh toán	Có — <code>payments.orderId</code> không đặt duy nhất	Không cần đổi cấu trúc. Chỉ cần bổ sung quy tắc: tổng tiền các giao dịch thành công phải bằng tổng đơn.
Một người có nhiều địa chỉ	Có — <code>addresses</code> đã là bảng riêng 1:N	Không cần đổi gì.
Sản phẩm có màu và cỡ	Có — đã có <code>product_variants</code> với ràng buộc ghép ba cột	Không cần đổi gì. Đây là điểm mạnh nên khoe.
Một đánh giá có nhiều ảnh	Một phần — có cột <code>media</code> nhưng đang rỗng ở mọi bản ghi	Hoặc dùng mảng trong <code>reviews</code> , hoặc tách bảng <code>review_media</code> nếu cần truy vấn theo ảnh.
Sản phẩm bán theo nhiều kho	Không — tồn kho là một số trong biến thể	Bảng <code>warehouses</code> và <code>variant_stock</code> khoá chính ghép (<code>variantId</code> , <code>warehouseId</code>).
Danh mục có cấp cha con	Không — <code>categories</code> phẳng	Thêm <code>parentId</code> trở về chính <code>categories</code> — quan hệ tự tham chiếu; gốc thì để rỗng.

PHẦN 12. ĐIỂM YẾU THẬT — KHÔNG BẢO VỆ MÙ QUÁNG

Mọi mục trong phần này đều là điểm yếu **có thật**, xác nhận được từ mã nguồn. Nếu bị hỏi đúng chỗ thì **không cố cãi** — thừa nhận, giải thích bối cảnh, nêu cách sửa.

Bảng 12.1: Mười điểm yếu có thật, xác nhận từ mã nguồn

Mã	Vấn đề	Bảng	Mức	Cách trả lời an toàn
Y-01	Chỉ mục email không đặt duy nhất; chặn trùng chỉ ở tầng ứng dụng.	users	Cao	Thừa nhận. Nêu rằng kiến trúc một luồng hiện che được, nhưng chạy nhiều bản sao là hỏng. Cách sửa: đặt chỉ mục duy nhất.
Y-02	Không có ràng buộc duy nhất (userId, productId) cho đánh giá.	reviews	Trung bình	Thừa nhận, và nêu rằng hệ thống đã làm đúng ở wishlist_items và review_reactions — nên đây là áp dụng thiếu nhất quán.
Y-03	Hai thực thể hình ảnh trùng vai trò trên sơ đồ; chỉ một collection tồn tại.	ERD	Trung bình	Thừa nhận là dấu vết của bản thiết kế đầu khi chưa hỗ trợ video. Nêu cách gộp.
Y-04	Kiểm tra toàn vẹn không chạy trên nhánh ghi xuống tệp dự phòng.	Toàn hệ thống	Cao	Thừa nhận. Nêu rằng nhánh dự phòng là chế độ suy giảm, và đúng ra phải kiểm tra ở cả hai nhánh.
Y-05	Hàm toàn vẹn chỉ phủ khoảng 9 nhóm bảng con trên 36 thực thể.	Toàn hệ thống	Trung bình	Nêu là lựa chọn ưu tiên theo giá trị nghiệp vụ, nhưng thừa nhận chưa đầy đủ.
Y-06	Đơn thanh toán treo pending có thể giữ kho nếu callback/reconcile không chạy.	orders	Trung bình	Không nói code chưa hoàn kho: nhánh failed/cancelled đã hoàn idempotent. Điểm thiếu là scheduler tự xử lý reservation treo.
Y-07	Không có bảng quy tắc chuyển trạng thái tập trung.	orders	Thấp	Nêu rằng đã có cột history lưu vết đầy đủ, nhưng quy tắc chặn bước nhảy sai còn rải rác.
Y-08	Không có chức năng xoá hay ẩn danh hoá tài khoản.	users	Thấp	Nêu rằng giữ chứng từ là đúng, nhưng thiếu ẩn danh hoá là thiếu về quyền riêng tư.
Y-09	Không dùng giao dịch của cơ sở dữ liệu.	Toàn hệ thống	Cao	Nói thẳng là không dùng. Giải thích cơ chế một luồng thay thế, và nêu giới hạn một tiến trình.
Y-10	Response có thể trả trước khi thay đổi được persist xuống MongoDB.	Toàn hệ thống	Cao	Timer debounce 40 ms bị reset khi write mới tới; không được khẳng định chỉ mất một write. Cần durable-before-ack/outbox.

PHẦN 13. KHI GIẢNG VIÊN CỐ TÌNH BẮT BÊ

Những câu dưới đây là **bẫy**: chúng phát biểu một điều nghe có vẻ đúng để xem sinh viên có gật theo không. Trả lời đúng là chỉ ra điều kiện áp dụng.

GV: Em tách nhiều bảng quá, nối bảng nhiều như vậy chẳng phải thiết kế kém sao?

Nối bảng là **cái giá phải trả** để có tính đúng đắn của dữ liệu, và ở quy mô này cái giá đó rất rẻ vì đã có chỉ mục trên các cột dùng để nối.

Nếu gộp bảng để tránh nối thì sẽ gặp ba loại bất thường ở Bảng 5.2: đổi tên danh mục phải sửa hàng nghìn dòng, không tạo được danh mục rỗng, và xoá sản phẩm cuối làm mất danh mục.

Nói cách khác, gộp bảng không xoá được độ phức tạp — nó chuyển độ phức tạp từ câu truy vấn sang mã ứng dụng, chỗ mà cơ sở dữ liệu không giúp được gì.

GV: Chuẩn hoá tới dạng chuẩn 3 thì lúc nào cũng tốt, đúng không?

Không. Chuẩn hoá tối ưu cho **tính đúng đắn khi ghi**, và đánh đổi bằng chi phí nối khi đọc.

Có hai trường hợp cố ý không chuẩn hoá và đều đúng. Thứ nhất là **bản chụp lịch sử** — giá và tên hàng trong dòng đơn, xem Phần 5. Thứ hai là **dữ liệu tra cứu đóng** — màu và cỡ nhúng vào biến thể, vì tách ra chỉ thêm một phép nối mà không được lợi gì.

Nguyên tắc: **chuẩn hoá cho tới khi có lý do cụ thể để không chuẩn hoá**, và lý do đó phải nói ra được.

GV: Đã có kiểm tra ở giao diện lập trình rồi thì cần ràng buộc ở cơ sở dữ liệu làm gì?

Vì hai lớp phục vụ hai mục đích khác nhau. Kiểm tra ở ứng dụng cho **thông báo lỗi dễ hiểu**; ràng buộc ở cơ sở dữ liệu bảo đảm **dữ liệu không bao giờ sai**, kể cả khi có hai yêu cầu đồng thời, khi ai đó chạy script trực tiếp, hoặc khi có thêm một ứng dụng thứ hai ghi vào cùng cơ sở dữ liệu.

Ví dụ cụ thể ở mục 8.2: hai yêu cầu đăng ký cùng email đều vượt qua bước kiểm tra vì cả hai đọc trước khi bên kia ghi.

GV: Đã có khoá duy nhất trên email rồi thì cần khoá chính làm gì nữa?

Vì hai thứ này giải quyết hai bài toán khác nhau. Khoá duy nhất bảo đảm **không trùng**. Khoá chính là **định danh ổn định để các bảng khác tham chiếu tới**.

Email đổi được, mà khoá chính thì không nên đổi vì nó đã được chép sang 21 bảng con. Chi tiết ở mục 2.1.4.

GV: Khoá ngoại làm cơ sở dữ liệu chậm đi, sao vẫn dùng?

Đúng là mỗi lần ghi phải kiểm tra thêm. Nhưng cái giá của việc **không** kiểm tra cao hơn nhiều: dữ liệu mờ côi không phát hiện được cho tới khi có người mở đúng bản ghi đó, và lúc đó thường đã hỏng dây chuyền.

Trong hệ thống này còn một điểm nữa: MongoDB không có khoá ngoại nên nhóm phải tự viết hàm kiểm tra. Chi phí đó nhóm chấp nhận, đổi lại là mọi lượt ghi tạo tham chiếu mờ côi đều bị chặn ngay với mã 409. [THỰC TẾ]

GV: Xoá mềm nghĩa là dữ liệu không bao giờ bị xoá thật à?

Không hẳn. Xoá mềm là để dữ liệu **biến mất khỏi nghiệp vụ nhưng còn để đối soát**. Sau một thời hạn lưu trữ theo quy định, dữ liệu vẫn nên được dọn thật.

Ngoài ra xoá mềm không giải quyết được quyền riêng tư: nếu người dùng yêu cầu xoá dữ liệu cá nhân thì đánh dấu đã xoá là không đủ, phải **ẩn danh hoá** — thay tên, email, số điện thoại bằng giá trị vô danh mà vẫn giữ khoá chính để đơn hàng còn nối được. [ĐỀ XUẤT]

GV: Chỉ mục càng nhiều thì càng nhanh, đúng không?

Không. Xem mục 8.3 — chỉ mục làm đọc nhanh nhưng ghi chậm, và tốn dung lượng. Với bảng nhật ký ghi nhiều đọc ít thì đặt chỉ mục là lỗ vốn.

GV: MongoDB không có lược đồ, vậy vẽ ERD làm gì?

MongoDB không **bắt buộc** lược đồ, nhưng dữ liệu vẫn **có** cấu trúc — chỉ là cấu trúc đó nằm trong mã ứng dụng thay vì trong cơ sở dữ liệu. Không vẽ ra thì mỗi người trong nhóm hiểu một kiểu.

Bằng chứng là hệ thống này vẫn có 21 collection với chỉ mục được khai báo tường minh, trong đó 12 ràng buộc duy nhất. [THỰC TẾ] Đó chính là lược đồ, chỉ là được khai báo ở nơi khác.

PHẦN 14. BUỔI PHẢN BIỆN MÔ PHỎNG

Sáu buổi mô phỏng dưới đây theo đúng cách hội đồng truy đuổi: mỗi câu trả lời sinh ra câu hỏi tiếp theo. Nên đọc thành tiếng để quen nhịp.

14.1. Mô phỏng — hội đồng chỉ vào `users`

GV: Bảng này để làm gì?

Đại diện cho một danh tính có thể đăng nhập. Nó chỉ giữ thứ cần cho xác thực và phân quyền; hồ sơ cá nhân nằm ở bảng profiles riêng.

GV: Sao phải tách profiles ra?

Vì hai nhóm dữ liệu có mức nhạy cảm và tần suất thay đổi khác nhau. Khách đổi ảnh đại diện liên tục, mà thao tác đó không nên chạm vào bản ghi chứa mã băm mật khẩu và vai trò.

GV: Khoá chính là gì, sao không dùng email?

Khoá chính là id. Email là thuộc tính nghiệp vụ, đổi được, và đã bị chép sang 21 bảng con — nếu là khoá chính thì mỗi lần đổi email phải cập nhật dây chuyền 21 nơi.

GV: Thêm id là dư thừa dữ liệu chứ?

Không. Dư thừa là khi cùng một sự thật lưu hai nơi. id và email là hai sự thật khác nhau: định danh nội bộ và địa chỉ liên lạc.

GV: Email có duy nhất không?

Về nghiệp vụ thì phải duy nhất, và mã đăng ký có chặn, trả lỗi 409. Nhưng em xin nói thẳng: chỉ mục trong cơ sở dữ liệu **không** đặt duy nhất. Đây là điểm yếu thật.

GV: Vậy hai người đăng ký cùng lúc cùng email thì sao?

Trong kiến trúc hiện tại thì không xảy ra, vì mọi lượt ghi chạy đồng bộ trên một luồng nên không có yêu cầu nào chen vào giữa bước kiểm tra và bước ghi. Nhưng đó là nhờ kiến trúc chứ không nhờ ràng buộc — chạy hai bản sao máy chủ là hỏng ngay.

GV: Mật khẩu lưu thế nào?

Băm bằng bcrypt 10 vòng, muối nhúng sẵn trong chuỗi kết quả. Không lưu mật khẩu gốc, và cột này không bao giờ trả về qua giao diện lập trình.

GV: Sao không dùng SHA-256 cho nhanh?

Vì nhanh chính là vấn đề. SHA-256 thiết kế để băm nhanh, đúng thứ kẻ tấn công cần khi dò hàng loạt. bcrypt cố tình chậm và chỉnh được độ chậm theo phần cứng.

GV: Xóa tài khoản thì đơn hàng sao?

Hệ thống không có chức năng xóa tài khoản, chỉ khoá bằng cột status. Đó là lựa chọn đúng vì đơn hàng là chứng từ. Nhưng cái còn thiếu là ẩn danh hoá khi khách yêu cầu xóa dữ liệu cá nhân — em ghi nhận là hạn chế.

GV: Một triệu người dùng thì đăng nhập có chậm không?

Đã có chỉ mục trên email nên tra cứu là lôgarit. Nhưng nút thắt thật không nằm ở đó: toàn bộ trạng thái đang giữ trong bộ nhớ một tiến trình, một triệu người dùng thì không nạp hết được và phải đổi kiến trúc sang truy vấn trực tiếp.

14.2. Mô phỏng — hội đồng chỉ vào `products`

GV: Tồn kho để ở đâu?

Ở `product_variants`, không ở `products`. Vì hết cỡ L không có nghĩa là hết áo. 35 sản phẩm sinh ra 317 biến thể, mỗi biến thể có tồn kho riêng.

GV: Sao cần `categoryId`, không lưu thẳng tên danh mục?

Vì tên danh mục là sự thật dùng chung. Chép vào từng sản phẩm thì đổi tên phải sửa hàng nghìn dòng, sót một dòng là mâu thuẫn.

GV: Một sản phẩm thuộc nhiều danh mục thì sao?

Thiết kế hiện tại không hỗ trợ, vì `categoryId` là giá trị đơn. Muốn hỗ trợ phải thêm bảng bắc cầu `product_categories` khoá chính ghép, và bỏ cột `categoryId`.

GV: Giá sản phẩm đổi từ 100 nghìn lên 150 nghìn thì đơn cũ hiện giá nào?

Vẫn 100 nghìn. Vì `order_items` chụp lại giá tại thời điểm đặt, không nối sang `products` để lấy giá hiện tại.

GV: Lưu giá hai chỗ là dư thừa, vi phạm chuẩn 3 chữ?

Không. Dư thừa là cùng một sự thật lưu hai nơi. Đây là hai sự thật tại hai thời điểm: giá hôm nay, và giá khách đã trả hôm đó. Cách phân biệt: nếu bản gốc đổi mà giá trị ở đây **không** phải đổi theo, thì đó là bản chụp lịch sử chứ không phải dư thừa.

GV: Sản phẩm đổi tên thì đơn cũ sao?

Vẫn đúng tên cũ, vì `order_items` cũng chụp `productName`. Ngoài tên còn chụp cả màu, mã màu, cỡ và đường dẫn.

GV: Sản phẩm bị xoá thì `order_items` sao?

Dòng hàng vẫn hiển thị đủ nhờ bản chụp. Cái mất là liên kết dẫn về trang sản phẩm. Về cơ chế, hàm kiểm tra toàn vẹn chặn việc xoá sản phẩm đã từng được đặt, tạo hiệu ứng tương đương RESTRICT — nhưng em xin nói rõ đó là hệ quả chứ nhóm chưa khai báo chính sách xoá tường minh.

GV: Hai người cùng mua cái cuối cùng thì sao?

Bước kiểm tra tồn và bước trừ kho nằm trong cùng một hàm sửa đổi đồng bộ, không có điểm chờ ở giữa, nên không ai chen vào được. Nhóm có kiểm thử 50 lượt mua đồng thời trên tồn kho 5, kết quả đúng 5 lượt thành công.

GV: Có dùng giao dịch không?

Không. Toàn bộ mã nguồn không có lời gọi tạo phiên giao dịch nào. Tính nguyên tử đến từ việc chạy đồng bộ một luồng, và tính chất đó chỉ đúng trong một tiến trình.

GV: Tồn kho có bị âm được không?

Không, vì phép trừ có chặn sản ở 0 và bước kiểm tra đã từ chối trước nếu không đủ. Kiểm thử khẳng định tồn kho về đúng 0 chứ không âm.

14.3. Mô phỏng — hội đồng chỉ vào `orders`

GV: Sao orders và order_items phải tách?

Vì về nghiệp vụ một đơn có nhiều sản phẩm và một sản phẩm nằm trong nhiều đơn — đó là quan hệ nhiều–nhiều. Mô hình quan hệ không biểu diễn trực tiếp được nên phải có bảng ở giữa.

GV: Sao nhiều–nhiều không nối thẳng được?

Vì khoá ngoại chỉ chứa được một giá trị. Đặt ở orders thì mỗi đơn một sản phẩm; đặt ở products thì mỗi sản phẩm một đơn. Cả hai đều sai.

GV: order_items ngoài hai khoá ngoại còn lưu gì?

Số lượng, giá lúc mua, tên hàng, đường dẫn, màu, mã màu, cỡ và thứ tự dòng. Đó là dữ liệu thuộc về dòng hàng — không thuộc đơn cũng không thuộc sản phẩm.

GV: Số lượng thuộc products hay order_items?

order_items. Vì "mua ba cái" là sự thật của dòng hàng này, không phải thuộc tính của sản phẩm.

GV: Giảm giá áp ở đâu, phí giao hàng ở đâu?

Ở mức đơn hàng — orders có subtotal, discount, voucherDiscount, paymentDiscount, vipDiscount, ship và total. Vì mã giảm giá áp cho cả đơn chứ không cho từng dòng.

GV: Sao không tính total khi cần mà phải lưu?

Vì nếu tính lại thì phải tính theo giá và quy tắc **hiện tại**, mà quy tắc đó đã đổi. Tổng tiền là con số khách đã trả nên phải đóng băng, giống như giá dòng hàng.

GV: Đơn bị huỷ thì có xoá không?

Không. Chuyển status sang cancelled. Dữ liệu giao dịch không xoá thật vì còn phải đối soát với cổng thanh toán và giải quyết khiếu nại.

GV: Có chuyển từ completed ngược về pending được không?

Về nghiệp vụ thì không. Hệ thống lưu vết mọi lần đổi trạng thái trong cột history nên dừng lại được đường đi. Nhưng em thừa nhận chưa có bảng quy tắc tập trung chặn mọi bước nhảy sai.

GV: Khách vắng lai không đăng nhập thì đặt hàng được không?

Thiết kế có chừa chỗ: hàm kiểm tra toàn vẹn bỏ qua khi userId mang giá trị khách vắng lai. Trong dữ liệu thật userId có mặt ở 99% bản ghi chứ không phải 100%.

GV: Mã đơn code sinh thế nào, có trùng được không?

Không trùng — có ràng buộc duy nhất trên orders (code). Cách sinh cũ lấy theo số lượng đơn hiện có, và đó là lỗi thật: xoá một đơn cũ thì đơn kế tiếp nhận lại mã đã dùng. Nay lấy mốc cao nhất từng dùng rồi tăng tiếp và dò tới khi chắc chắn chưa ai giữ mã đó.

GV: Khách bấm đặt hàng hai lần thì sao?

clientRequestId khiến lần sau trả lại order đã có trong state. Nhưng MongoDB chưa có unique index cho field này, nên cơ chế chưa đủ khi chạy nhiều backend instance.

GV: MongoDB cho nhúng items, tại sao lại có collection order_items?

Physical source tách collection để index/query theo order hoặc product; khi đọc, hydrateOrders() dựng lại mảng orders.items cho runtime. Đây là một source vật lý với hai representation trong bộ nhớ, không phải hai collection nguồn cùng sửa độc lập.

14.4. Mô phỏng — hội đồng chỉ vào `payments`

GV: Sao thanh toán không để mấy cột trong orders?

Vì một đơn có thể phát sinh nhiều lượt giao dịch: thử lại sau khi thất bại, hoàn tiền từng phần. Một cột chỉ ghi được một lần.

GV: Nhưng dữ liệu hiện tại mỗi đơn chỉ có một giao dịch mà?

Đúng, 94 đơn và 94 giao dịch. Nhưng đó là dữ liệu hiện tại, không phải ràng buộc thiết kế. Bằng chứng: chỉ mục trên `payments.orderId` không đặt duy nhất.

GV: Cổng thanh toán gửi lại thông báo năm lần thì sao?

Stripe/VNPay kiểm trạng thái đã paid, không thêm history trùng theo transaction id, refund được upsert theo refund id, và hoàn kho có `stockRestoredAt`. Provider API còn nhận idempotency key khi tạo payment/refund. Tuy vậy transactionCode chưa unique ở MongoDB.

GV: Nếu cổng báo thành công mà máy chủ em chết đúng lúc đó?

Hai lớp xử lý. Cổng tự gửi lại nhiều lần, và các cờ chống trùng bảo đảm chỉ ghi một lần. Nếu vẫn lệch thì có điểm cuối đối soát chủ động, hỏi ngược cổng về trạng thái thật rồi cập nhật theo câu trả lời của cổng.

GV: Ai là nguồn sự thật về tiền?

Cổng thanh toán, không phải cơ sở dữ liệu của em. Khi hai bên lệch nhau thì tin cổng.

GV: Ghi nhận thanh toán xong nhưng cập nhật đơn hàng hỏng thì sao?

Nếu cả hai nằm trong cùng một hàm sửa đổi thì không xảy ra, vì hệ thống chạy trên bản sao rồi mới thay thế — hỏng giữa chừng thì bản sao bị bỏ, trạng thái gốc chưa bị chạm. Nhưng nếu việc hỏng nằm ở phía cổng bên ngoài thì phải nhờ đối soát.

GV: Có lưu số thẻ ngân hàng không?

Không. Chỉ lưu bốn số cuối, thương hiệu thẻ và mã khách phía cổng. Mã bảo mật thẻ thì không bao giờ được lưu.

GV: Hoàn tiền từng phần lưu ở đâu?

Trong bản ghi thanh toán có mảng các lần hoàn, kèm số tiền đã hoàn và số đang chờ hoàn. Nhờ vậy khách trả lại một trong ba món thì hoàn đúng phần đó mà vẫn giữ nguyên lịch sử.

GV: Trạng thái thanh toán có bao nhiêu giá trị?

Sáu: chờ, chưa trả, đã trả, đã hoàn, thất bại, đã hủy.

GV: Nếu khách bấm tải lại trang thanh toán giữa chừng?

Mã yêu cầu từ ứng dụng bảo đảm không tạo đơn thứ hai; nếu đã có đơn với cùng mã yêu cầu thì trả về đơn cũ.

GV: transactionCode có unique ở database không?

Chưa. Live Mongo chỉ có index trên id, orderId, userId; đây là điểm yếu nếu nhiều instance cùng xử lý một giao dịch provider.

GV: Hoàn tiền một phần có làm cả order thành returned không?

Không. Refund được cộng theo từng entry; order chỉ chuyển returned khi toàn bộ amount/items đã hoàn. Partial refund giữ order ở trạng thái phù hợp để lịch sử không sai.

14.5. Mô phỏng — hội đồng chỉ vào một bảng khó: `voucher\_redemptions`

GV: Bảng này để làm gì?

Ghi lại từng lượt một phiếu giảm giá được sử dụng: ai dùng, cho đơn nào, lúc nào.

GV: Sao không để một bộ đếm trong vouchers?

Vì bộ đếm chỉ trả lời "đã dùng mấy lần". Nghiệp vụ cần biết thêm ai dùng, cho đơn nào, lúc nào — để chặn một người dùng lại phiếu giới hạn mỗi người một lần, và để đối soát khi khách khiếu nại.

GV: Bảng này thuộc loại dữ liệu gì?

Vừa bắc cầu vừa giao dịch. Nó nối vouchers, users và orders, nhưng đồng thời ghi lại một sự kiện đã xảy ra nên không được xoá tùy tiện.

GV: Nó có mấy khoá ngoại?

Ba: voucherId, userId và orderId. Cả ba đều có chỉ mục riêng, đúng ba chiều tra cứu mà nghiệp vụ cần.

GV: Sao không đặt khoá chính ghép ba cột đó luôn?

Vì như vậy sẽ ngăn cấm cùng một người dùng cùng một phiếu cho cùng một đơn hai lần — nghe hợp lý, nhưng nó không phải quy tắc nghiệp vụ mà em muốn khoá cứng ở mức khoá chính. Dùng khoá thay thế rồi đặt ràng buộc duy nhất riêng thì linh hoạt hơn khi quy tắc đổi.

GV: Phiếu hết hạn đúng lúc khách bấm thanh toán thì sao?

Hạn phiếu được kiểm tra ở máy chủ tại thời điểm tạo đơn, không phải lúc khách mở giỏ hàng. Nên nếu hết hạn giữa chừng thì đơn bị từ chối kèm thông báo, chứ không tạo đơn với mức giảm không còn hiệu lực.

GV: Nếu đơn bị huỷ thì lượt sử dụng phiếu có trả lại không?

Đây là chỗ em phải trả lời trung thực: hệ thống chưa xử lý tự động việc hoàn lại lượt dùng phiếu khi đơn bị huỷ. Cùng nhóm với hạn chế không tự hoàn kho cho đơn trực tuyến thất bại.

GV: Bảng này có được kiểm tra toàn vẹn không?

Không. Hàm kiểm tra toàn vẹn hiện chỉ phủ khoảng chín nhóm bảng con có giá trị nghiệp vụ cao nhất, và voucher_redemptions không nằm trong đó. Em thừa nhận đây là chỗ chưa đầy đủ.

GV: Ba index voucherId, userId, orderId có chống dùng trùng không?

Không. Chúng chỉ tăng tốc tra cứu. Muốn chống lặp phải có unique composite đúng theo policy và xử lý duplicate-key ở tầng ứng dụng.

GV: Nếu order huỷ thì xoá redemption được không?

Không nên xoá mất audit. Nên ghi trạng thái reverse/released và cập nhật ledger theo cùng một transaction hoặc thao tác idempotent.

14.6. Mô phỏng — kiến trúc database, consistency và scale

GV: MongoDB không có schema, tại sao vẫn vẽ ERD?

MongoDB không bắt buộc schema ở database, nhưng dữ liệu vẫn có cấu trúc, logical refs và indexes. ERD giúp cả nhóm thống nhất invariant và vòng đời dữ liệu.

GV: Năm mươi đường nối có phải foreign key constraint không?

Không. Đó là logical FK trong ERD. MongoDB không tự enforce; một phần được `relationshipErrors()` kiểm tra ở application.

GV: Kiểm tra đó có đủ hết không?

Không. Nó phủ các nhóm lỗi như product/order/payment/review/VIP và một số bảng theo user; telemetry, notification, voucher redemption và vài quan hệ khác chưa được phủ.

GV: Nếu MongoDB lỗi và fallback JSON thì sao?

`writeFile()` không gọi `assertValid()`, nên enforcement khác Mongo mode. Đây là điểm yếu thật của degraded mode.

GV: Hệ thống có transaction MongoDB không?

Không có `startSession` hay `withTransaction`. Một process dựa vào synchronous mutator trên bản clone, không được gọi đó là database transaction.

GV: Vay tính nguyên tử hiện tại mạnh tới đâu?

Mutator throw thì `write()` không chạy, nên state gốc trong process không đổi. Nhưng external provider, nhiều process và durable persistence nằm ngoài phạm vi bảo đảm đó.

GV: Response trả rồi dữ liệu đã bền chưa?

Chưa chắc. Mongo persistence được debounce 40 ms và chạy async; write liên tục còn reset timer. Có thể còn nhiều thay đổi chưa flush.

GV: Muốn chạy ba backend instance thì đối đầu tiên ở đâu?

Bỏ whole-state-in-memory làm nguồn điều phối; query/update collection trực tiếp, dùng unique constraint, conditional update và transaction ở invariant đa document.

GV: Index càng nhiều càng tốt đúng không?

Không. Phải gắn với query. Ví dụ `orders(userId, createdAt)` phục vụ lịch sử đơn; thêm index vào event log làm write chậm và tốn RAM/disk.

GV: Nếu làm production, ba ưu tiên dữ liệu là gì?

Một: DB constraints cho email/review/idempotency. Hai: atomic stock/payment operations. Ba: durable-before-ack cùng outbox/reconcile cho side effects và provider.

PHẦN 15. TỰ CHẤM NHƯ MỘT GIẢNG VIÊN PHẢN BIỆN

Bảng dưới đây chấm thiết kế hiện tại theo góc nhìn phản biện, kèm lý do. Biết trước điểm yếu của mình là cách phòng thủ tốt nhất.

Bảng 15.1: Tự chấm theo mười một tiêu chí

Tiêu chí	Điểm	Nhận xét
Hiệu nghiệp vụ	9/10	Mô hình phản ánh đúng bài toán bán thời trang: biến thể màu–cỡ, đánh giá phải có đơn, phiếu giảm giá có lượt dùng. Trừ điểm vì hai thực thể hình ảnh trùng vai trò.
Thiết kế thực thể	8/10	Tách bảng hợp lý theo vòng đời dữ liệu. Trừ điểm vì Hình Ảnh thừa và hai bảng tra cứu màu–cỡ trên sơ đồ không có thật khi cài đặt.
Khoá chính	9/10	Dùng khoá thay thế nhất quán, tách định danh khỏi dữ liệu nghiệp vụ. Không có chỗ nào dùng thuộc tính nghiệp vụ làm khoá chính.
Khoá ngoại	7/10	Đặt đúng phía trong mọi quan hệ. Trừ điểm vì việc thực thi chỉ phủ khoảng chín nhóm bảng và không chạy trên nhánh ghi tệp dự phòng.
Bản số quan hệ	9/10	Xác định đúng, kể cả chỗ tinh tế: đơn–thanh toán để mở cho một–nhiều thay vì khoá cứng một–một.
Chuẩn hoá	9/10	Chuẩn hoá dữ liệu gốc, cố ý không chuẩn hoá chứng từ. Bảy nhóm dữ liệu chụp lại đều có lý do rõ ràng.
Toàn vẹn dữ liệu	6/10	Có hàm kiểm tra thật và chặn được, nhưng phủ chưa hết, thiếu ràng buộc duy nhất trên email và trên cặp người–sản phẩm của đánh giá.
Giao dịch	6/10	Đạt tính nguyên tử thật trong một tiến trình và có kiểm thử chứng minh. Trừ điểm vì không dùng giao dịch cơ sở dữ liệu nên không mở rộng ngang được.
Chỉ mục và hiệu năng	8/10	21 collection có chỉ mục, đều gắn với truy vấn thật. Chỉ mục ghép <code>orders(userId, createdAt)</code> cho thấy hiểu đúng.
Khả năng mở rộng	5/10	Toàn bộ trạng thái trong bộ nhớ một tiến trình là trần cứng. Có hướng mở rộng rõ nhưng chưa triển khai.
Khả năng bảo vệ	9/10	Gần như mọi quyết định đều truy được về một quy tắc nghiệp vụ, và các điểm yếu đều xác định được kèm cách sửa.

Tổng: 85/110. Điểm trừ tập trung ở toàn vẹn, giao dịch và khả năng mở rộng — cả ba đều là hệ quả của cùng một quyết định kiến trúc: giữ trạng thái trong bộ nhớ một tiến trình. Nếu hội đồng hỏi *"nếu làm lại thì em đổi gì"*, câu trả lời tốt nhất là đổi đúng chỗ đó.

PHẦN 16. NĂM MƯƠI CÂU HỎI THEO CẤP ĐỘ

Câu hỏi được xếp theo năm cấp. Cấp 1 và 2 phải trả lời được ngay không cần nghĩ. Cấp 4 và 5 là chỗ phân loại điểm.

Cấp 1 — Nhận biết

#	Câu hỏi	Trả lời ngắn
1	Sơ đồ có bao nhiêu bảng và bao nhiêu quan hệ?	36 thực thể logic, 50 quan hệ, 407 dòng cột.
2	Bảng nào là trung tâm?	Người Dùng nhận 21 quan hệ, Sản Phẩm nhận 14. Hai bảng chiếm 35 trên 50.
3	Khoá chính của users là gì?	Cột id, kiểu chuỗi.
4	order_items có mấy khoá ngoại?	Hai: orderId và productId.
5	return_requests có mấy khoá ngoại?	Ba: userId, orderId và paymentId, trong đó paymentId là tùy chọn.
6	Đơn hàng có bao nhiêu trạng thái?	Bảy: chờ, chờ thanh toán, đã xác nhận, đang giao, đã nhận, đã huỷ, đã trả.
7	Có bao nhiêu vai trò người dùng?	Bốn, xếp thứ bậc: khách, nhân viên, quản trị, quản trị cấp cao.
8	Tồn kho lưu ở bảng nào?	product_variants, không phải products.

Cấp 2 — Giải thích

#	Câu hỏi	Trả lời ngắn
1	Vì sao khoá ngoại nằm ở phía nhiều?	Vì phía nhiều chỉ trở về một bản ghi nên chỉ cần một ô. Đặt ngược lại thì phải chứa danh sách, vi phạm chuẩn 1.
2	Vì sao orders và order_items phải tách?	Vì đơn và sản phẩm là quan hệ nhiều–nhiều, và dòng hàng còn mang dữ liệu riêng của nó.
3	Vì sao tồn kho ở biến thể?	Vì hết cỡ L không có nghĩa là hết áo. Tồn kho phải ở mức người ta lấy hàng khỏi kệ.
4	Vì sao products dùng categoryId chứ không lưu tên danh mục?	Tránh bất thường khi sửa, khi thêm và khi xoá. Đổi tên danh mục chỉ sửa một bản ghi.
5	Vì sao địa chỉ tách bảng riêng?	Vì một người có nhiều địa chỉ, và sửa đề sẽ làm sai lịch sử giao hàng của đơn cũ.
6	Vì sao profiles tách khỏi users?	Vì hai nhóm dữ liệu khác mức nhạy cảm; đối ảnh đại diện không nên chạm vào bản ghi có mặt khẩu.
7	Vì sao thanh toán là bảng riêng?	Vì một đơn có thể có nhiều lượt giao dịch, và giao dịch mang dữ liệu riêng của cổng thanh toán.
8	Vì sao đánh giá phải nối tới đơn hàng?	Để xác minh người viết đã thực sự mua — chống đánh giá giả.
9	Vì sao cart_items ràng buộc bốn cột?	Vì cùng một áo khác màu hoặc khác cỡ là hai dòng giỏ hàng hợp lệ.
10	Vì sao dùng khoá chuỗi chứ không phải số tự tăng?	MongoDB không có tự tăng dựng sẵn; khoá chuỗi sinh được ở phía ứng dụng, cần thiết khi chạy nhiều bản sao.

Cấp 3 — Tình huống thực tế

#	Câu hỏi	Trả lời ngắn
1	Giá sản phẩm đổi thì đơn cũ hiện giá nào?	Giá cũ, vì <code>order_items.price</code> là bản chụp.
2	Sản phẩm đổi tên thì đơn cũ sao?	Vẫn tên cũ, vì có chụp <code>productName</code> .
3	Sản phẩm bị xoá thì đơn cũ sao?	Vẫn hiển thị đủ nhờ bản chụp; chỉ mất liên kết về trang sản phẩm.
4	Người dùng xoá tài khoản thì đơn sao?	Hệ thống không có chức năng xoá, chỉ khoá. Thiếu ẩn danh hoá là hạn chế.
5	Người dùng đổi email thì khoá ngoại sao?	Không ảnh hưởng, vì bảng con trỏ về <code>id</code> chứ không phải email.
6	Khách đổi địa chỉ mặc định thì đơn cũ sao?	Không đổi, vì đơn chụp địa chỉ tại thời điểm đặt.
7	Hai người cùng mua món cuối cùng?	Một process serialize check-decrement và unit test 50/5 đạt; nhiều instance vẫn cần atomic update phía MongoDB.
8	Cổng thanh toán gửi lại thông báo nhiều lần?	Handler kiểm <code>paid/transaction/refund id</code> và cờ hoàn kho; nhưng <code>transactionCode</code> chưa unique ở DB.
9	Khách bấm đặt hàng hai lần?	Mã yêu cầu từ ứng dụng khiến lần hai trả về đơn đã tạo.
10	Khách trả lại một trong ba món?	Hoàn tiền từng phần, ghi vào mảng các lần hoàn trong bản ghi thanh toán.
11	Phiếu giảm giá hết hạn đúng lúc thanh toán?	Kiểm tra hạn ở máy chủ lúc tạo đơn, nên đơn bị từ chối chứ không tạo với mức giảm hết hiệu lực.
12	Đơn trực tuyến thất bại thì kho có được hoàn?	Có khi handler xác nhận <code>failed/cancelled/expired</code> ; order treo pending vẫn cần reconcile/timeout.

Cấp 4 — Phản biện

#	Câu hỏi	Trả lời ngắn
1	Tách nhiều bảng, nối nhiều, có phải thiết kế kém?	Không. Gộp bảng không xoá độ phức tạp mà chuyển nó sang mã ứng dụng, chỗ cơ sở dữ liệu không giúp được.
2	Chuẩn 3 lúc nào cũng tốt?	Không. Bản chụp lịch sử và dữ liệu tra cứu đóng là hai chỗ cố ý không chuẩn hoá.
3	Lưu giá hai chỗ là dư thừa?	Không. Dư thừa là cùng một sự thật hai nơi; đây là hai sự thật tại hai thời điểm.
4	Ứng dụng kiểm tra rồi, cần ràng buộc cơ sở dữ liệu làm gì?	Vì giữa lúc đọc và lúc ghi có khoảng trống cho yêu cầu khác chen vào.
5	Nếu bổ sung UNIQUE email rồi, còn cần khoá chính <code>id</code> không?	Có. UNIQUE chống trùng; <code>id</code> là định danh ổn định để tham chiếu khi email đổi.
6	Khoá ngoại làm chậm, sao vẫn dùng?	Vì cái giá của dữ liệu mờ côi cao hơn nhiều, và thường chỉ phát hiện khi đã hỏng dây chuyền.
7	Chỉ mục càng nhiều càng nhanh?	Không. Đọc nhanh hơn nhưng ghi chậm hơn và tốn dung lượng.
8	Xoá mềm nghĩa là không bao giờ xoá thật?	Không. Sau thời hạn lưu trữ vẫn nên dọn, và quyền riêng tư cần ẩn danh hoá chứ không phải đánh dấu.
9	MongoDB không lược đồ thì vẽ ERD làm gì?	Vì dữ liệu vẫn có cấu trúc, chỉ là được khai báo trong mã. Bảng chứng: 21 collection có chỉ mục tường minh.
10	Sao không dùng cơ sở dữ liệu quan hệ cho đúng bài?	Vì dữ liệu sản phẩm có cấu trúc biến động — biến thể, thẻ, phương tiện. Đánh đổi là phải tự thực thi toàn vẹn, và nhóm chấp nhận đánh

		đối đó một cách có ý thức.
--	--	----------------------------

Cấp 5 — Khó

#	Câu hỏi	Trả lời ngắn
1	Hệ thống có dùng giao dịch không?	Không. Toàn bộ mã nguồn không có lời gọi tạo phiên giao dịch. Nguyên tử đến từ chạy đồng bộ một luồng, chỉ đúng trong một tiến trình.
2	Nếu bước bốn trong luồng đặt hàng hỏng thì sao?	Bản sao bị bỏ, trạng thái gốc chưa bị chạm — tương đương quay lui. Nhưng chỉ đúng khi việc hỏng nằm trong hàm sửa đổi.
3	Muốn chạy nhiều máy chủ thì phải đổi gì?	Chuyển sang cập nhật có điều kiện, khoá lạc quan, hoặc giao dịch thật. Và bỏ mô hình giữ toàn bộ trạng thái trong bộ nhớ.
4	Mức cô lập nào đang dùng?	Không áp dụng, vì không có song song thật trong một tiến trình một luồng. Khi chuyển sang nhiều bản sao thì mới phải chọn.
5	Toàn vẹn tham chiếu có phủ hết không?	Không. Chỉ khoảng chín nhóm bảng con, và không chạy trên nhánh ghi tệp dự phòng.
6	Dữ liệu có thể mất không?	Có. Response có thể đi trước durable persist; debounce reset theo write nên không thể giới hạn là chỉ một lượt cuối.
7	Một triệu người dùng thì gây ở đâu trước?	Không phải chỉ mục, mà là việc giữ toàn bộ trạng thái trong bộ nhớ một tiến trình.
8	Bảng nào phình nhanh nhất?	interactions — mỗi lượt xem một bản ghi. Cần gộp sẵn theo ngày và dọn bản ghi cũ.
9	Nếu làm lại thì đổi gì đầu tiên?	Bỏ mô hình trạng thái trong bộ nhớ, chuyển sang lớp truy cập dữ liệu theo từng bảng với cập nhật có điều kiện. Ba điểm trừ lớn nhất đều từ quyết định đó.
10	Điểm mạnh nhất của thiết kế là gì?	Bản chụp dữ liệu ở dòng đơn hàng — chụp năm thuộc tính nên đơn cũ luôn đúng dù sản phẩm đổi giá, đổi tên hay bị xoá.

PHẦN 17. TRANG TRA NHANH TRƯỚC KHI VÀO PHÒNG

17.1. Nếu hội đồng chỉ vào bảng này thì nói câu này trước

Bảng 17.1: Câu mở đầu cho từng bảng

Bảng	Câu mở đầu	Để bị hỏi tiếp
users	Danh tính đăng nhập và phân quyền; hồ sơ cá nhân nằm ở bảng riêng.	Sao không dùng email làm khoá chính · Email có duy nhất không
products	Mặt hàng theo cách khách nghĩ; thứ lấy khỏi kho là biến thể.	Tồn kho ở đâu · Đổi giá thì đơn cũ sao
product_variants	Một tổ hợp màu và cỡ cụ thể — đơn vị thật của kho.	Ràng buộc ghép ba cột · Giá riêng dùng khi nào
orders	Chứng từ ghi lại một lần mua, đóng băng dữ liệu tại thời điểm đặt.	Sao tách order_items · Trạng thái chuyển ngược được không
order_items	Dòng hàng — giải quyết nhiều–nhiều và giữ bản chụp giá, tên, màu, cỡ.	Lưu giá hai chỗ có dư thừa không
payments	Một lượt giao dịch tiền; một đơn có thể có nhiều lượt.	Sao không gộp vào đơn · Gửi lại thông báo nhiều lần thì sao
reviews	Đánh giá có kèm bằng chứng đã mua qua orderId.	Chưa mua có đánh giá được không · Chặn trùng bằng gì
wishlist_items	Bảng bắc cầu, ràng buộc duy nhất trên cặp người và sản phẩm.	Sao không lưu mảng trong users
cart_items	Bảng bắc cầu, ràng buộc duy nhất trên bốn cột.	Sao phải bốn cột chứ không phải hai
categories	Danh mục tách riêng để tránh bất thường khi sửa và khi xoá.	Một sản phẩm nhiều danh mục thì sao
addresses	Một người nhiều địa chỉ; đơn hàng chụp lại chứ không nối.	Sao không để cột trong users
return_requests	Ba khoá ngoại là ba câu hỏi: ai, đơn nào, hoàn qua giao dịch nào.	Vì sao paymentId được rỗng
vouchers	Phiếu giảm giá; hai đường nối tới người dùng là người nhận và người cấp.	Sao có hai dây tới users
voucher_redemptions	Từng lượt sử dụng phiếu, không phải bộ đếm.	Sao không để usedCount

17.2. Mười con số phải nhớ

Bảng 17.2: Con số cần thuộc

36 / 35	thực thể logic / collection vật lý
50	quan hệ trên sơ đồ
21 và 14	số dây vào Người Dùng và Sản Phẩm — chiếm 35 trên 50
35 → 317	sản phẩm sinh ra biến thể, trung bình 9 biến thể một sản phẩm
21	collection có chỉ mục ngoài khoá chính

12	ràng buộc duy nhất thật
4	vai trò xếp thứ bậc
0	lời gọi tạo phiên giao dịch trong toàn bộ mã nguồn
50 trên tồn 5	kiểm thử đồng thời — đúng 5 lượt thành công
40 mili giây	thời gian debounce tối thiểu trước persist; write mới reset timer

17.3. Ba câu cứu nguy khi bí

KHI KHÔNG BIẾT CÂU TRẢ LỜI

"Chỗ này em chưa kiểm chứng nên không dám khẳng định. Theo cách cài đặt hiện tại thì em nghĩ là... , nhưng để chắc chắn thì phải xem lại phần... trong mã nguồn."

Đoán bừa là cách mất điểm nhanh nhất. Nói không biết rồi chỉ ra chỗ sẽ tra là câu trả lời chuyên nghiệp.

KHI BỊ CHỈ ĐÚNG ĐIỂM YẾU

"Ở phiên bản hiện tại nhóm triển khai theo cách này. Khi rà soát lại chúng em nhận thấy nó có hạn chế ở tình huống thầy cô vừa nêu. Thiết kế tốt hơn là... , và lý do nhóm chưa làm là... "

KHI BỊ HỎI VỀ PHẠM VI CHƯA LÀM

"Thiết kế hiện tại đáp ứng đúng phạm vi nghiệp vụ của phiên bản này. Nếu mở rộng sang... thì nhóm sẽ cần thêm bảng... với khoá chính ghép... , và đây là thay đổi một lần chuyển dữ liệu chứ không phải viết lại hệ thống."

PHẦN 18. DEFENSE CARD CHO TOÀN BỘ 36 THỰC THỂ

Phần 2–3 đào sâu các bảng trung tâm. Phần này bảo đảm hội đồng chỉ **bất kỳ** bảng nào cũng có một hồ sơ ngăn theo cùng khung: vai trò, lý do tồn tại, PK/field, nullable/index, relationship, delete, tình huống, điểm yếu và hướng mở rộng. Thông tin field/index lấy từ snapshot MongoDB thật ngày 18/08/2026; quan hệ lấy từ `erd.drawio`.

18.1. Danh Mục Sản Phẩm

Defense Card 01/36 — Danh Mục Sản Phẩm

Hạng · physical	A · categories — 5 document
Vai trò · nếu bỏ	Nhóm phân loại dùng chung để duyệt catalog và đổi tên tập trung. Nếu bỏ: Tên danh mục bị lặp trong products và phát sinh update/insert/delete anomaly.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã danh mục · Tên danh mục · Tên tiếng Nhật
NULL · UNIQUE · INDEX	Có mặt 100%: id, name, kanji. Tùy chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: id UNIQUE
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: Sản Phẩm qua <code>products.categoryId</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	RESTRICT khi còn product, hoặc reassign trước khi xoá. Tình huống: Đổi tên danh mục một lần, mọi product theo <code>categoryId</code> thấy tên mới.
Điểm yếu · truy đuổi · mở rộng	Yếu: Thiết kế phẳng, chưa có parent-child; mỗi product chỉ có một category. GV: Nếu một product nhiều category, migration hiện tại ảnh hưởng gì? [ĐỀ XUẤT] Thêm <code>parentId</code> tự tham chiếu hoặc bảng <code>product_categories</code> cho N:N.

18.2. Sản Phẩm

Defense Card 02/36 — Sản Phẩm

Hạng · physical	S · products — 35 document
Vai trò · nếu bỏ	Mặt hàng theo cách khách tìm kiếm: tên, slug, danh mục, giá chung và trạng thái. Nếu bỏ: Catalog, tìm kiếm, wishlist, cart, review và dòng đơn đều mất đối tượng trung tâm.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã sản phẩm · Mã danh mục · Đường dẫn định danh · Tên sản phẩm · Tên tiếng Nhật · Mã quản lý kho · Thương hiệu · Giá bán hiện tại · Giá niêm yết
NULL · UNIQUE · INDEX	Có mặt 100%: id, slug, name, sku, brand, price, status, categoryId. Tùy chọn/không phủ hết sample: kanji(97%), createdAt(97%), updatedAt(3%). Index thật: id UNIQUE · slug UNIQUE · categoryId index
Nối · FK owner · bản số	Đi ra: <code>products.categoryId</code> → Danh Mục Sản Phẩm (category 1 → N products). Nhận vào: Chi Tiết Sản Phẩm qua <code>product_details.productId</code> ; Biến Thể Sản Phẩm qua <code>product_variants.productId</code> ; Hình Ảnh qua <code>legacy_image.productId</code> ; Mô Tả Sản Phẩm Tạo Bởi AI qua <code>ai_descriptions.productId</code> ; Chi Tiết Đơn Hàng qua <code>order_items.productId</code> ; Chi Tiết Giỏ Hàng qua <code>cart_items.productId</code> ; Danh Sách Sản Phẩm Yêu Thích qua <code>wishlist_items.productId</code> ; Mục Tiêu Tiết Kiệm qua <code>goals.productId</code> ;

	Tin Nhắn qua chats.productId; Tương Tác Người Dùng qua interactions.productId; Lịch Sử Thử Đồ qua tryon_history.productId; Đánh Giá Sản Phẩm qua reviews.productId; Hình Ảnh Giao Diện Sản Phẩm qua product_media.productId; Thẻ Địa Danh qua flagcards.recommendedProductIds[]. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Ưu tiên archive; dữ liệu chi tiết có thể cascade, chứng từ phải giữ snapshot. Tình huống: Giá/tên hiện tại đổi nhưng dòng đơn cũ vẫn giữ giá/tên tại lúc mua.
Điểm yếu · truy đuổi · mở rộng	Yếu: Xoá thật bị toàn vẹn chặn khi đã có dòng đơn; chính sách vòng đời chưa được khai báo bằng deletedAt. GV: Vì sao slug unique nhưng không dùng làm PK? [ĐỀ XUẤT] Soft-delete/archived; thêm product_categories nếu một sản phẩm thuộc nhiều danh mục.

18.3. Chi Tiết Sản Phẩm

Defense Card 03/36 — Chi Tiết Sản Phẩm

Hạng · physical	B · product_details — 35 document
Vai trò · nếu bỏ	Khối mô tả dài, câu chuyện, tags, rating và sold tách khỏi lõi catalog. Nếu bỏ: Product bị phình hoặc mất nội dung marketing/thuộc tính phục vụ gợi ý.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã chi tiết sản phẩm · Mã sản phẩm · Mô tả sản phẩm · Câu chuyện sản phẩm · Mã màu đại diện · Thẻ từ khóa · Thẻ nhận diện hình ảnh · Điểm đánh giá trung bình · Số lượng đã bán
NULL · UNIQUE · INDEX	Có mặt 100%: productId, description, story, colorHex, tags, visualTags, rating, sold. Tuỳ chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: productId UNIQUE
Nối · FK owner · bản số	Đi ra: product_details.productId → Sản Phẩm (product 1 → 0..1 detail). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Sống theo product; có thể cascade khi product chưa từng giao dịch bị xoá. Tình huống: Admin sửa câu chuyện sản phẩm mà không làm thay đổi mã, giá hay tồn kho.
Điểm yếu · truy đuổi · mở rộng	Yếu: rating/sold là dữ liệu dẫn xuất, có nguy cơ lệch nếu cập nhật không đồng bộ. GV: Nếu rating tính từ reviews, tại sao còn lưu rating? [ĐỀ XUẤT] Tính lại định kỳ hoặc dùng pipeline sự kiện; version nội dung nếu cần audit.

18.4. Biến Thể Sản Phẩm

Defense Card 04/36 — Biến Thể Sản Phẩm

Hạng · physical	S · product_variants — 317 document
Vai trò · nếu bỏ	Một tổ hợp màu-cỡ/SKU cụ thể, là đơn vị thật để định giá riêng và giữ tồn kho. Nếu bỏ: Không biết size/màu nào còn hàng; hết L bị hiểu sai thành hết cả sản phẩm.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã biến thể · Mã sản phẩm · Mã màu sắc · Mã kích thước · Mã SKU biến thể · Số lượng tồn kho · Giá riêng (tuỳ chọn) · Thứ tự hiển thị
NULL · UNIQUE · INDEX	Có mặt 100%: colorName, colorHex, size, sku, stock, id, productId, position. Tuỳ chọn/không phủ hết sample: price(1%). Index thật: id UNIQUE · productId index · sku UNIQUE · productId+colorName+size

	UNIQUE
Nối · FK owner · bản số	Đi ra: <code>product_variants.productId</code> → Sản Phẩm (product 1 → N variants); <code>product_variants.colorName/colorHex</code> → Màu Sắc (color 1 → N variants (logic)); <code>product_variants.size</code> → Kích Thước (size 1 → N variants (logic)). Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Cascade theo product nếu chưa có tham chiếu giao dịch; lịch sử vẫn do <code>order_items</code> snapshot. Tình huống: Áo size M còn 5 nhưng size L còn 0; chỉ L phải báo hết.
Điểm yếu · truy đuổi · mở rộng	Yếu: An toàn check-then-decrement chỉ được kiểm thử trong một process, chưa an toàn nhiều instance. GV: Hai backend cùng mua món cuối cùng thì thao tác nào phải atomic? [ĐỀ XUẤT] Atomic conditional update theo <code>_id/stock</code> hoặc transaction; thêm <code>variant_stock</code> khi nhiều kho.

18.5. Hình Ảnh

Defense Card 05/36 — Hình Ảnh

Hạng · physical	B · Không có collection vật lý độc lập
Vai trò · nếu bỏ	Thực thể logic cũ cho ảnh sản phẩm, hiện trùng chức năng với <code>product_media</code> . Nếu bỏ: Bỏ riêng thực thể này không làm hỏng runtime vì không có collection tương ứng.
PK · cột trọng yếu	Logical PK có trên ERD; physical value đang được nhúng hoặc thực thể là phần dư thiết kế. ERD: Mã hình ảnh · Mã sản phẩm · Đường dẫn hình ảnh · Thứ tự · Hình ảnh chính
NULL · UNIQUE · INDEX	Field logic nằm trên ERD; physical implementation không có collection độc lập. Index thật: Không có collection vật lý; xem quyết định nhúng/trùng vai trò.
Nối · FK owner · bản số	Đi ra: <code>legacy_image.productId</code> → Sản Phẩm (product 1 → N images (logic)). Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Loại khỏi mô hình logic sau khi migration; không có dữ liệu runtime để cascade. Tình huống: Giảng viên chỉ vào hai bảng ảnh và hỏi tại sao trùng.
Điểm yếu · truy đuổi · mở rộng	Yếu: Đây là dư thừa thật trên ERD, không nên cố bảo vệ như hai nguồn dữ liệu độc lập. GV: Tại sao ERD có hai bảng ảnh nhưng MongoDB chỉ có một collection? [ĐỀ XUẤT] Gộp vào Hình Ảnh Giao Diện Sản Phẩm và dùng trường <code>type=image video</code> .

18.6. Mô Tả Sản Phẩm Tạo Bởi AI

Defense Card 06/36 — Mô Tả Sản Phẩm Tạo Bởi AI

Hạng · physical	C · ai_descriptions — 10 document
Vai trò · nếu bỏ	Lưu kết quả và dấu vết mô tả AI theo sản phẩm để tái sử dụng/audit. Nếu bỏ: Phải gọi model lại mỗi lần hoặc không truy được nội dung đã sinh cho product nào.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ <code>id</code> làm domain/API identity ổn định. ERD: Mã bản ghi mô tả AI · Mã sản phẩm · Thời gian khởi tạo · Chi tiết nội dung mô tả
NULL · UNIQUE · INDEX	Có mặt 100%: <code>productId</code> , <code>generatedAt</code> , <code>description</code> , <code>id</code> . Tuỳ chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>ai_descriptions.productId</code> → Sản Phẩm (product 1 → N generations). Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một

	cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Cascade hoặc archive cùng product, tùy yêu cầu audit nội dung. Tình huống: Admin sinh lại mô tả sau khi đối ảnh và cần biết kết quả cũ thuộc model/lần nào.
Điểm yếu · truy đuổi · mở rộng	Yếu: Không nằm trong <code>relationshipErrors()</code> ; retention/version nguồn sinh chưa chặt. GV: AI sinh sai thì source of truth là trường nào? [ĐỀ XUẤT] Thêm <code>modelVersion</code> , <code>promptVersion</code> , <code>approvalStatus</code> và <code>retention</code> .

18.7. Đơn Hàng

Defense Card 07/36 — Đơn Hàng

Hạng · physical	S · orders — 94 document
Vai trò · nếu bỏ	Chứng từ một lần mua, đóng băng khách, địa chỉ, giảm giá, tổng tiền và lịch sử trạng thái. Nếu bỏ: Không có chứng từ để giao hàng, đối soát, hoàn tiền hay chứng minh giá đã chốt.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã đơn hàng · Mã người dùng · Mã phiếu giảm giá · Mã hiển thị đơn hàng · Thông tin người nhận · Địa chỉ giao hàng · Tổng giá trị đơn hàng · Phí vận chuyển · Trạng thái đơn hàng
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>code</code> , <code>customer</code> , <code>address</code> , <code>total</code> , <code>ship</code> , <code>status</code> , <code>createdAt</code> . Tùy chọn/không phủ hết sample: <code>userId</code> (99%), <code>subtotal</code> (61%), <code>discount</code> (61%), <code>discountCode</code> (61%), <code>flagcardAward</code> (5%), <code>addressDetails</code> (57%). Index thật: <code>id</code> UNIQUE · <code>code</code> UNIQUE · <code>userId+createdAt</code> index
Nối · FK owner · bản số	Đi ra: <code>orders.userId</code> → Người Dùng (user 1 → N orders); <code>orders.voucherId</code> → Phiếu Giảm Giá (voucher 1 → 0..N orders). Nhận vào: Chi Tiết Đơn Hàng qua <code>order_items.orderId</code> ; Thanh Toán qua <code>payments.orderId</code> ; Yêu Cầu Trả Hàng qua <code>return_requests.orderId</code> ; Lướt Sử Dụng Phiếu qua <code>voucher_redemptions.orderId</code> ; Đánh Giá Sản Phẩm qua <code>reviews.orderId</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Không hard-delete; huỷ bằng status và giữ history. Tình huống: Double-click dùng <code>clientRequestId</code> ; đổi giá sau mua không đổi tổng đơn cũ.
Điểm yếu · truy đuổi · mở rộng	Yếu: <code>clientRequestId</code> chưa có unique index DB; state lưu trong memory và ghi Mongo sau debounce 40 ms. GV: Nếu process chết trước lượt flush Mongo thì những write nào có thể mất? [ĐỀ XUẤT] Unique theo (<code>userId</code> , <code>clientRequestId</code>); state machine tập trung; write-through/transaction.

18.8. Chi Tiết Đơn Hàng

Defense Card 08/36 — Chi Tiết Đơn Hàng

Hạng · physical	S · order_items — 154 document
Vai trò · nếu bỏ	Dòng hàng nối order-product và giữ qty, giá, tên, slug, màu, cỡ tại lúc mua. Nếu bỏ: Quan hệ N:N không có chỗ lưu số lượng/giá lịch sử; đơn cũ phụ thuộc catalog hiện tại.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã chi tiết đơn hàng · Mã đơn hàng · Mã sản phẩm · Tên sản phẩm · Đường dẫn sản phẩm · Tên màu sắc · Mã màu Hex · Kích cỡ · Số lượng đặt mua
NULL · UNIQUE · INDEX	Có mặt 100%: <code>productId</code> , <code>colorName</code> , <code>colorHex</code> , <code>size</code> , <code>qty</code> , <code>price</code> , <code>id</code> , <code>orderId</code> . Tùy chọn/không phủ hết sample: không có field thiếu trong sample.

	Index thật: id UNIQUE · orderId index · productId index
Nối · FK owner · bản số	Đi ra: order_items.orderId → Đơn Hàng (order 1 → N items); order_items.productId → Sản Phẩm (product 1 → N sold lines). Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Sống theo order về vòng đời nhưng chứng từ không bị hard-delete. Tình huống: Product đổi giá/tên hoặc bị archive, dòng đơn vẫn hiển thị đúng lịch sử.
Điểm yếu · truy đuổi · mở rộng	Yếu: Physical Mongo chỉ tách collection; runtime hydrate lại thành orders.items, cần một source of truth rõ. GV: MongoDB cho embed array; tại sao implementation vẫn tách collection? [ĐỀ XUẤT] Giữ order_items làm nguồn vật lý; thêm variantId/SKU snapshot nếu kho cần đối soát sâu.

18.9. Thanh Toán

Defense Card 09/36 — Thanh Toán

Hạng · physical	S · payments — 94 document
Vai trò · nếu bỏ	Một payment attempt/giao dịch tiền gần đơn, provider và lịch sử refund. Nếu bỏ: Không biểu diễn retry, trạng thái cổng, đối soát và hoàn từng phần.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã thanh toán · Mã đơn hàng · Mã người dùng · Mã hiển thị thanh toán · Mã hiển thị đơn hàng · Nhà cung cấp dịch vụ · Phương thức thanh toán · Trạng thái giao dịch · Số tiền thanh toán
NULL · UNIQUE · INDEX	Có mặt 100%: id, code, orderId, orderCode, userId, provider, method, status. Tùy chọn/không phủ hết sample: paymentIntentId(33%), checkoutSessionId(32%), originalAmount(20%), discount(20%), voucherDiscount(20%), paymentDiscount(20%). Index thật: id UNIQUE · orderId index · userId index
Nối · FK owner · bản số	Đi ra: payments.orderId → Đơn Hàng (order 1 → N payment attempts); payments.userId → Người Dùng (user 1 → N payments). Nhận vào: Yêu Cầu Trả Hàng qua return_requests.paymentId. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Không xoá; refund/cancel bằng trạng thái và lưu mảng refund. Tình huống: Webhook/return/reconcile có thể lặp; code kiểm tra status/id provider trước khi ghi lặp.
Điểm yếu · truy đuổi · mở rộng	Yếu: orderId không unique nên schema cho phép N attempt, nhưng transactionCode cũng chưa unique ở DB. GV: Webhook lặp ở hai instance thì database constraint nào là lớp cuối? [ĐỀ XUẤT] Unique/sparse theo provider transaction id; event log webhook; outbox cho side effects.

18.10. Yêu Cầu Trả Hàng

Defense Card 10/36 — Yêu Cầu Trả Hàng

Hạng · physical	A · return_requests — 8 document
Vai trò · nếu bỏ	Hồ sơ huỷ/đối/trả: ai yêu cầu, đơn nào, payment nào, món nào và timeline xử lý. Nếu bỏ: Hoàn tiền mất căn cứ, không biết trả toàn bộ hay từng món, không audit quyết định admin.

PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã yêu cầu trả hàng · Mã đơn hàng · Mã người dùng · Mã thanh toán · Mã hiển thị trả hàng · Mã hiển thị đơn hàng · Mã hiển thị thanh toán · Trạng thái yêu cầu · Lý do trả hàng
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>code</code> , <code>orderId</code> , <code>orderCode</code> , <code>userId</code> , <code>paymentId</code> , <code>paymentCode</code> , <code>status</code> . Tùy chọn/không phủ hết sample: <code>adminNote</code> (75%), <code>refundId</code> (25%), <code>refundStatus</code> (25%), <code>kind</code> (88%), <code>photos</code> (88%), <code>codManualRefund</code> (50%). Index thật: <code>id</code> UNIQUE · <code>orderId</code> index · <code>paymentId</code> index · <code>userId</code> index
Nối · FK owner · bản số	Đi ra: <code>return_requests.orderId</code> → Đơn Hàng (order 1 → N requests); <code>return_requests.userId</code> → Người Dùng (user 1 → N requests); <code>return_requests.paymentId</code> → Thanh Toán (payment 1 → 0..N requests). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Không hard-delete sau xử lý; ảnh có retention riêng. Tình huống: Một order có thể có nhiều yêu cầu theo các nhóm item khác nhau.
Điểm yếu · truy đuổi · mở rộng	Yếu: Quy tắc chống chồng lấn item nằm ở code; ảnh chứng minh là URL ngoài DB. GV: Vì sao <code>paymentId</code> nullable, và COD hoàn thể nào? [ĐỀ XUẤT] Unique/conditional rule cho active requests và item-level refund ledger.

18.11. Người Dùng

Defense Card 11/36 — Người Dùng

Hạng · physical	S · users — 36 document
Vai trò · nếu bỏ	Danh tính đăng nhập và điểm neo phân quyền; hồ sơ phong cách được tách sang profiles. Nếu bỏ: Mất đăng nhập, phân quyền và đích tham chiếu của phần lớn dữ liệu theo người dùng.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã người dùng · Họ và tên · Email · Mật khẩu mã hóa · Vai trò · Trạng thái · Số lượt thư'đô · Ngày đăng ký · Mã khách hàng Stripe
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>name</code> , <code>email</code> , <code>role</code> , <code>status</code> , <code>joinedAt</code> . Tùy chọn/không phủ hết sample: <code>tryons</code> (97%), <code>passwordHash</code> (97%), <code>stripeCustomerId</code> (6%), <code>resetCodeHash</code> (3%), <code>resetCodeExpiresAt</code> (3%), <code>googleId</code> (3%). Index thật: <code>id</code> UNIQUE · <code>email</code> index
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: Đơn Hàng qua <code>orders.userId</code> ; Thanh Toán qua <code>payments.userId</code> ; Yêu Cầu Trả Hàng qua <code>return_requests.userId</code> ; Địa Chỉ qua <code>addresses.userId</code> ; Hồ Sơ Người Dùng qua <code>profiles.userId</code> ; Thông Báo qua <code>notifications.userId</code> ; Chi Tiết Giỏ Hàng qua <code>cart_items.userId</code> ; Danh Sách Sản Phẩm Yêu Thích qua <code>wishlist_items.userId</code> ; Mục Tiêu Tiết Kiệm qua <code>goals.userId</code> ; Lịch Sử Tìm Kiếm qua <code>search_logs.userId</code> ; Tin Nhắn qua <code>chats.userId</code> ; Tương Tác Người Dùng qua <code>interactions.userId</code> ; Lịch Sử Thử Đồ qua <code>tryon_history.userId</code> ; Phiếu Giảm Giá qua <code>vouchers.ownerId</code> ; Phiếu Giảm Giá qua <code>vouchers.userId</code> ; Lướt Sử Dụng Phiếu qua <code>voucher_redemptions.userId</code> ; Thành Viên VIP qua <code>vip_memberships.userId</code> ; Đánh Giá Sản Phẩm qua <code>reviews.userId</code> ; Tương Tác Đánh Giá qua <code>review_reactions.userId</code> ; Đánh Giá Địa Điểm Nhật Bản qua <code>japan_spot_reviews.userId</code> ; Bộ Sưu Tập Thẻ Của Người Dùng qua <code>flagcard_collections.userId</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Không hard-delete chứng từ; khoá bằng <code>status</code> , sau này thêm quy trình ẩn danh hoá. Tình huống: Khoá tài khoản không được làm mất đơn và thanh toán đã phát sinh.
Điểm yếu · truy đuổi · mở rộng	Yếu: <code>users.email</code> chỉ có index thường; chạy nhiều backend vẫn có race đăng ký trùng email. GV: Nếu hai request đăng ký cùng email tới hai instance thì lớp nào chặn? [ĐỀ XUẤT] Thêm UNIQUE email; nếu một người nhiều vai trò/provider thì tách <code>user_roles</code> và

	user_identities.
--	------------------

18.12. Địa Chỉ

Defense Card 12/36 — Địa Chỉ

Hạng · physical	A · addresses — 11 document
Vai trò · nếu bỏ	Số địa chỉ nhận hàng có thể tái sử dụng của một người dùng. Nếu bỏ: Khách phải nhập lại mỗi lần; không quản lý được nhiều địa chỉ/mặc định.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã địa chỉ · Mã người dùng · Nhân địa chỉ · Tên người nhận · Số điện thoại · Số nhà, tên đường · Mã phường / xã · Tên phường / xã · Mã tỉnh / thành phố
NULL · UNIQUE · INDEX	Có mặt 100%: id, userId, title, name, phone, street, wardCode, ward. Tùy chọn/không phủ hết sample: demoBatch(55%). Index thật: id UNIQUE · userId index
Nối · FK owner · bản số	Đi ra: addresses.userId → Người Dùng (user 1 → N addresses). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Có thể xoá địa chỉ khỏi số; không sửa snapshot trong đơn đã tạo. Tình huống: Khách đổi địa chỉ mặc định nhưng đơn cũ vẫn giữ snapshot địa chỉ đã giao.
Điểm yếu · truy đuổi · mở rộng	Yếu: Cần quy tắc chỉ một isDefault=true cho mỗi user; index hiện tại chỉ theo userId. GV: Hai request cùng đặt hai địa chỉ mặc định thì sao? [ĐỀ XUẤT] Dùng unique partial index cho địa chỉ mặc định hoặc transaction khi đổi mặc định.

18.13. Hồ Sơ Người Dùng

Defense Card 13/36 — Hồ Sơ Người Dùng

Hạng · physical	B · profiles — 12 document
Vai trò · nếu bỏ	Thông tin cơ thể, phong cách và ngân sách dùng để cá nhân hoá gợi ý. Nếu bỏ: Tư vấn size/phong cách mất dữ liệu đầu vào hoặc phải nhồi dữ liệu riêng tư vào users.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã hồ sơ người dùng · Mã người dùng · Giới tính · Phong cách yêu thích · Tông màu da · Dịp mặc · Ngân sách dự kiến · Chiều cao (cm) · Cân nặng (kg)
NULL · UNIQUE · INDEX	Có mặt 100%: userId, preferredStyles, heightCm, weightKg, updatedAt, id. Tùy chọn/không phủ hết sample: gender(92%), skinTone(92%), occasion(92%), budget(92%), usualSize(92%), demoBatch(42%). Index thật: userId UNIQUE
Nối · FK owner · bản số	Đi ra: profiles.userId → Người Dùng (user 1 → 0..1 profile). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Xoá/ẩn danh theo chủ tài khoản; không kéo theo đơn hàng. Tình huống: Khách đổi cân nặng nhưng tài khoản và lịch sử đơn không đổi.
Điểm yếu · truy đuổi · mở rộng	Yếu: Toàn vẹn profiles.userId có kiểm tra, nhưng dữ liệu nhạy cảm chính sách lưu giữ rõ hơn. GV: Vì sao profiles.userId unique và tại sao không gộp vào users? [ĐỀ XUẤT] Tách lịch sử số đo nếu cần theo dõi biến đổi; mã hoá/giới hạn truy cập các trường nhạy cảm.

18.14. Cấu Hình Cửa Hàng

Defense Card 14/36 — Cấu Hình Cửa Hàng

Hạng · physical	C · settings — 3 document
Vai trò · nếu bỏ	Singleton/nhóm document cấu hình phí ship, phương thức thanh toán và logo. Nếu bỏ: Các route phải hard-code cấu hình và dễ lệch giữa admin/app.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ <code>id</code> làm domain/API identity ổn định. ERD: Mã cấu hình · Tên cửa hàng · Hotline hỗ trợ · Email liên hệ · Địa chỉ cửa hàng · Phí vận chuyển · Thanh toán khi nhận hàng · Thanh toán Stripe · Thanh toán VNPAY
NULL · UNIQUE · INDEX	Có mặt 100%: không kết luận từ sample. Tùy chọn/không phủ hết sample: mongo(33%), clouinary(33%), ai(33%), name(33%), hotline(33%), email(33%). Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Không xoá singleton; cập nhật có kiểm soát và lưu lịch sử. Tình huống: Admin đổi phí ship, order mới dùng giá mới còn order cũ giữ ship snapshot.
Điểm yếu · truy đuổi · mở rộng	Yếu: Nhiều loại settings nằm chung collection; cần whitelist và audit ai đổi. GV: Đổi phí ship giữa lúc checkout thì đơn lấy giá nào? [ĐỀ XUẤT] Version/audit log và schema validator theo <code>_id</code> từng nhóm.

18.15. Thông Báo

Defense Card 15/36 — Thông Báo

Hạng · physical	B · notifications — 47 document
Vai trò · nếu bỏ	Inbox thông báo nghiệp vụ cho user, có type/action và thời điểm. Nếu bỏ: Không có lịch sử những gì hệ thống đã báo cho khách.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ <code>id</code> làm domain/API identity ổn định. ERD: Mã thông báo · Mã người dùng · Tiêu đề thông báo · Nội dung thông báo · Phân loại thông báo · Hành động · Lượt tiếp cận · Thời gian gửi
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>title</code> , <code>body</code> , <code>type</code> , <code>reach</code> , <code>at</code> . Tùy chọn/không phủ hết sample: <code>userId</code> (94%), <code>action</code> (81%), <code>demoBatch</code> (23%). Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>notifications.userId</code> → Người Dùng (<code>user 1</code> → <code>N notifications</code>). Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	TTL/dọn theo retention; xoá user thì anonymize hoặc xoá inbox. Tình huống: Order đổi trạng thái tạo notification có action mở đúng order.
Điểm yếu · truy đuổi · mở rộng	Yếu: <code>userId</code> logical chưa nằm trong <code>relationshipErrors</code> ; <code>reach</code> không thay thế trạng thái đã đọc. GV: Ghi DB thành công nhưng gửi push thất bại thì source of truth là gì? [ĐỀ XUẤT] Thêm <code>readAt/delivery attempts</code> và <code>outbox</code> để tránh mất/nhân đôi.

18.16. Chi Tiết Giỏ Hàng

Defense Card 16/36 — Chi Tiết Giỏ Hàng

Hạng · physical	A · cart_items — 6 document
Vai trò · nếu bỏ	Lựa chọn tạm thời của user theo product–màu–size và quantity. Nếu bỏ: Không đồng bộ giỏ giữa thiết bị và không phân biệt hai biến thể cùng product.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã chi tiết giỏ hàng · Mã người dùng · Mã sản phẩm · Màu sắc · Số lượng · Kích thước · Thời gian cập nhật
NULL · UNIQUE · INDEX	Có mặt 100%: userId, productId, color, size, quantity, updatedAt, id. Tùy chọn/không phủ hết sample: demoBatch(67%). Index thật: id UNIQUE · userId+productId+color+size UNIQUE
Nối · FK owner · bản số	Đi ra: cart_items.userId → Người Dùng (user 1 → N cart lines); cart_items.productId → Sản Phẩm (product 1 → N cart lines). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Xoá tự do khi quantity về 0/checkout; không ảnh hưởng product. Tình huống: Cùng áo size M và L là hai dòng hợp lệ; duplicate đúng selection bị unique chặn.
Điểm yếu · truy đuổi · mở rộng	Yếu: Route ghi bằng userId trong body ở một số luồng; cần nhất quán authorization theo JWT. GV: Vì sao unique bốn cột mà không chỉ (userId, productId)? [ĐỀ XUẤT] Dùng variantId làm FK và TTL/cleanup cho giỏ bỏ quên.

18.17. Danh Sách Sản Phẩm Yêu Thích

Defense Card 17/36 — Danh Sách Sản Phẩm Yêu Thích

Hạng · physical	A · wishlist_items — 14 document
Vai trò · nếu bỏ	Bảng bắc cầu user–product cho một lựa chọn yêu thích. Nếu bỏ: Không đồng bộ được wishlist và phải nhúng mảng vào user/product.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã yêu thích · Mã người dùng · Mã sản phẩm · Ngày thêm
NULL · UNIQUE · INDEX	Có mặt 100%: id, userId, createdAt, productId. Tùy chọn/không phủ hết sample: demoBatch(57%). Index thật: id UNIQUE · userId+productId UNIQUE
Nối · FK owner · bản số	Đi ra: wishlist_items.userId → Người Dùng (user 1 → N wishlist rows); wishlist_items.productId → Sản Phẩm (product 1 → N wishlist rows). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Xoá tự do khi bỏ thích; product archive thì ẩn hoặc giữ để thông báo. Tình huống: Người dùng bấm yêu thích hai lần vẫn chỉ có một cặp.
Điểm yếu · truy đuổi · mở rộng	Yếu: Integrity có kiểm tra, nhưng sản phẩm archive cần quy tắc hiển thị rõ. GV: Tại sao wishlist có composite unique còn reviews lại chưa có? [ĐỀ XUẤT] Thêm created source/folder nếu cần nhiều danh sách.

18.18. Mục Tiêu Tiết Kiệm

Defense Card 18/36 — Mục Tiêu Tiết Kiệm

Hạng · physical	B · goals — 8 document
Vai trò · nếu bỏ	Kế hoạch tích quỹ của user cho một sản phẩm, kèm tiến độ và các lần nạp. Nếu bỏ: Không theo dõi được mục tiêu, số dư và sản phẩm đích.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã mục tiêu · Mã người dùng · Mã sản phẩm · Thông tin sản phẩm · Thông tin mục tiêu · Kế hoạch thực hiện · Ngày tạo · Ngày cập nhật · Quỹ tiết kiệm
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>userId</code> , <code>productId</code> , <code>product</code> , <code>input</code> , <code>plan</code> , <code>createdAt</code> , <code>updatedAt</code> . Tùy chọn/không phủ hết sample: <code>fund(50%)</code> , <code>demoBatch(38%)</code> . Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>goals.userId</code> → Người Dùng (<code>user 1</code> → <code>N goals</code>); <code>goals.productId</code> → Sản Phẩm (<code>product 1</code> → <code>N goals</code>). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	User có thể đóng mục tiêu; ledger tài chính không nên hard-delete. Tình huống: Giá product đổi khiến goal phải phân biệt target snapshot và giá hiện tại.
Điểm yếu · truy đuổi · mở rộng	Yếu: <code>relationshipErrors</code> kiểm user/product nhưng cấu trúc quỹ nhúng cần invariant tổng tiền. GV: Nếu product bị xoá hoặc đổi giá thì target của goal ra sao? [ĐỀ XUẤT] Tách deposit ledger khi cần audit/đối soát tiền thật.

18.19. Lịch Sử Tìm Kiếm

Defense Card 19/36 — Lịch Sử Tìm Kiếm

Hạng · physical	C · search_logs — 13 document
Vai trò · nếu bỏ	Log từ khoá, số kết quả và user để phân tích nhu cầu/zero-result. Nếu bỏ: Không biết khách tìm gì mà catalog chưa đáp ứng.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã lịch sử tìm kiếm · Mã người dùng · Từ khóa tìm kiếm · Số lượng kết quả · Thời gian tìm kiếm
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>userId</code> , <code>query</code> , <code>resultCount</code> , <code>createdAt</code> . Tùy chọn/không phủ hết sample: <code>demoBatch(62%)</code> . Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>search_logs.userId</code> → Người Dùng (<code>user 1</code> → <code>N search events</code>). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Dọn theo retention; <code>userId</code> có thể anonymize. Tình huống: Từ khoá có 0 kết quả vẫn được ghi để admin cải thiện catalog.
Điểm yếu · truy đuổi · mở rộng	Yếu: Có thể chứa dữ liệu nhạy cảm; physical collection có thể rỗng dù route tồn tại. GV: Tại sao không lưu mọi keystroke tìm kiếm? [ĐỀ XUẤT] TTL, chuẩn hoá query, ẩn danh user và aggregate hot terms.

18.20. Tin Nhắn

Defense Card 20/36 — Tin Nhắn

Hạng · physical	C · chats — 68 document
Vai trò · nếu bỏ	Lịch sử hội thoại trợ lý theo user, có product context tùy chọn. Nếu bỏ: Trợ lý mất ngữ cảnh và không audit được câu trả lời đã phát.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã tin nhắn · Mã người dùng · Mã sản phẩm · Vai trò người gửi · Nội dung tin nhắn · Thời gian gửi
NULL · UNIQUE · INDEX	Có mặt 100%: id, role, message, createdAt. Tùy chọn/không phủ hết sample: userId(71%), productIds(46%), engine(19%), intent(19%), confidence(19%), modelTrace(15%). Index thật: Chỉ có _id; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: chats.userId → Người Dùng (user 1 → N messages); chats.productId → Sản Phẩm (product 1 → 0..N contextual messages). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Dọn theo retention; product bị archive không được phá nội dung cũ. Tình huống: User hỏi về một product; productId có thể rỗng khi câu hỏi chung.
Điểm yếu · truy đuổi · mở rộng	Yếu: Hai logical refs chưa được relationshipErrors kiểm tra; retention nội dung hội thoại chưa rõ. GV: Vì sao productId nullable và khi product bị xoá thì chat sao? [ĐỀ XUẤT] Tách conversation/message nếu cần nhiều phiên, thêm retention/consent.

18.21. Tương Tác Người Dùng

Defense Card 21/36 — Tương Tác Người Dùng

Hạng · physical	C · interactions — 675 document
Vai trò · nếu bỏ	Event hành vi xem/thích/cart dùng cho analytics và gợi ý. Nếu bỏ: Không đo được funnel hay cá nhân hoá theo hành vi.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã tương tác · Mã người dùng · Mã sản phẩm · Nguồn tương tác · Thời gian tương tác · Loại tương tác
NULL · UNIQUE · INDEX	Có mặt 100%: id, productId, type, createdAt, source. Tùy chọn/không phủ hết sample: userId(79%), value(85%), metadata(42%), demoBatch(22%). Index thật: Chỉ có _id; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: interactions.userId → Người Dùng (user 1 → N events); interactions.productId → Sản Phẩm (product 1 → N events). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Cho phép TTL/aggregation; không cascade làm hỏng master data. Tình huống: Mỗi lượt xem có thể sinh event nên collection tăng nhanh hơn master data.
Điểm yếu · truy đuổi · mở rộng	Yếu: Route /interactions nhận userId từ body; collection phình nhanh và refs chưa được assert. GV: Index nào phục vụ query cụ thể mà không làm write quá đắt? [ĐỀ XUẤT] Xác thực principal, partition/TTL, aggregate theo ngày.

18.22. Lịch Sử Thử Đồ

Defense Card 22/36 — Lịch Sử Thử Đồ

Hạng · physical	C · tryon_history — 75 document
Vai trò · nếu bỏ	Lưu một lần try-on theo user/product/engine để xem lại và đo usage. Nếu bỏ: Không theo dõi lượt dùng, kết quả hoặc sản phẩm đã thử.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã lịch sử thử đồ · Mã sản phẩm · Mã người dùng · Công nghệ AI · Ngày thử đồ
NULL · UNIQUE · INDEX	Có mặt 100%: id, productId, accessoryIds, engine, createdAt. Tùy chọn/không phủ hết sample: userId(81%), productIds(29%), appliedAccessories(16%), skippedAccessories(16%), demoBatch(13%). Index thật: Chỉ có _id; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: tryon_history.productId → Sản Phẩm (product 1 → N try-ons); tryon_history.userId → Người Dùng (user 1 → N try-ons). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Cho phép user xoá; cần xoá cả asset ngoài DB theo workflow. Tình huống: User thử cùng product nhiều lần bằng engine khác nhau, nên quan hệ là 1:N.
Điểm yếu · truy đuổi · mở rộng	Yếu: Ảnh người dùng nhạy cảm; logical refs chưa được relationshipErrors kiểm tra. GV: Nếu xoá metadata nhưng ảnh Cloudinary còn thì đã đáp ứng quyền xoá chưa? [ĐỀ XUẤT] Retention ngắn, consent, xoá asset Cloudinary và metadata đồng bộ.

18.23. Phiếu Giảm Giá

Defense Card 23/36 — Phiếu Giảm Giá

Hạng · physical	A · vouchers — 4 document
Vai trò · nếu bỏ	Định nghĩa voucher: code, loại/giá trị, hạn, limit, người cấp và người nhận tùy chọn. Nếu bỏ: Checkout không có quy tắc giảm, giới hạn và nguồn phát hành để kiểm tra.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã phiếu giảm giá · Mã người tạo · Mã người dùng · Mã hiển thị · Loại giảm giá · Giá trị giảm · Giá trị đơn tối thiểu · Ngày hết hạn · Giới hạn sử dụng
NULL · UNIQUE · INDEX	Có mặt 100%: code, type, value, min, expiry, limit, used, active. Tùy chọn/không phủ hết sample: appliesTo(25%), ownerUserId(25%), source(25%), reason(25%), issuedBy(25%), issuedAt(25%). Index thật: id UNIQUE · code UNIQUE
Nối · FK owner · bản số	Đi ra: vouchers.ownerId → Người Dùng (user 1 → N issued vouchers); vouchers.userId → Người Dùng (user 1 → 0..N assigned vouchers). Nhận vào: Đơn Hàng qua orders.voucherId; Lượt Sử Dụng Phiếu qua voucher_redemptions.voucherId. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Archive khi hết hạn; giữ redemption và order snapshot. Tình huống: Voucher hết hạn ngay lúc tạo order thì server từ chối theo thời điểm checkout.
Điểm yếu · truy đuổi · mở rộng	Yếu: used và redemption có thể lệch nếu write giữa chừng; hai quan hệ tới users dễ bị hiểu nhầm. GV: Nếu huỷ order thì used/redemption có được hoàn không? [ĐỀ XUẤT] Lấy redemption làm ledger nguồn; atomic consume/unique per-user theo policy.

18.24. Lướt Sử Dụng Phiếu

Defense Card 24/36 — Lướt Sử Dụng Phiếu

Hạng · physical	A · voucher_redemptions — 5 document
Vai trò · nếu bỏ	Ledger từng lần voucher được dùng bởi user cho order và số tiền giảm. Nếu bỏ: Một counter không trả lời được ai dùng, đơn nào, lúc nào và giảm bao nhiêu.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã lướt sử dụng phiếu · Mã phiếu giảm giá · Mã người dùng · Mã đơn hàng · Mã áp dụng · Số tiền giảm · Ngày sử dụng
NULL · UNIQUE · INDEX	Có mặt 100%: id, code, userId, orderId, discount, redeemedAt, voucherId. Tùy chọn/không phủ hết sample: demoBatch(60%). Index thật: id UNIQUE · voucherId index · userId index · orderId index
Nối · FK owner · bản số	Đi ra: voucher_redemptions.voucherId → Phiếu Giảm Giá (voucher 1 → N redemptions); voucher_redemptions.userId → Người Dùng (user 1 → N redemptions); voucher_redemptions.orderId → Đơn Hàng (order 1 → 0..N redemptions). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Không xoá tùy tiện; reverse bằng trạng thái/event để giữ audit. Tình huống: Khách khiếu nại voucher không áp dụng có thể truy đúng redemption/order.
Điểm yếu · truy đuổi · mở rộng	Yếu: Ba logical FK có index nhưng collection này chưa được relationshipErrors kiểm tra. GV: Vì sao ba index thường chưa đủ chống dùng voucher lặp? [ĐỀ XUẤT] Composite unique theo quy tắc per-user/per-order và quy trình reverse khi order huỷ.

18.25. Quy Tắc Giảm Giá

Defense Card 25/36 — Quy Tắc Giảm Giá

Hạng · physical	B · discount_rules — 3 document
Vai trò · nếu bỏ	Cấu hình chương trình giảm có scope, điều kiện, mức giảm và thời hạn. Nếu bỏ: VIP membership phải lập phần trăm/ngưỡng, đối rule không quản trị tập trung.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã quy tắc giảm giá · Mã chương trình · Tên chương trình giảm giá · Phạm vi áp dụng · Hình thức giảm giá · Giá trị giảm · Số lượng tối đa · Loại điều kiện đặt ra · Giá trị điều kiện cần đạt
NULL · UNIQUE · INDEX	Có mặt 100%: id, code, name, scope, type, value, active. Tùy chọn/không phủ hết sample: maxUnitsPerOrder(33%), qualificationType(33%), qualificationValue(33%), validityDays(33%), demoBatch(67%). Index thật: id UNIQUE · code UNIQUE
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: Thành Viên VIP qua vip_memberships.discountRuleId. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Archive rule đã được tham chiếu, không hard-delete. Tình huống: VIP membership trở discountRuleId, còn số tiền đã áp dụng snapshot trong order.
Điểm yếu · truy đuổi · mở rộng	Yếu: Rule thay đổi có thể ảnh hưởng membership đang active nếu không version. GV: Rule đổi giữa kỳ VIP thì membership cũ theo rule nào? [ĐỀ XUẤT] Version/immutable rule; membership trở version đã cấp.

18.26. Thành Viên VIP

Defense Card 26/36 — Thành Viên VIP

Hạng · physical	B · vip_memberships — 5 document
Vai trò · nếu bỏ	Một giai đoạn user đạt VIP theo rule và tập đơn đủ điều kiện. Nếu bỏ: Không biết ai đang VIP, hạn tới đâu và dựa trên đơn nào.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã thành viên VIP · Mã người dùng · Mã quy tắc giảm giá · Kỳ xét duyệt VIP · Danh sách đơn hàng xét duyệt · Tổng chi tiêu đủ điều kiện · Ngày bắt đầu VIP · Ngày hết hạn VIP · Trạng thái
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>userId</code> , <code>discountRuleId</code> , <code>qualifyingPeriod</code> , <code>qualifyingOrderIds</code> , <code>qualifiedSpend</code> , <code>startedAt</code> , <code>expiresAt</code> . Tùy chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: <code>id</code> UNIQUE · <code>userId+status</code> index · <code>discountRuleId</code> index
Nối · FK owner · bản số	Đi ra: <code>vip_memberships.userId</code> → Người Dùng (user 1 → N membership periods); <code>vip_memberships.discountRuleId</code> → Quy Tắc Giảm Giá (rule 1 → N memberships). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Hết hạn bằng status; giữ lịch sử membership để audit ưu đãi. Tình huống: Đạt ngưỡng chi tiêu thì membership lưu <code>qualifyingOrderIds</code> và <code>expiresAt</code> .
Điểm yếu · truy đuổi · mở rộng	Yếu: Index (<code>userId</code> , <code>status</code>) không unique; có thể tồn tại nhiều active membership nếu app sai. GV: Hai membership active cho một user bị lớp nào chặn? [ĐỀ XUẤT] Unique partial cho active membership; ledger qualification riêng nếu quy mô lớn.

18.27. Đánh Giá Sản Phẩm

Defense Card 27/36 — Đánh Giá Sản Phẩm

Hạng · physical	A · reviews — 31 document
Vai trò · nếu bỏ	Nhận xét đã xác minh mua hàng nhờ nối user–product–order. Nếu bỏ: Không có social proof; bỏ <code>orderId</code> thì khó chứng minh đã mua và nhận hàng.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã đánh giá · Mã sản phẩm · Mã người dùng · Mã đơn hàng · Tên người dùng · Mã hiển thị đơn hàng · Điểm số đánh giá · Nội dung nhận xét · Hình ảnh & Video
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>productId</code> , <code>userId</code> , <code>userName</code> , <code>orderId</code> , <code>orderCode</code> , <code>rating</code> , <code>comment</code> . Tùy chọn/không phủ hết sample: <code>adminNote</code> (32%), <code>moderatedAt</code> (32%), <code>demoBatch</code> (29%). Index thật: <code>id</code> UNIQUE · <code>productId</code> index · <code>userId</code> index · <code>orderId</code> index
Nối · FK owner · bản số	Đi ra: <code>reviews.productId</code> → Sản Phẩm (product 1 → N reviews); <code>reviews.userId</code> → Người Dùng (user 1 → N reviews); <code>reviews.orderId</code> → Đơn Hàng (order 1 → 0..N reviews). Nhận vào: Tương Tác Đánh Giá qua <code>review_reactions.reviewId</code> ; Mẫu Kiểm Duyệt qua <code>moderation_samples.reviewId</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trở tới đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Ưu tiên đổi status/ẩn; reaction nên cascade khi review thật sự bị xoá. Tình huống: Sau moderation async, code recheck trùng trong update trước khi push review.
Điểm yếu · truy đuổi · mở rộng	Yếu: Thiếu UNIQUE (<code>userId</code> , <code>productId</code>) ở MongoDB; hai instance vẫn có thể tạo trùng. GV: Hai request review qua hai instance cùng vượt application check thì sao? [ĐỀ XUẤT] Thêm composite unique; tách <code>review_media</code> nếu một review nhiều media có metadata.

18.28. Tương Tác Đánh Giá

Defense Card 28/36 — Tương Tác Đánh Giá

Hạng · physical	B · review_reactions — 88 document
Vai trò · nếu bỏ	Một user đánh dấu helpful/not helpful cho một review. Nếu bỏ: Không đếm phản hồi tin cậy hoặc phải nhúng danh sách user vào review.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã tương tác đánh giá · Mã đánh giá · Mã người dùng · Giá trị tương tác · Ngày cập nhật
NULL · UNIQUE · INDEX	Có mặt 100%: id, reviewId, userId, value, updatedAt. Tùy chọn/không phủ hết sample: demoBatch(27%). Index thật: id UNIQUE · reviewId+userId UNIQUE
Nối · FK owner · bản số	Đi ra: review_reactions.reviewId → Đánh Giá Sản Phẩm (review 1 → N reactions); review_reactions.userId → Người Dùng (user 1 → N reactions). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Cascade theo review; user bị ẩn danh có thể xoá phản ứng. Tình huống: Bấm cùng lựa chọn lần nữa thì bỏ; đổi lựa chọn thì cập nhật cùng cấp.
Điểm yếu · truy đuổi · mở rộng	Yếu: ID sinh theo thời gian có thể va chạm lý thuyết; giá trị enum do app kiểm tra. GV: Tại sao reaction là entity thay vì hai counter trong reviews? [ĐỀ XUẤT] Dùng compound key làm natural uniqueness và schema validator cho enum.

18.29. Mẫu Kiểm Duyệt

Defense Card 29/36 — Mẫu Kiểm Duyệt

Hạng · physical	C · moderation_samples — 6 document
Vai trò · nếu bỏ	Mẫu nội dung bị từ chối để hỗ trợ kiểm duyệt các review sau. Nếu bỏ: Bộ kiểm duyệt không học lại từ quyết định admin và không audit được mẫu nguồn.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng _id và thường giữ id làm domain/API identity ổn định. ERD: Mã mẫu kiểm duyệt · Mã đánh giá · Nhãn phân loại · Văn bản đã chuẩn hóa · Cụm từ nhận diện được · Nguồn thu thập · Ngày cập nhật
NULL · UNIQUE · INDEX	Có mặt 100%: id, reviewId, label, normalizedText, learnedPhrases, source, updatedAt. Tùy chọn/không phủ hết sample: demoBatch(33%). Index thật: Chỉ có _id; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: moderation_samples.reviewId → Đánh Giá Sản Phẩm (review 1 → 0..N samples). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Có thể cascade/anonymize theo review sau thời hạn audit. Tình huống: Admin reject review, mẫu chuẩn hoá được tạo/cập nhật theo reviewId.
Điểm yếu · truy đuổi · mở rộng	Yếu: Có logical FK nhưng không nằm trong relationshipErrors; dữ liệu học có thể giữ PII. GV: Nếu review bị xoá vì quyền riêng tư, mẫu học xử lý thế nào? [ĐỀ XUẤT] Thêm retention, version model và quy trình xoá/ẩn dữ liệu cá nhân.

18.30. Đánh Giá Địa Điểm Nhật Bản

Defense Card 30/36 — Đánh Giá Địa Điểm Nhật Bản

Hạng · physical	B · japan_spot_reviews — 5 document
Vai trò · nếu bỏ	Nội dung người dùng đánh giá một địa điểm Nhật, tách khỏi review sản phẩm. Nếu bỏ: Không có UGC địa điểm và sẽ trộn hai loại rating khác nghiệp vụ.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã đánh giá địa điểm · Mã người dùng · Tên địa điểm · Tỉnh / Thành phố · Tên người đánh giá · Điểm đánh giá · Nội dung bình luận · Tập đính kèm · Ngày đánh giá
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>place</code> , <code>prefecture</code> , <code>userId</code> , <code>userName</code> , <code>rating</code> , <code>comment</code> , <code>createdAt</code> . Tùy chọn/không phủ hết sample: <code>media</code> (80%), <code>status</code> (80%), <code>moderation</code> (80%), <code>demoBatch</code> (60%). Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>japan_spot_reviews.userId</code> → Người Dùng (<code>user 1</code> → <code>N spot reviews</code>). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Ấn/moderate thay vì xoá thẳng; <code>media</code> có retention riêng. Tình huống: Một user đánh giá Fushimi Inari, nội dung không ảnh hưởng rating sản phẩm.
Điểm yếu · truy đuổi · mở rộng	Yếu: Chỉ logical <code>userId</code> ; chưa có entity địa điểm chuẩn nên <code>place/prefecture</code> lặp. GV: Tại sao không có bảng <code>japan_spots</code> làm cha? [ĐỀ XUẤT] Thêm <code>japan_spots</code> và unique theo <code>user-spot</code> nếu áp quy tắc một review.

18.31. Hình Ảnh Giao Diện Sản Phẩm

Defense Card 31/36 — Hình Ảnh Giao Diện Sản Phẩm

Hạng · physical	B · product_media — 115 document
Vai trò · nếu bỏ	Metadata media sản phẩm; file thật ở Cloudinary, MongoDB giữ URL/type/vị trí. Nếu bỏ: Không sắp thứ tự được ảnh/video và product phải chứa mảng media công kênh.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã hình ảnh giao diện · Mã sản phẩm · Đường dẫn URL · Loại tệp truyền thông · Thứ tự hiển thị · Ảnh đại diện
NULL · UNIQUE · INDEX	Có mặt 100%: <code>url</code> , <code>id</code> , <code>productId</code> , <code>type</code> , <code>position</code> , <code>isPrimary</code> . Tùy chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: <code>id</code> UNIQUE · <code>productId+position</code> index
Nối · FK owner · bản số	Đi ra: <code>product_media.productId</code> → Sản Phẩm (<code>product 1</code> → <code>N media</code>). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Cascade metadata khi xoá product; xoá asset Cloudinary cần quy trình bù trừ. Tình huống: Đổi ảnh chính chỉ đổi metadata; không nhét binary vào MongoDB.
Điểm yếu · truy đuổi · mở rộng	Yếu: Sơ đồ còn thêm thực thể Hình Ảnh trùng vai trò với collection này. GV: Nếu xoá Mongo thành công nhưng xoá Cloudinary thất bại thì xử lý ra sao? [ĐỀ XUẤT] Gộp hai thực thể logic; thêm unique partial index cho một media chính mỗi product.

18.32. Thẻ Địa Danh

Defense Card 32/36 — Thẻ Địa Danh

Hạng · physical	B · flagcards — 7 document
Vai trò · nếu bỏ	Nội dung gamification về địa danh và danh sách sản phẩm gợi ý. Nếu bỏ: Không có catalog thẻ để trao thưởng và kể nội dung văn hoá.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã thẻ địa danh · Mã sản phẩm · Thứ tự hiển thị · Biểu tượng · Mã màu chủ đạo · Tên địa danh · Tên tiếng Nhật · Khu vực · Mô tả ngắn
NULL · UNIQUE · INDEX	Có mặt 100%: id, order, glyph, accent, title, japanese, region, summary. Tùy chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>flagcards.recommendedProductIds[]</code> → Sản Phẩm (flagcard N ↔ N products (array)). Nhận vào: Bộ Sưu Tập Thẻ Của Người Dùng qua <code>flagcard_collections.cardIds[]</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Archive card; không xoá khỏi lịch sử award đã trao. Tình huống: Đơn đủ điều kiện trao một card; card dẫn tới outfit/sản phẩm liên quan.
Điểm yếu · truy đuổi · mở rộng	Yếu: <code>recommendedProductIds[]</code> là quan hệ N:N nhúng, <code>relationshipErrors</code> chưa phủ. GV: Tại sao dùng mảng product IDs mà <code>order_items</code> lại tách collection? [ĐỀ XUẤT] Tách <code>flagcard_products</code> nếu cần ranking/analytics theo từng mapping.

18.33. Bộ Sưu Tập Thẻ Của Người Dùng

Defense Card 33/36 — Bộ Sưu Tập Thẻ Của Người Dùng

Hạng · physical	B · flagcard_collections — 4 document
Vai trò · nếu bỏ	Tiến trình sưu tầm card và lịch sử award của một user. Nếu bỏ: Không biết user đã nhận thẻ nào, từ đơn nào và đã mở thưởng chưa.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã bộ sưu tầm · Mã người dùng · Mã thẻ địa danh · Danh sách phần thưởng · Ngày bắt đầu sưu tầm · Ngày cập nhật
NULL · UNIQUE · INDEX	Có mặt 100%: id, userId, cardIds, awards, createdAt, updatedAt. Tùy chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: <code>flagcard_collections.userId</code> → Người Dùng (user 1 → 0..N collections); <code>flagcard_collections.cardIds[]</code> → Thẻ Địa Danh (collection N ↔ N flagcards (array)). Nhận vào: không có bảng con trở vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Giữ award history; ẩn danh user theo chính sách. Tình huống: Webhook lập không được trao cùng card hai lần; collection kiểm tra <code>cardIds</code> trước khi push.
Điểm yếu · truy đuổi · mở rộng	Yếu: <code>cardIds[]</code> nhúng N:N, không có index/DB constraint cho từng phần tử. GV: Hai instance cùng trao một card thì lớp nào chống duplicate? [ĐỀ XUẤT] Tách award ledger nếu cần chống trùng ở nhiều instance và audit theo order.

18.34. Quảng Cáo

Defense Card 34/36 — Quảng Cáo

Hạng · physical	C · banners — 3 document
Vai trò · nếu bỏ	Banner có ảnh, link, trạng thái và thứ tự hiển thị. Nếu bỏ: Không quản trị được nội dung hero/campaign từ backend.
PK · cột trọng yếu	Logical PK là cột mã trên ERD; physical Mongo dùng <code>_id</code> và thường giữ id làm domain/API identity ổn định. ERD: Mã banner · Tiêu đề banner · Đường dẫn hình ảnh · Đường dẫn điều hướng · Trạng thái hiển thị · Thứ tự ưu tiên
NULL · UNIQUE · INDEX	Có mặt 100%: <code>id</code> , <code>title</code> , <code>img</code> , <code>link</code> , <code>active</code> , <code>order</code> . Tùy chọn/không phủ hết sample: không có field thiếu trong sample. Index thật: Chỉ có <code>_id</code> ; chưa có index nghiệp vụ riêng.
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: không có bảng con trỏ vào. FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Archive; asset ngoài DB được dọn sau khi không còn tham chiếu. Tình huống: Banner draft không hiện; reorder không đổi URL asset.
Điểm yếu · truy đuổi · mở rộng	Yếu: Không có owner/audit/khung thời gian publish; ảnh ngoài Cloudinary cần đồng bộ vòng đời. GV: Nếu hai banner cùng position thì thứ tự nào thắng? [ĐỀ XUẤT] Thêm <code>startAt/endAt</code> , <code>ownerId</code> , <code>placement</code> và <code>audit</code> .

18.35. Màu Sắc

Defense Card 35/36 — Màu Sắc

Hạng · physical	B · Không có collection vật lý độc lập
Vai trò · nếu bỏ	Từ điển màu ở mô hình logic cho biến thể sản phẩm. Nếu bỏ: Nếu màu trở thành dữ liệu quản trị toàn cục thì thiếu nơi chuẩn hoá tên/mã màu.
PK · cột trọng yếu	Logical PK có trên ERD; physical value đang được nhúng hoặc thực thể là phần dư thiết kế. ERD: Mã màu sắc · Tên màu sắc · Mã màu
NULL · UNIQUE · INDEX	Field logic nằm trên ERD; physical implementation không có collection độc lập. Index thật: Không có collection vật lý; xem quyết định nhúng/trùng vai trò.
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: Biến Thể Sản Phẩm qua <code>product_variants.colorName/colorHex</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trỏ tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Hiện sống trong variant; xoá variant là xoá giá trị nhúng. Tình huống: Hiện màu Sumi được nhúng bằng <code>colorName</code> và <code>colorHex</code> trong từng variant.
Điểm yếu · truy đuổi · mở rộng	Yếu: Không có collection thật; đổi tên một màu toàn hệ thống phải cập nhật nhiều variant. GV: Khi nào nhúng màu không còn phù hợp? [ĐỀ XUẤT] Chỉ vật lý hoá thành <code>colors</code> khi màu có vòng đời/truy vấn độc lập.

18.36. Kích Thước

Defense Card 36/36 — Kích Thước

Hạng · physical	B · Không có collection vật lý độc lập
Vai trò · nếu bỏ	Từ điển kích cỡ logic và thứ tự sắp xếp size.

	Nếu bỏ: Nếu size thay đổi theo bảng quy chuẩn/quốc gia thì thiếu nơi quản trị tập trung.
PK · cột trọng yếu	Logical PK có trên ERD; physical value đang được nhúng hoặc thực thể là phần dư thiết kế. ERD: Mã kích thước · Tên kích thước · Thứ tự sắp xếp
NULL · UNIQUE · INDEX	Field logic nằm trên ERD; physical implementation không có collection độc lập. Index thật: Không có collection vật lý; xem quyết định nhúng/trùng vai trò.
Nối · FK owner · bản số	Đi ra: không có logical FK đi ra. Nhận vào: Biến Thể Sản Phẩm qua <code>product_variants.size</code> . FK/reference nằm ở object phía con vì mỗi bản ghi con trở tối đa một cha; array refs được ghi rõ là ngoại lệ MongoDB.
Xoá cha/con · thực tế	Hiện sống trong variant; không có record cha để xoá. Tình huống: Hiện S/M/L/XL/XXL được nhúng trực tiếp vào <code>product_variants.size</code> .
Điểm yếu · truy đuổi · mở rộng	Yếu: Không có collection thật; thứ tự/chuẩn size nằm ở code thay vì dữ liệu. GV: Nếu bán giày EU/US/JP thì thiết kế size hiện tại đối thế nào? [ĐỀ XUẤT] Tách <code>size_charts</code> và <code>sizes</code> khi hỗ trợ nhiều hệ size/quốc gia.

PHẦN 19. WHY TREE CHO SÁU THỰC THỂ HÀNG S

WHY TREE dùng để nhớ theo chuỗi nguyên nhân thay vì học thuộc đoạn văn. Đọc từ gốc xuống lá; mỗi lá phải nối được tới field/index/code thật.

19.1. WHY TREE — `users`

CÂY WHY

USERS	—	danh	tính	ổn	định
Why id?	→	email	đối được;	21	dây không phải cập nhật dây chuyên
Why email?	→	login/liên	lạc;	index	tăng tốc nhưng chưa UNIQUE
Why passwordHash nullable?	→	tài khoản	Google-only	có thể	chưa có password
Why googleId/authProviders?	→	phân biệt	danh tính	ngoài	và cách đăng nhập
Why role trực tiếp?	→	current business	một role/user;	nhiều	role phải tách bridge
Why status?	→	khoá login	mà không	xoá	order/payment chứng từ

19.2. WHY TREE — `products`

CÂY WHY

PRODUCTS	—	mặt	hàng	khách	duyet
Why categoryId?	→	đối tên	category	một chỗ;	tránh update anomaly
Why slug UNIQUE?	→	URL	dễ đọc	nhưng	vẫn không phải PK vì có thể đổi
Why price?	→	giá	hiện tại;	order_items.price	là giá lịch sử
Why variants?	→	màu/cỡ/SKU/stock	thuộc	đơn vị	kho cụ thể
Why media separate?	→	nhiều	ảnh/video;	file	thật ở Cloudinary
Why archive tốt hơn delete?	→	giữ link/history	và	tránh	orphan

19.3. WHY TREE — `product\_variants`

CÂY WHY

PRODUCT_VARIANTS	—	đơn	vị	kho
Why (productId,colorName,size) UNIQUE?	→	một selection	chỉ có	một ô tồn
Why sku UNIQUE?	→	mã vận	hành	kho độc lập tên hiển thị
Why stock ở đây?	→	hết	L không	đồng nghĩa hết product
Why price nullable?	→	chỉ	override	khi variant có giá riêng
Why color/size nhúng?	→	lookup	đóng,	đọc cùng variant
Why atomic update khi scale?	→	check-decrement	trong	RAM chỉ an toàn một process

19.4. WHY TREE — `orders`

CÂY WHY

ORDERS — chứng từ một lần mua

- └ Why code UNIQUE? → mã khách/nhân viên tra cứu, không thay PK nội bộ
- └ Why clientId? → double-click/retry; DB unique còn thiếu
- └ Why customer/address snapshot? → user/address book đổi không sửa lịch sử
- └ Why subtotal/discount/ship/total? → đóng băng phép tính đã chốt
- └ Why history? → dừng lại đường đi trạng thái, nhưng chưa là audit bất biến
- └ Why không hard-delete? → đối soát, refund, khiếu nại và kế toán

19.5. WHY TREE — `order\_items`

CÂY WHY

ORDER_ITEMS — dòng hàng và snapshot

- └ Why orderId? → order 1:N lines
- └ Why productId? → product 1:N sold lines; giải order N:N product
- └ Why qty? → số lượng thuộc lần mua, không thuộc product
- └ Why price/productName/color/size? → historical snapshot
- └ Why position? → hydrate order giữ thứ tự ổn định
- └ Why collection riêng? → index/analytics; runtime vẫn hydrate orders.items

19.6. WHY TREE — `payments`

CÂY WHY

PAYMENTS — payment attempt và refund ledger

- └ Why tách order? → retry/failed/paid/refund có vòng đời riêng
- └ Why orderId non-unique? → allowed design order 1:N attempts
- └ Why provider IDs? → đối soát và idempotency với Stripe/VNPay
- └ Why paidAt/status checks? → callback lặp không ghi nhận tiền hai lần
- └ Why refunds[]? → partial refund và upsert theo refund id
- └ Why không lưu CVV/PAN? → Stripe thu thập; DB chỉ giữ last4/brand/token IDs

PHẦN 20. GIẢNG VIÊN CHỈ NGẪU NHIÊN

Khi con trở dưng ở một bảng, nói **câu mở đầu** trước, rồi chủ động dẫn sang PK/FK/field và một điểm yếu. Cách này biến câu hỏi ngẫu nhiên thành cấu trúc mình kiểm soát.

NẾU CON TRỎ CHỈ VÀO MỘT COLUMN HOẶC MỘT DÂY

Column: ý nghĩa thực tế → ai ghi → khi nào đổi → NULL/UNIQUE/INDEX → snapshot hay state hiện tại → hậu quả nếu sai → app validation hay DB constraint.

Dây: hai thực thể → business relationship → cardinality → logical FK → bên sở hữu → enforcement thật → delete → một ví dụ.

Random Pointer 01–09

Bảng bị chỉ	Câu đầu tiên nên nói	Chủ động dẫn tiếp	Câu bắt bẻ
Danh Mục Sản Phẩm	Đây là master data; product giữ categoryId thay vì chép tên.	Field thật: <code>_id</code> , <code>id</code> , <code>name</code> , <code>kanji</code>	Nếu một product nhiều category, migration hiện tại ảnh hưởng gì?
Sản Phẩm	Đây là mặt hàng khách duyệt; thứ kho xuất thực tế là <code>product_variants</code> .	Field thật: <code>_id</code> , <code>id</code> , <code>slug</code> , <code>name</code>	Vì sao <code>slug</code> unique nhưng không dùng làm PK?
Chi Tiết Sản Phẩm	Đây là phần mở rộng 0..1 của sản phẩm; <code>productId</code> unique chứng minh bản số.	Field thật: <code>_id</code> , <code>productId</code> , <code>description</code> , <code>story</code>	Nếu rating tính từ reviews, tại sao còn lưu <code>rating</code> ?
Biến Thể Sản Phẩm	Đây là đơn vị kho thật; UNIQUE (<code>productId</code> , <code>colorName</code> , <code>size</code>) ngăn tách đôi tồn.	Field thật: <code>_id</code> , <code>colorName</code> , <code>colorHex</code> , <code>size</code>	Hai backend cùng mua món cuối cùng thì thao tác nào phải atomic?
Hình Ảnh	Đây là dấu vết thiết kế cũ; runtime chỉ dùng <code>product_media</code> .	Đối chiếu logical PK/FK và quyết định nhúng.	Tại sao ERD có hai bảng ảnh nhưng MongoDB chỉ có một collection?
Mô Tả Sản Phẩm Tạo Bởi AI	Đây là artifact AI gắn product, không phải mô tả nguồn duy nhất.	Field thật: <code>_id</code> , <code>productId</code> , <code>generatedAt</code> , <code>description</code>	AI sinh sai thì source of truth là trường nào?
Đơn Hàng	Đây là chứng từ đóng băng một lần mua; items tách để biểu diễn dòng hàng.	Field thật: <code>_id</code> , <code>id</code> , <code>code</code> , <code>userId</code>	Nếu process chết trước lượt flush Mongo thì những write nào có thể mất?
Chi Tiết Đơn Hàng	Đây vừa là bảng nối N:N vừa là historical snapshot, không phải duplicate vô nghĩa.	Field thật: <code>_id</code> , <code>productId</code> , <code>colorName</code> , <code>colorHex</code>	MongoDB cho embed array; tại sao implementation vẫn tách collection?
Thanh Toán	Đây là một payment attempt; 94/94 là current data, không phải cardinality bắt buộc.	Field thật: <code>_id</code> , <code>id</code> , <code>code</code> , <code>orderId</code>	Webhook lặp ở hai instance thì database constraint nào là lớp cuối?

Random Pointer 10–18

Bảng bị chỉ	Câu đầu tiên nên nói	Chủ động dẫn tiếp	Câu bắt bẻ
Yêu Cầu Trả Hàng	Đây là workflow hậu mãi tách khỏi order để giữ nhiều lần yêu cầu và timeline.	Field thật: <code>_id</code> , <code>id</code> , <code>code</code> , <code>orderId</code>	Vì sao <code>paymentId</code> nullable, và COD hoàn thế nào?
Người Dùng	Đây là danh tính đăng nhập và phân quyền; <code>id</code> ổn định còn email có thể đổi.	Field thật: <code>_id</code> , <code>id</code> , <code>name</code> , <code>email</code>	Nếu hai request đăng ký cùng email tới hai instance thì lớp nào chặn?
Địa Chỉ	Đây là số địa chỉ 1:N; chứng từ đơn hàng chụp lại địa chỉ thay vì trở sống.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>title</code>	Hai request cùng đặt hai địa chỉ mặc định thì sao?

Hồ Sơ Người Dùng	Đây là hồ sơ cá nhân hoá 0..1 của một user, tách khỏi thông tin xác thực.	Field thật: <code>_id</code> , <code>userId</code> , <code>gender</code> , <code>preferredStyles</code>	Vì sao <code>profiles.userId</code> unique và tại sao không gộp vào <code>users</code> ?
Cấu Hình Cửa Hàng	Đây là cấu hình vận hành, không phải giao dịch; order chụp kết quả áp dụng.	Field thật: <code>_id</code> , <code>mongo</code> , <code>cloudinary</code> , <code>ai</code>	Đối phí ship giữa lúc checkout thì đơn lấy giá nào?
Thông Báo	Đây là bản ghi inbox; push token chỉ là kênh giao, không phải thông báo.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>title</code>	Ghi DB thành công nhưng gửi push thất bại thì source of truth là gì?
Chi Tiết Giỏ Hàng	Đây là mapping có dữ liệu riêng; unique bốn trường giữ đúng một dòng mỗi lựa chọn.	Field thật: <code>_id</code> , <code>userId</code> , <code>productId</code> , <code>color</code>	Vì sao unique bốn cột mà không chỉ (<code>userId</code> , <code>productId</code>)?
Danh Sách Sản Phẩm Yêu Thích	Đây là N:N thuần, UNIQUE (<code>userId</code> , <code>productId</code>) là lớp chống trùng cuối.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>createdAt</code>	Tại sao wishlist có composite unique còn reviews lại chưa có?
Mục Tiêu Tiết Kiệm	Đây là aggregate kế hoạch gắn user và product, khác với payment mua hàng.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>productId</code>	Nếu product bị xoá hoặc đổi giá thì target của goal ra sao?

Random Pointer 19–27

Bảng bị chi	Câu đầu tiên nên nói	Chủ động dẫn tiếp	Câu bắt bẻ
Lịch Sử Tìm Kiếm	Đây là telemetry; collection rỗng không chứng minh entity vô dụng.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>query</code>	Tại sao không lưu mọi keystroke tìm kiếm?
Tin Nhắn	Đây là log hội thoại; product context là optional nên quan hệ 0..1.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>role</code>	Vì sao <code>productId</code> nullable và khi product bị xoá thì chat sao?
Tương Tác Người Dùng	Đây là event log append-heavy, không phải trạng thái nguồn của wishlist/cart.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>productId</code>	Index nào phục vụ query cụ thể mà không làm write quá đắt?
Lịch Sử Thu 'Đồ'	Đây là event AI nhạy cảm gắn user/product, không phải ảnh catalog.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>productId</code>	Nếu xoá metadata nhưng ảnh Cloudinary còn thì đã đáp ứng quyền xoá chưa?
Phiếu Giảm Giá	Đây là rule voucher; hai đây tới users là người cấp và người nhận.	Field thật: <code>_id</code> , <code>code</code> , <code>type</code> , <code>value</code>	Nếu huỷ order thì used/redemption có được hoàn không?
Lượt Sử Dụng Phiếu	Đây vừa là bridge ba phía vừa là transaction ledger, không chỉ là bộ đếm.	Field thật: <code>_id</code> , <code>id</code> , <code>code</code> , <code>userId</code>	Vì sao ba index thường chưa đủ chống dùng voucher lặp?
Quy Tắc Giảm Giá	Đây là master rule dùng chung; order chỉ chụp kết quả tiền giảm.	Field thật: <code>_id</code> , <code>id</code> , <code>code</code> , <code>name</code>	Rule đổi giữa kỳ VIP thì membership cũ theo rule nào?
Thành Viên VIP	Đây là trạng thái có thời hạn nối user và discount rule, không phải cờ boolean trong users.	Field thật: <code>_id</code> , <code>id</code> , <code>userId</code> , <code>discountRuleId</code>	Hai membership active cho một user bị lỗi nào chặn?
Đánh Giá Sản Phẩm	Đây là đánh giá verified purchase vì giữ <code>orderId</code> , nhưng DB chưa chặn duplicate pair.	Field thật: <code>_id</code> , <code>id</code> , <code>productId</code> , <code>userId</code>	Hai request review qua hai instance cùng vượt application check thì sao?

Random Pointer 28–36

Bảng bị chi	Câu đầu tiên nên nói	Chủ động dẫn tiếp	Câu bắt bẻ
Tương Tác Đánh Giá	Đây là bảng bắc cầu có thuộc tính value; composite unique ngăn một user phản ứng hai lần.	Field thật: <code>_id</code> , <code>id</code> , <code>reviewId</code> , <code>userId</code>	Tại sao reaction là entity thay vì hai counter trong reviews?
Mẫu Kiểm Duyệt	Đây là training/audit artifact phát sinh từ	Field thật: <code>_id</code> , <code>id</code> ,	Nếu review bị xoá vì quyền

	review bị từ chối.	reviewId, label	riêng tư, mẫu học xử lý thể nào?
Đánh Giá Địa Điểm Nhật Bản	Đây là review địa điểm, không nối products vì đối tượng đánh giá khác.	Field thật: _id, id, place, prefecture	Tại sao không có bảng japan_spots làm cha?
Hình Ảnh Giao Diện Sản Phẩm	Đây là metadata media; file ở Cloudinary, database chỉ giữ liên kết và thứ tự.	Field thật: _id, url, id, productId	Nếu xoá Mongo thành công nhưng xoá Cloudinary thất bại thì xử lý ra sao?
Thẻ Địa Danh	Đây là master content; quan hệ sản phẩm đang được biểu diễn bằng mảng logical refs.	Field thật: _id, id, order, glyph	Tại sao dùng mảng product IDs mà order_items lại tách collection?
Bộ Sưu Tập Thẻ Của Người Dùng	Đây là aggregate tiến trình của user; card IDs nhưng để đọc một lần.	Field thật: _id, id, userId, cardIds	Hai instance cùng trao một card thì lớp nào chống duplicate?
Quảng Cáo	Đây là content vận hành độc lập, không cần FK trong ERD hiện tại.	Field thật: _id, id, title, img	Nếu hai banner cùng position thì thứ tự nào thắng?
Màu Sắc	Đây là thực thể logic nhưng implementation chủ ý nhúng màu vào variant.	Đối chiếu logical PK/FK và quyết định nhúng.	Khi nào nhúng màu không còn phù hợp?
Kích Thước	Đây là lookup logic được nhúng vì tập giá trị đóng và hiếm đối.	Đối chiếu logical PK/FK và quyết định nhúng.	Nếu bán giày EU/US/JP thì thiết kế size hiện tại đối thể nào?

PHẦN 21. NĂM CHEAT SHEET CUỐI

21.1. SHEET 1 — ALL TABLES

All Tables 01–12

Table logic	Purpose	PK vật lý/domain	Main logical FK	Tier
Danh Mục Sản Phẩm	Nhóm phân loại dùng chung để duyệt catalog và đổi tên tập trung.	<code>_id + id</code>	—	A
Sản Phẩm	Mặt hàng theo cách khách tìm kiếm: tên, slug, danh mục, giá chung và trạng thái.	<code>_id + id</code>	<code>products.categoryId</code>	S
Chi Tiết Sản Phẩm	Khối mô tả dài, câu chuyện, tags, rating và sold tách khỏi lỗi catalog.	<code>_id + id</code>	<code>product_details.productId</code>	B
Biến Thể Sản Phẩm	Một tổ hợp màu-cỡ/SKU cụ thể, là đơn vị thật để định giá riêng và giữ tồn kho.	<code>_id + id</code>	<code>product_variants.productId · product_variants.colorName/colorHex · product_variants.size</code>	S
Hình Ảnh	Thực thể logic cũ cho ảnh sản phẩm, hiện trùng chức năng với <code>product_media</code> .	PK logic; nhúng/không vật lý	<code>legacy_image.productId</code>	B
Mô Tả Sản Phẩm Tạo Bởi AI	Lưu kết quả và dấu vết mô tả AI theo sản phẩm để tái sử dụng/audit.	<code>_id + id</code>	<code>ai_descriptions.productId</code>	C
Đơn Hàng	Chứng từ một lần mua, đóng băng khách, địa chỉ, giảm giá, tổng tiền và lịch sử trạng thái.	<code>_id + id</code>	<code>orders.userId · orders.voucherId</code>	S
Chi Tiết Đơn Hàng	Dòng hàng nối order-product và giữ qty, giá, tên, slug, màu, cỡ tại lúc mua.	<code>_id + id</code>	<code>order_items.orderId · order_items.productId</code>	S
Thanh Toán	Một payment attempt/giao dịch tiền gần đơn, provider và lịch sử refund.	<code>_id + id</code>	<code>payments.orderId · payments.userId</code>	S
Yêu Cầu Trả Hàng	Hồ sơ huỷ/đổi/trả: ai yêu cầu, đơn nào, payment nào, món nào và timeline xử lý.	<code>_id + id</code>	<code>return_requests.orderId · return_requests.userId · return_requests.paymentId</code>	A
Người Dùng	Danh tính đăng nhập và điểm neo phân quyền; hồ sơ phong cách được tách sang <code>profiles</code> .	<code>_id + id</code>	—	S
Địa Chỉ	Số địa chỉ nhận hàng có thể tái sử dụng của một người dùng.	<code>_id + id</code>	<code>addresses.userId</code>	A

All Tables 13–24

Table logic	Purpose	PK vật lý/domain	Main logical FK	Tier
Hồ Sơ Người Dùng	Thông tin cơ thể, phong cách và ngân sách dùng để cá nhân hoá gợi ý.	<code>_id + id</code>	<code>profiles.userId</code>	B
Cấu Hình Cửa Hàng	Singleton/nhóm document cấu hình phí ship, phương thức thanh toán và logo.	<code>_id + id</code>	—	C
Thông Báo	Inbox thông báo nghiệp vụ cho user, có type/action và thời điểm.	<code>_id + id</code>	<code>notifications.userId</code>	B
Chi Tiết Giỏ Hàng	Lựa chọn tạm thời của user theo product-màu-size và quantity.	<code>_id + id</code>	<code>cart_items.userId · cart_items.productId</code>	A
Danh Sách Sản Phẩm Yêu Thích	Bảng bắc cầu user-product cho một lựa chọn yêu thích.	<code>_id + id</code>	<code>wishlist_items.userId · wishlist_items.productId</code>	A

			ctId	
Mục Tiêu Tiết Kiệm	Kế hoạch tích quỹ của user cho một sản phẩm, kèm tiền độ và các lần nạp.	_id + id	goals.userId · goals.productId	B
Lịch Sử Tìm Kiếm	Log từ khoá, số kết quả và user để phân tích nhu cầu/zero-result.	_id + id	search_logs.userId	C
Tin Nhắn	Lịch sử hội thoại trợ lý theo user, có product context tùy chọn.	_id + id	chats.userId · chats.productId	C
Tương Tác Người Dùng	Event hành vi xem/thích/cart dùng cho analytics và gợi ý.	_id + id	interactions.userId · interactions.productId	C
Lịch Sử Thử Đồ	Lưu một lần try-on theo user/product/engine để xem lại và đo usage.	_id + id	tryon_history.productId · tryon_history.userId	C
Phiếu Giảm Giá	Định nghĩa voucher: code, loại/giá trị, hạn, limit, người cấp và người nhận tùy chọn.	_id + id	vouchers.ownerId · vouchers.userId	A
Lượt Sử Dụng Phiếu	Ledger từng lần voucher được dùng bởi user cho order và số tiền giảm.	_id + id	voucher_redemptions.voucherId · voucher_redemptions.userId · voucher_redemptions.orderId	A

All Tables 25–36

Table logic	Purpose	PK vật lý/domain	Main logical FK	Tier
Quy Tắc Giảm Giá	Cấu hình chương trình giảm có scope, điều kiện, mức giảm và thời hạn.	_id + id	—	B
Thành Viên VIP	Một giai đoạn user đạt VIP theo rule và tập đơn đủ điều kiện.	_id + id	vip_memberships.userId · vip_memberships.discountRuleId	B
Đánh Giá Sản Phẩm	Nhận xét đã xác minh mua hàng nhờ nối user–product–order.	_id + id	reviews.productId · reviews.userId · reviews.orderId	A
Tương Tác Đánh Giá	Một user đánh dấu helpful/not helpful cho một review.	_id + id	review_reactions.reviewId · review_reactions.userId	B
Mẫu Kiểm Duyệt	Mẫu nội dung bị từ chối để hỗ trợ kiểm duyệt các review sau.	_id + id	moderation_samples.reviewId	C
Đánh Giá Địa Điểm Nhật Bản	Nội dung người dùng đánh giá một địa điểm Nhật, tách khỏi review sản phẩm.	_id + id	japan_spot_reviews.userId	B
Hình Ảnh Giao Diện Sản Phẩm	Metadata media sản phẩm; file thật ở Cloudinary, MongoDB giữ URL/type/vị trí.	_id + id	product_media.productId	B
Thẻ Địa Danh	Nội dung gamification về địa danh và danh sách sản phẩm gợi ý.	_id + id	flagcards.recommendedProductIds[]	B
Bộ Sưu Tập Thẻ Của Người Dùng	Tiến trình sưu tầm card và lịch sử award của một user.	_id + id	flagcard_collections.userId · flagcard_collections.cardIds[]	B
Quảng Cáo	Banner có ảnh, link, trạng thái và thứ tự hiển thị.	_id + id	—	C
Màu Sắc	Từ điển màu ở mô hình logic cho biến thể sản phẩm.	PK logic; nhúng/không vật lý	—	B
Kích Thước	Từ điển kích cỡ logic và thứ tự sắp xếp size.	PK logic; nhúng/không vật lý	—	B

21.2. SHEET 2 — RELATIONSHIPS

Relationship Cheat Sheet R01–R17

ID	A (con)	B (cha)	Cardinality	FK · WHY
R01	Sản Phẩm	Danh Mục Sản Phẩm	category 1 → N products	products.categoryId · Một danh mục gom nhiều sản phẩm.
R02	Chi Tiết Sản Phẩm	Sản Phẩm	product 1 → 0..1 detail	product_details.productId · Một product có tối đa một khối chi tiết.
R03	Biến Thể Sản Phẩm	Sản Phẩm	product 1 → N variants	product_variants.productId · Một product có nhiều tổ hợp màu–cỡ.
R04	Biến Thể Sản Phẩm	Màu Sắc	color 1 → N variants (logic)	product_variants.colorName/colorHex · Nhiều variant dùng cùng khái niệm màu.
R05	Biến Thể Sản Phẩm	Kích Thước	size 1 → N variants (logic)	product_variants.size · Nhiều variant dùng cùng kích cỡ.
R06	Hình Ảnh	Sản Phẩm	product 1 → N images (logic)	legacy_image.productId · Ảnh cũ thuộc một product.
R07	Mô Tả Sản Phẩm Tạo Bởi AI	Sản Phẩm	product 1 → N generations	ai_descriptions.productId · Một product có thể sinh mô tả nhiều lần.
R08	Chi Tiết Đơn Hàng	Đơn Hàng	order 1 → N items	order_items.orderId · Một đơn gồm nhiều dòng hàng.
R09	Chi Tiết Đơn Hàng	Sản Phẩm	product 1 → N sold lines	order_items.productId · Một product xuất hiện ở nhiều đơn.
R10	Đơn Hàng	Người Dùng	user 1 → N orders	orders.userId · Một user đặt nhiều đơn; guest là ngoại lệ logic.
R11	Đơn Hàng	Phiếu Giảm Giá	voucher 1 → 0..N orders	orders.voucherId · Voucher có thể áp cho nhiều đơn; order có tối đa một voucher record.
R12	Thanh Toán	Đơn Hàng	order 1 → N payment attempts	payments.orderId · Một đơn có thể retry thanh toán.
R13	Thanh Toán	Người Dùng	user 1 → N payments	payments.userId · Một user phát sinh nhiều giao dịch.
R14	Yêu Cầu Trả Hàng	Đơn Hàng	order 1 → N requests	return_requests.orderId · Một order có thể trả theo nhiều đợt/item.
R15	Yêu Cầu Trả Hàng	Người Dùng	user 1 → N requests	return_requests.userId · User tạo nhiều yêu cầu hậu mãi.
R16	Yêu Cầu Trả Hàng	Thanh Toán	payment 1 → 0..N requests	return_requests.paymentId · Refund online trả đúng payment; COD có thể rỗng.
R17	Địa Chỉ	Người Dùng	user 1 → N addresses	addresses.userId · Một user lưu nhiều địa chỉ.

Relationship Cheat Sheet R18–R34

ID	A (con)	B (cha)	Cardinality	FK · WHY
R18	Hồ Sơ Người Dùng	Người Dùng	user 1 → 0..1 profile	profiles.userId · Một user có tối đa một profile hiện tại.
R19	Thông Báo	Người Dùng	user 1 → N notifications	notifications.userId · Một user nhận nhiều thông báo.
R20	Chi Tiết Giỏ Hàng	Người Dùng	user 1 → N cart lines	cart_items.userId · Một user có nhiều dòng giỏ.
R21	Chi Tiết Giỏ Hàng	Sản Phẩm	product 1 → N cart lines	cart_items.productId · Một product nằm trong nhiều giỏ.
R22	Danh Sách Sản Phẩm Yêu Thích	Người Dùng	user 1 → N wishlist rows	wishlist_items.userId · Một user thích nhiều product.
R23	Danh Sách Sản Phẩm	Sản Phẩm	product 1 → N wishlist	wishlist_items.productId · Một product được

	Yêu Thích		rows	nhiều user thích.
R24	Mục Tiêu Tiết Kiệm	Người Dùng	user 1 → N goals	goals.userId · Một user có nhiều mục tiêu.
R25	Mục Tiêu Tiết Kiệm	Sản Phẩm	product 1 → N goals	goals.productId · Một product là đích của nhiều goal.
R26	Lịch Sử Tìm Kiếm	Người Dùng	user 1 → N search events	search_logs.userId · Một user phát sinh nhiều lượt tìm.
R27	Tin Nhắn	Người Dùng	user 1 → N messages	chats.userId · Một user có nhiều message.
R28	Tin Nhắn	Sản Phẩm	product 1 → 0..N contextual messages	chats.productId · Message có thể hỏi về một product.
R29	Tương Tác Người Dùng	Người Dùng	user 1 → N events	interactions.userId · Một user tạo nhiều event hành vi.
R30	Tương Tác Người Dùng	Sản Phẩm	product 1 → N events	interactions.productId · Một product nhận nhiều event.
R31	Lịch Sử Thử Đồ	Sản Phẩm	product 1 → N try-ons	tryon_history.productId · Một product được thử nhiều lần.
R32	Lịch Sử Thử Đồ	Người Dùng	user 1 → N try-ons	tryon_history.userId · Một user thử nhiều product/lần.
R33	Phiếu Giảm Giá	Người Dùng	user 1 → N issued vouchers	vouchers.ownerId · Một staff/admin cấp nhiều voucher.
R34	Phiếu Giảm Giá	Người Dùng	user 1 → 0..N assigned vouchers	vouchers.userId · Voucher có thể cấp riêng một user.

Relationship Cheat Sheet R35–R50

ID	A (con)	B (cha)	Cardinality	FK · WHY
R35	Lượt Sử Dụng Phiếu	Phiếu Giảm Giá	voucher 1 → N redemptions	voucher_redemptions.voucherId · Một voucher có nhiều lượt dùng.
R36	Lượt Sử Dụng Phiếu	Người Dùng	user 1 → N redemptions	voucher_redemptions.userId · Một user có nhiều lượt dùng voucher.
R37	Lượt Sử Dụng Phiếu	Đơn Hàng	order 1 → 0..N redemptions	voucher_redemptions.orderId · Lượt dùng phải đối soát về order.
R38	Thành Viên VIP	Người Dùng	user 1 → N membership periods	vip_memberships.userId · User có nhiều kỳ VIP theo thời gian.
R39	Thành Viên VIP	Quy Tắc Giảm Giá	rule 1 → N memberships	vip_memberships.discountRuleId · Nhiều membership dùng cùng rule.
R40	Đánh Giá Sản Phẩm	Sản Phẩm	product 1 → N reviews	reviews.productId · Một product có nhiều review.
R41	Đánh Giá Sản Phẩm	Người Dùng	user 1 → N reviews	reviews.userId · Một user review nhiều product.
R42	Đánh Giá Sản Phẩm	Đơn Hàng	order 1 → 0..N reviews	reviews.orderId · Order hoàn tất chứng minh verified purchase.
R43	Tương Tác Đánh Giá	Đánh Giá Sản Phẩm	review 1 → N reactions	review_reactions.reviewId · Một review nhận nhiều phản ứng.
R44	Tương Tác Đánh Giá	Người Dùng	user 1 → N reactions	review_reactions.userId · Một user phản ứng nhiều review.
R45	Mẫu Kiểm Duyệt	Đánh Giá Sản Phẩm	review 1 → 0..N samples	moderation_samples.reviewId · Mẫu học truy về review nguồn.
R46	Đánh Giá Địa Điểm Nhật Bản	Người Dùng	user 1 → N spot reviews	japan_spot_reviews.userId · Một user đánh giá nhiều địa điểm.
R47	Hình Ảnh Giao Diện Sản Phẩm	Sản Phẩm	product 1 → N media	product_media.productId · Một product có nhiều ảnh/video.
R48	Thẻ Địa Danh	Sản Phẩm	flagcard N ↔ N products (array)	flagcards.recommendedProductIds[] · Một card gợi ý nhiều product và ngược lại.
R49	Bộ Sưu Tập Thẻ Của	Người Dùng	user 1 → 0..N	flagcard_collections.userId · Collection ghi tiến

	Người Dùng		collections	trình/award của user.
R50	Bộ Sưu Tầm Thẻ Của Người Dùng	Thẻ Địa Danh	collection N ↔ N flagcards (array)	flagcard_collections.cardIds[] · Một collection chứa nhiều card; card thuộc nhiều user.

21.3. SHEET 3 — TOP 25 QUESTIONS

Top 25 câu phải trả lời không cần nghĩ

#	Câu	Đáp án chốt
1	Email unique ở đâu?	App check có; Mongo index hiện không unique .
2	Hai register cùng email?	Một process serialize; nhiều instance cần UNIQUE DB.
3	Email làm PK được không?	Không nên: đổi được; id giữ 21 refs ổn định.
4	Stock ở đâu?	product_variants.stock, vì màu/cỡ có tồn khác nhau.
5	Hai khách mua món cuối?	Test một process đặt; scale cần conditional decrement.
6	Giá hai nơi có sai 3NF?	Không: current price và historical transaction price.
7	Product bị xoá?	Snapshot giữ order; Mongo integrity hiện tạo hiệu ứng RESTRICT.
8	Vì sao order_items?	N:N + qty + historical snapshot + query/index.
9	Mongo embed được mà?	Được; project tách physical rồi hydrate array cho runtime.
10	94/94 payment là 1:1?	Current data 1:1; design 1:N vì orderId non-unique.
11	Webhook lặp?	Status/txn/refund-id/restock markers; transactionCode DB unique còn thiếu.
12	Ai là truth về tiền?	Provider; DB là local projection/audit.
13	Có lưu CVV?	Không; Stripe thu thập, DB chỉ last4/brand/provider IDs.
14	Review có verified purchase?	Có: completed successful order chứa đúng product.
15	Review duplicate?	App recheck; DB composite unique còn thiếu.
16	50 dây có phải FK constraint?	Không; logical FK, application enforce một phần.
17	Fallback JSON có giống Mongo?	Không; writeFile bỏ qua assertValid.
18	Có Mongo transaction?	Không có startSession/withTransaction.
19	40 ms mất một write?	Không thể kết luận; debounce reset, nhiều write có thể chưa flush.
20	Một triệu user gây ở đâu?	Whole state RAM + clone mỗi read + snapshot persist.
21	Index càng nhiều càng tốt?	Không; query nhanh đổi lấy write/RAM/disk.
22	3NF luôn tốt?	Không; snapshot/embedding có lý do vật lý rõ.
23	Một user nhiều role?	roles + user_roles N:N, migrate role hiện tại.
24	Một product nhiều category?	product_categories bridge, composite unique.
25	Một order nhiều shipment?	shipments + shipment_items, aggregate status.

21.4. SHEET 4 — PROJECT WEAKNESSES

Mười điểm yếu không nên cố cải

Mã	Hiện trạng thật	Tình huống lộ lỗi	Cách nói/cách sửa
Y-01	Email index non-unique.	Hai instance cùng register.	Thừa nhận; UNIQUE email chuẩn hoá.
Y-02	Review thiếu UNIQUE user+product.	Hai instance cùng review.	Thừa nhận; composite unique.
Y-03	Hai entity ảnh trùng trên ERD.	Hội đồng so hai bảng.	Gộp vào product_media.
Y-04	JSON fallback bỏ assertValid.	Mongo down rồi ghi orphan.	Validate trước mọi storage adapter.
Y-05	relationshipErrors phủ chưa hết.	Orphan telemetry/redemption.	Schema registry/validators đầy đủ.
Y-06	Pending payment có thể giữ kho.	Mất callback, không reconcile.	Reservation timeout + worker.
Y-07	Order state rules phân tán.	Nhảy trạng thái sai.	State machine tập trung.
Y-08	Chưa có anonymization.	User yêu cầu xoá PII.	Retention + anonymize snapshots.
Y-09	Không có DB transaction.	Invariant đa collection/multi-instance.	Atomic ops/transaction theo phạm vi.
Y-10	Ack trước durable persist.	Process chết khi write chưa flush.	Write-through/outbox; không nói mất tối đa 1 write.

21.5. SHEET 5 — CÂU CỨU NGUY

KHI CHƯA XÁC MINH

"Chỗ này em chưa kiểm chứng nên không dám khẳng định. Em sẽ đối chiếu field/index/route ...; với evidence hiện có em chỉ kết luận tới mức ..."

KHI BỊ BẮT ĐÚNG LỖI

"Hiện tại rule nằm ở application. Trong một process nó che được tình huống này, nhưng production nhiều instance thì chưa đủ. Lỗi đúng là ... ở database."

KHI REQUIREMENT ĐỔI

"Current design đáp ứng quan hệ ... Nếu đổi sang ... em sẽ thêm entity ..., PK ..., FK ..., cardinality ...; migration sẽ đi theo hai bước ..."

KHI BỊ HỎI PRODUCTION SCALE

"Em không suy rộng unit test một process thành bảo đảm phân tán. Khi scale, invariant này phải chuyển xuống atomic update/unique constraint/transaction ở database."

KHI KHÔNG ĐỒNG Ý VỚI TIỀN ĐỀ CÂU HỎI

"Dữ liệu mẫu đang 1:1, nhưng constraint cho phép 1:N. Em xin tách current data, allowed design và business rule trước khi kết luận."